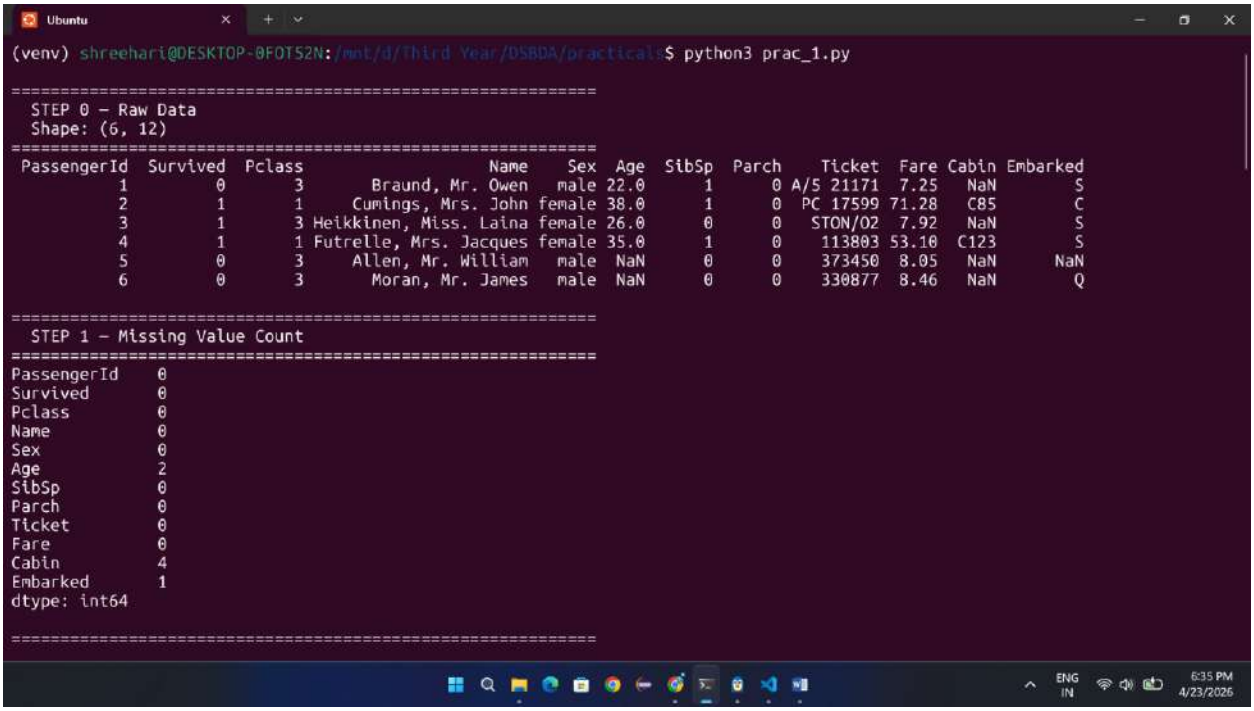
PRACTICAL 1

- Source Code

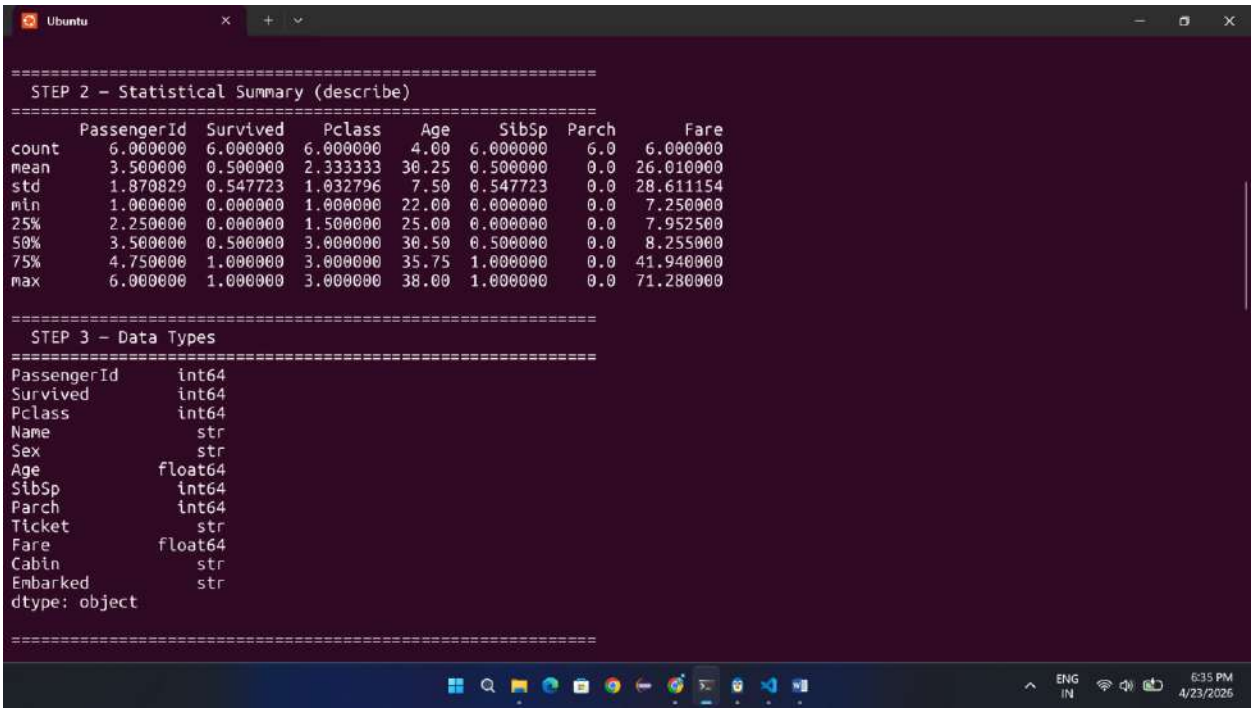
```
import pandas as pd
import numpy as np

df = pd.read_csv("prac1_sample.csv")

DIVIDER = "=" * 60
print(f"\n{DIVIDER}")
print("  STEP 0 — Raw Data")
print(f"  Shape: {df.shape}")
print(DIVIDER)
print(df.to_string(index=False))

print(f"\n{DIVIDER}")
print("  STEP 1 — Missing Value Count")
print(DIVIDER)
print(df.isnull().sum())

print(f"\n{DIVIDER}")
print("  STEP 2 — Statistical Summary (describe)")
print(DIVIDER)
print(df.describe())

print(f"\n{DIVIDER}")
print("  STEP 3 — Data Types")
print(DIVIDER)
print(df.dtypes)

df = df.drop(['Cabin', 'PassengerId', 'Name', 'Ticket'], axis=1)

print(f"\n{DIVIDER}")
print("  STEP 4 — After Dropping Cabin, PassengerId, Name, Ticket")
print(f"  Shape: {df.shape}")
print(DIVIDER)
print(df.to_string(index=False))

df['Age'] = df['Age'].fillna(df['Age'].mean())
df['Embarked'] = df['Embarked'].fillna(df['Embarked'].mode()[0])

print(f"\n{DIVIDER}")
```

```
print("  STEP 5 — After Filling Missing Values")
print("  Age filled with mean | Embarked filled with mode")
print(DIVIDER)
print(df.to_string(index=False))

df['Fare'] = (df['Fare'] - df['Fare'].min()) / (df['Fare'].max() - df['Fare'].min())

print(f"\n{DIVIDER}")
print("  STEP 6 — After Normalizing Fare column (min-max)")
print(DIVIDER)
print(df.to_string(index=False))

df['Age'] = df['Age'].astype(float)

print(f"\n{DIVIDER}")
print("  STEP 7 — Age converted to float")
print(DIVIDER)
print(df.dtypes)

df['Sex'] = df['Sex'].map({'male': 0, 'female': 1})

print(f"\n{DIVIDER}")
print("  STEP 8 — After Label Encoding: Sex (male=0, female=1)")
print(DIVIDER)
print(df.to_string(index=False))

df = pd.get_dummies(df, columns=['Embarked'])

print(f"\n{DIVIDER}")
print("  STEP 9 — FINAL: After One-Hot Encoding Embarked column")
print(f"  Shape: {df.shape}")
print(DIVIDER)
print(df.to_string(index=False))

print(f"\n{'='*60}")
print("  PRACTICAL 1 COMPLETE")
print(f"{'='*60}\n")
```

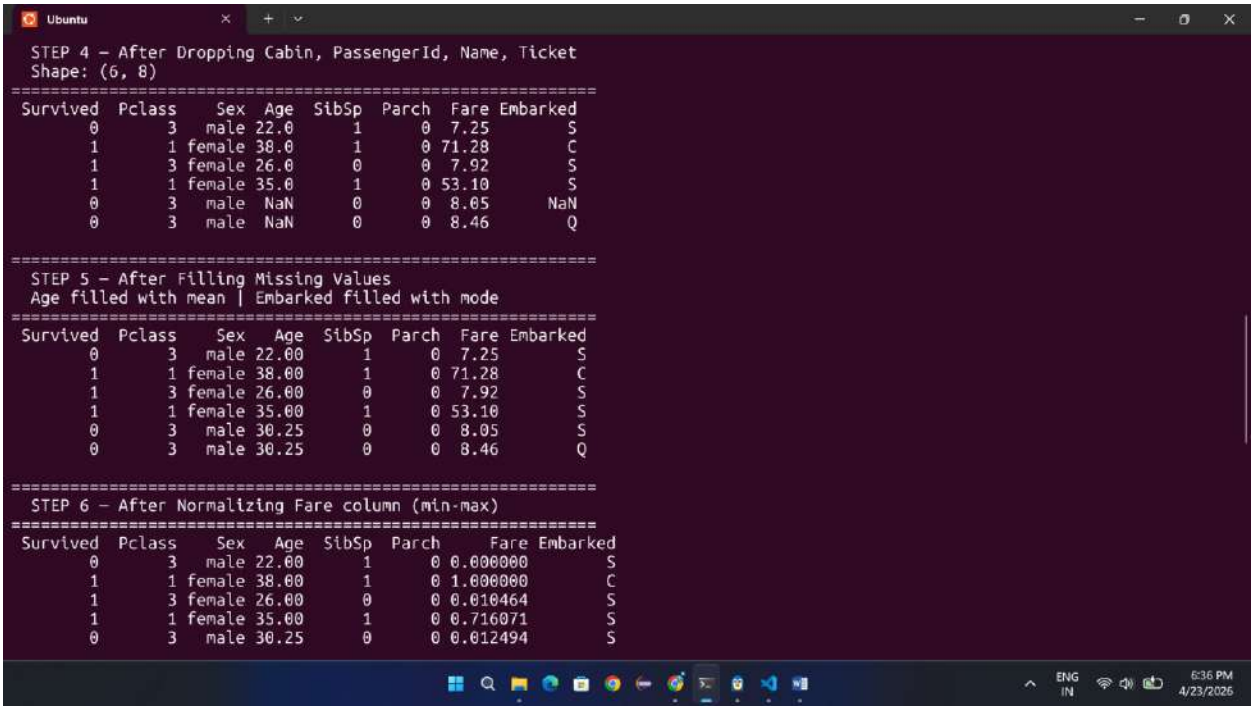
➢ Output

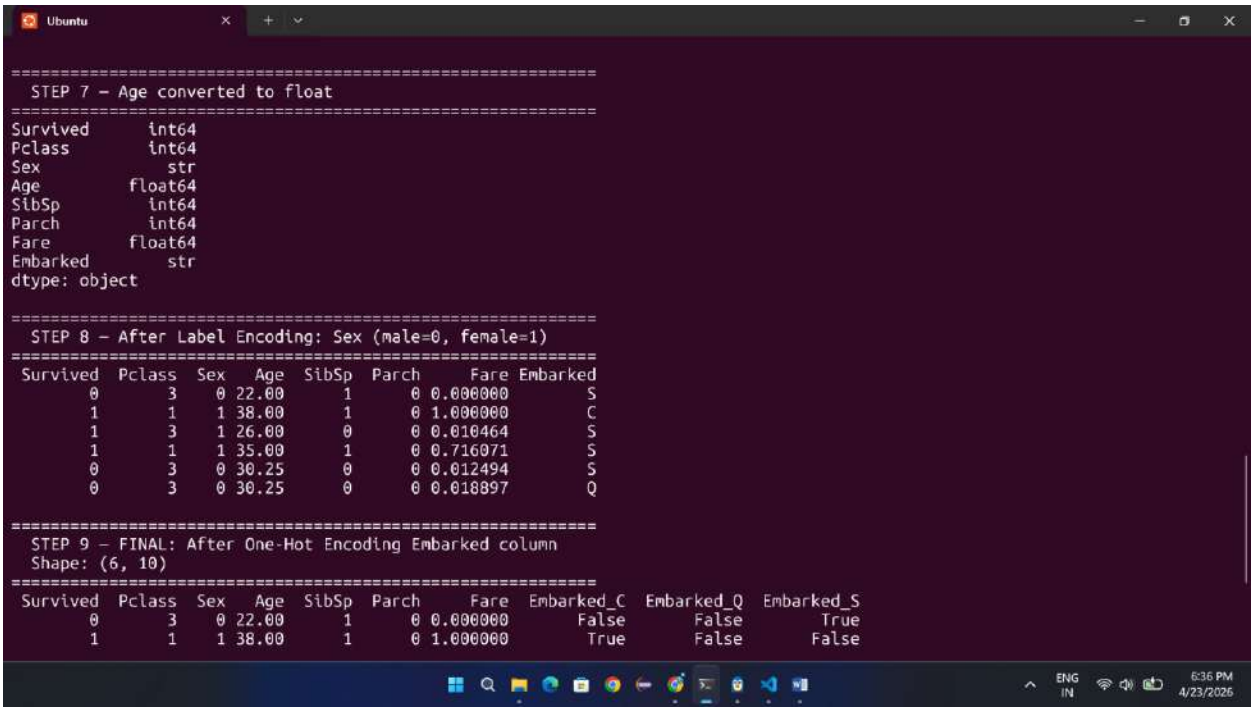


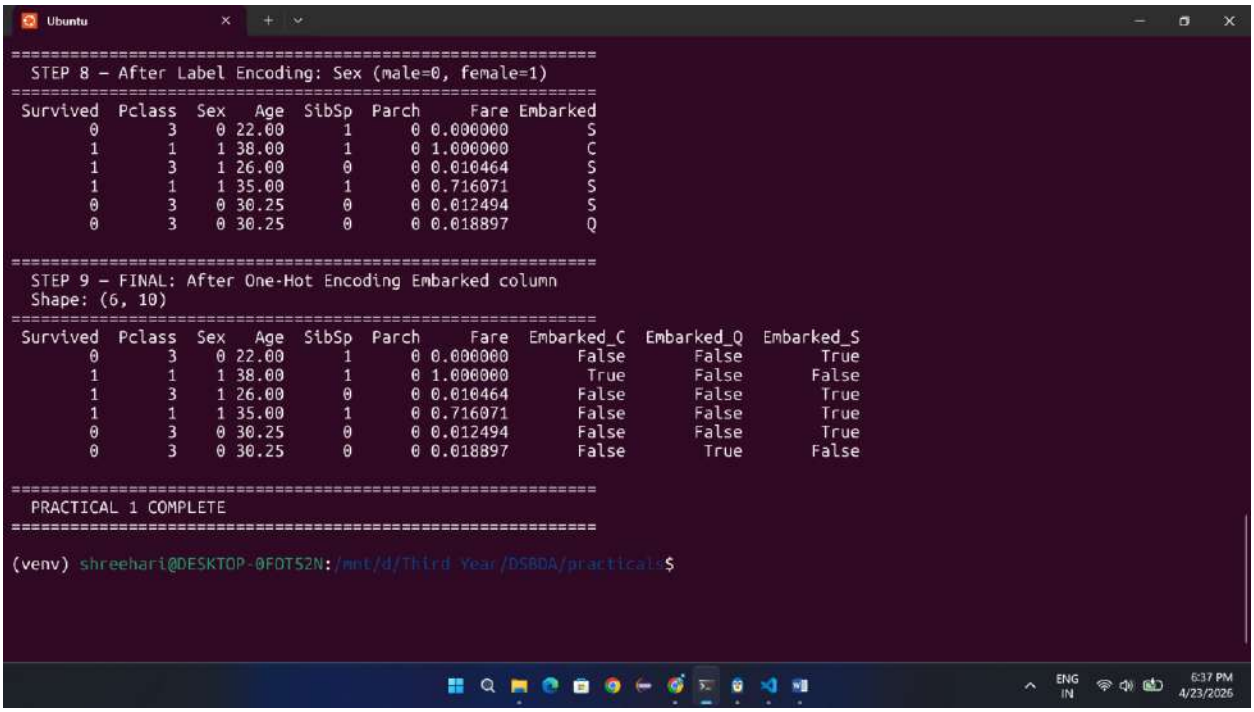
```
Ubuntu
(venv) shreehari@DESKTOP-0FOT52N:/mnt/d/Third Year/DSBDA/practicals$ python3 prac_1.py

============================================================
  STEP 0 — Raw Data
  Shape: (6, 12)
============================================================
 PassengerId  Survived  Pclass                Name     Sex  Age  SibSp  Parch    Ticket  Fare Cabin Embarked
           1         0       3    Braund, Mr. Owen    male 22.0      1      0 A/5 21171  7.25   NaN        S
           2         1       1  Cumings, Mrs. John female 38.0      1      0  PC 17599 71.28   C85        C
           3         1       3 Heikkinen, Miss. Laina female 26.0      0      0  STON/O2  7.92   NaN        S
           4         1       1 Futrelle, Mrs. Jacques female 35.0      1      0    113803 53.10  C123        S
           5         0       3    Allen, Mr. William    male  NaN      0      0    373450  8.05   NaN      NaN
           6         0       3      Moran, Mr. James    male  NaN      0      0    330877  8.46   NaN        Q

============================================================
  STEP 1 — Missing Value Count
============================================================
PassengerId    0
Survived       0
Pclass         0
Name           0
Sex            0
Age            2
SibSp          0
Parch          0
Ticket         0
Fare           0
Cabin          4
Embarked       1
dtype: int64

============================================================
```



```
Ubuntu

============================================================
  STEP 2 — Statistical Summary (describe)
============================================================
       PassengerId  Survived    Pclass     Age     SibSp  Parch       Fare
count     6.000000  6.000000  6.000000    4.00  6.000000    6.0   6.000000
mean      3.500000  0.500000  2.333333   30.25  0.500000    0.0  26.010000
std       1.870829  0.547723  1.032796    7.50  0.547723    0.0  28.611154
min       1.000000  0.000000  1.000000   22.00  0.000000    0.0   7.250000
25%       2.250000  0.000000  1.500000   25.00  0.000000    0.0   7.952500
50%       3.500000  0.500000  3.000000   30.50  0.500000    0.0   8.255000
75%       4.750000  1.000000  3.000000   35.75  1.000000    0.0  41.940000
max       6.000000  1.000000  3.000000   38.00  1.000000    0.0  71.280000

============================================================
  STEP 3 — Data Types
============================================================
PassengerId      int64
Survived         int64
Pclass           int64
Name               str
Sex                str
Age              float64
SibSp            int64
Parch            int64
Ticket             str
Fare             float64
Cabin              str
Embarked           str
dtype: object

============================================================
```


Ubuntu
STEP 4 – After Dropping Cabin, PassengerId, Name, Ticket
Shape: (6, 8)
Survived Pclass Sex Age SibSp Parch Fare Embarked
0 3 male 22.0 1 0 7.25 S
1 1 female 38.0 1 0 71.28 C
1 3 female 26.0 0 0 7.92 S
1 1 female 35.0 1 0 53.10 S
0 3 male NaN 0 0 8.05 NaN
0 3 male NaN 0 0 8.46 Q
STEP 5 – After Filling Missing Values
Age filled with mean | Embarked filled with mode
Survived Pclass Sex Age SibSp Parch Fare Embarked
0 3 male 22.00 1 0 7.25 S
1 1 female 38.00 1 0 71.28 C
1 3 female 26.00 0 0 7.92 S
1 1 female 35.00 1 0 53.10 S
0 3 male 30.25 0 0 8.05 S
0 3 male 30.25 0 0 8.46 Q
STEP 6 – After Normalizing Fare column (min-max)
Survived Pclass Sex Age SibSp Parch Fare Embarked
0 3 male 22.00 1 0 0.000000 S
1 1 female 38.00 1 0 1.000000 C
1 3 female 26.00 0 0 0.010464 S
1 1 female 35.00 1 0 0.716071 S
0 3 male 30.25 0 0 0.012494 S
ENG IN
6:36 PM 4/23/2026


Ubuntu
STEP 7 – Age converted to float
Survived int64
Pclass int64
Sex str
Age float64
SibSp int64
Parch int64
Fare float64
Embarked str
dtype: object
STEP 8 – After Label Encoding: Sex (male=0, female=1)
Survived Pclass Sex Age SibSp Parch Fare Embarked
0 3 0 22.00 1 0 0.000000 S
1 1 1 38.00 1 0 1.000000 C
1 3 1 26.00 0 0 0.010464 S
1 1 1 35.00 1 0 0.716071 S
0 3 0 30.25 0 0 0.012494 S
0 3 0 30.25 0 0 0.018897 Q
STEP 9 – FINAL: After One-Hot Encoding Embarked column
Shape: (6, 10)
Survived Pclass Sex Age SibSp Parch Fare Embarked_C Embarked_Q Embarked_S
0 3 0 22.00 1 0 0.000000 False False True
1 1 1 38.00 1 0 1.000000 True False False
ENG IN
6:36 PM 4/23/2026


Ubuntu
STEP 8 – After Label Encoding: Sex (male=0, female=1)
Survived Pclass Sex Age SibSp Parch Fare Embarked
0 3 0 22.00 1 0 0.000000 S
1 1 1 38.00 1 0 1.000000 C
1 3 1 26.00 0 0 0.010464 S
1 1 1 35.00 1 0 0.716071 S
0 3 0 30.25 0 0 0.012494 S
0 3 0 30.25 0 0 0.018897 Q
STEP 9 – FINAL: After One-Hot Encoding Embarked column
Shape: (6, 10)
Survived Pclass Sex Age SibSp Parch Fare Embarked_C Embarked_Q Embarked_S
0 3 0 22.00 1 0 0.000000 False False True
1 1 1 38.00 1 0 1.000000 True False False
1 3 1 26.00 0 0 0.010464 False False True
1 1 1 35.00 1 0 0.716071 False False True
0 3 0 30.25 0 0 0.012494 False False True
0 3 0 30.25 0 0 0.018897 False True False
PRACTICAL 1 COMPLETE
(venv) shreehari@DESKTOP-0FOT52N:/mnt/d/Third Year/DSBDA/practicals$
ENG
IN
6:37 PM
4/23/2026

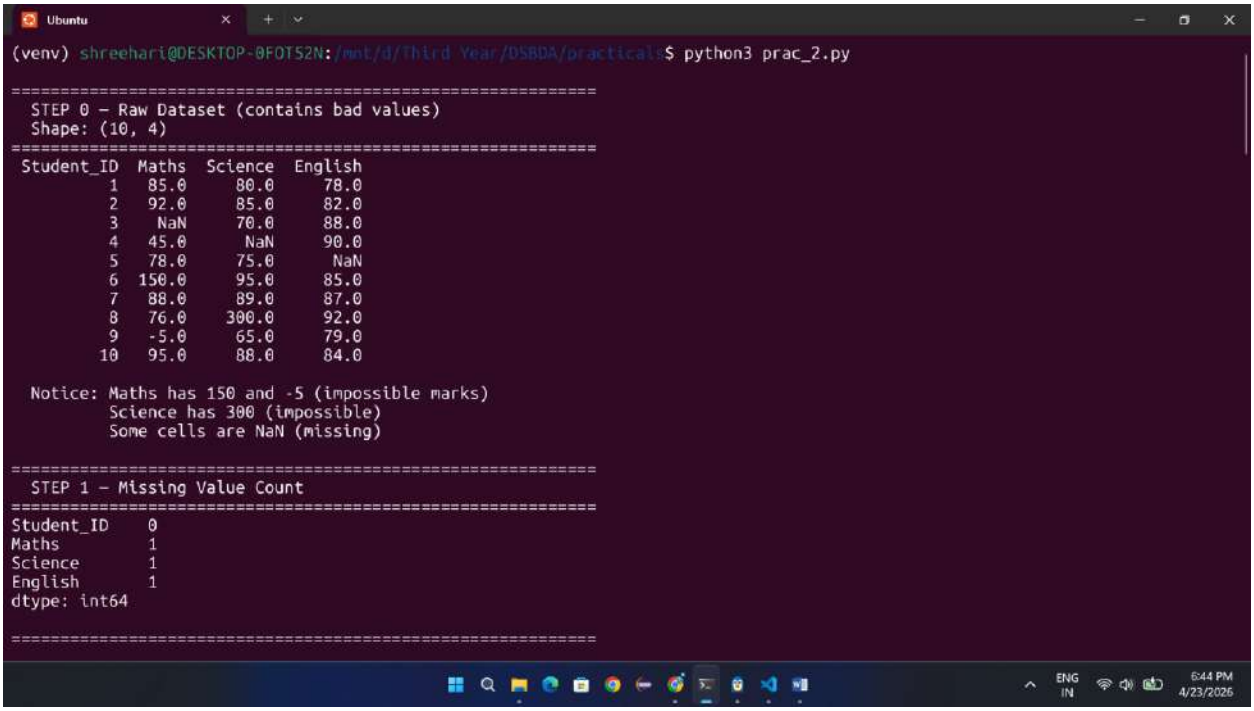
PRACTICAL 2

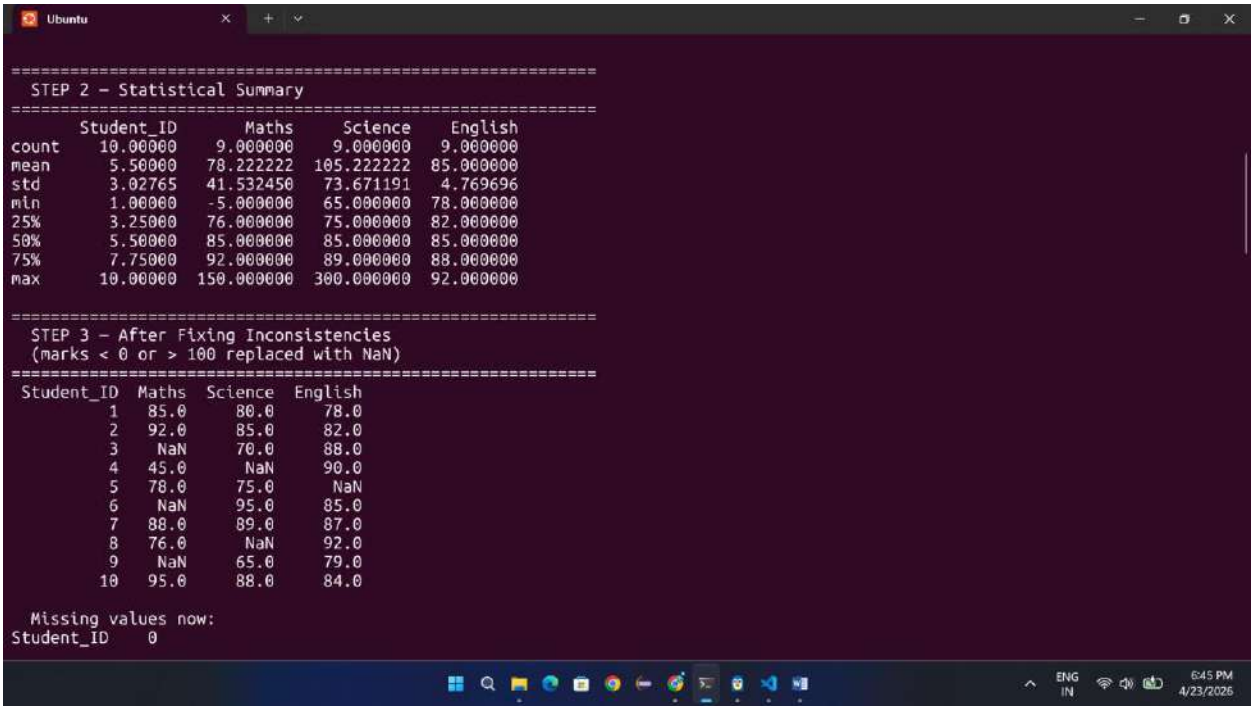
- Source Code

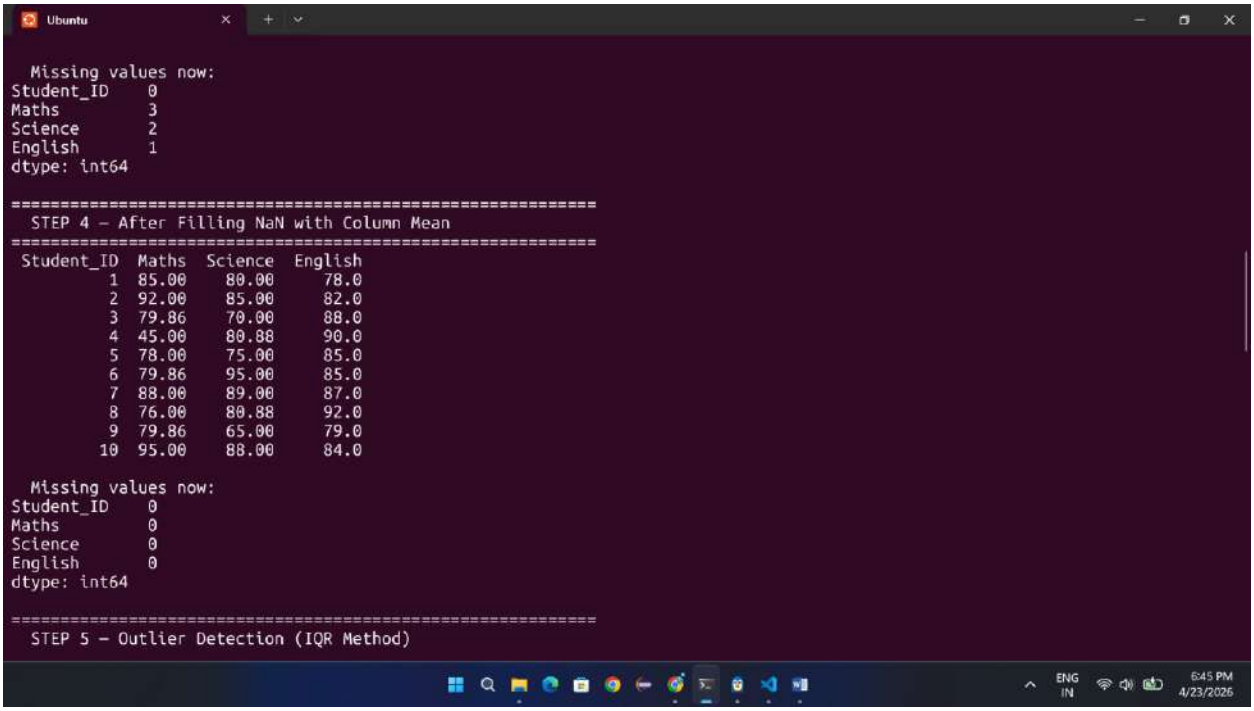
```
import pandas as pd
import numpy as np
DIVIDER = "=" * 60
df = pd.read_csv("prac2_samples.csv")
print(f"\n{DIVIDER}")
print("  STEP 0 — Raw Dataset (contains bad values)")
print(f"  Shape: {df.shape}")
print(DIVIDER)
print(df.to_string(index=False))
print("\n  Notice: Maths has 150 and -5 (impossible marks)")
print("          Science has 300 (impossible)")
print("          Some cells are NaN (missing)")

print(f"\n{DIVIDER}")
print("  STEP 1 — Missing Value Count")
print(DIVIDER)
print(df.isnull().sum())

print(f"\n{DIVIDER}")
print("  STEP 2 — Statistical Summary")
print(DIVIDER)
print(df.describe())
df['Maths']   = df['Maths'].apply(lambda x: np.nan if pd.notna(x) and (x < 0 or x > 100) else x)
df['Science'] = df['Science'].apply(lambda x: np.nan if pd.notna(x) and x > 100 else x)

print(f"\n{DIVIDER}")
print("  STEP 3 — After Fixing Inconsistencies")
print("  (marks < 0 or > 100 replaced with NaN)")
print(DIVIDER)
print(df.to_string(index=False))
print(f"\n  Missing values now:\n{df.isnull().sum()}")

df.fillna(df.mean(numeric_only=True), inplace=True)

print(f"\n{DIVIDER}")
print("  STEP 4 — After Filling NaN with Column Mean")
print(DIVIDER)
print(df.round(2).to_string(index=False))
print(f"\n  Missing values now:\n{df.isnull().sum()}")
```

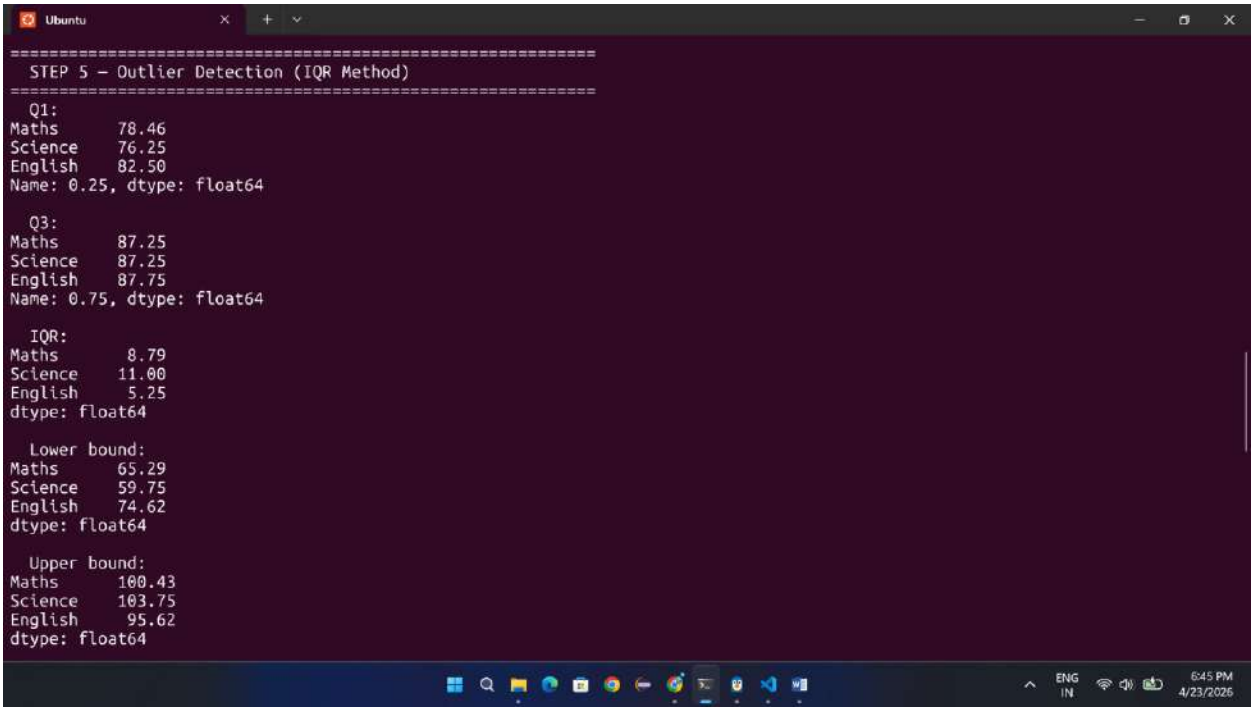
```
cols = ['Maths', 'Science', 'English']
Q1  = df[cols].quantile(0.25)
Q3  = df[cols].quantile(0.75)
IQR = Q3 - Q1

print(f"\n{DIVIDER}")
print("  STEP 5 — Outlier Detection (IQR Method)")
print(DIVIDER)
print(f"  Q1:\n{Q1.round(2)}\n")
print(f"  Q3:\n{Q3.round(2)}\n")
print(f"  IQR:\n{IQR.round(2)}\n")

lower = Q1 - 1.5 * IQR
upper = Q3 + 1.5 * IQR
print(f"  Lower bound:\n{lower.round(2)}\n")
print(f"  Upper bound:\n{upper.round(2)}")
outlier_mask = ((df[cols] < lower) | (df[cols] > upper)).any(axis=1)
print(f"\n  Rows identified as outliers:\n{df[outlier_mask].round(2)}")
df_clean = df[~outlier_mask].copy()
print(f"\n  Shape before removing outliers: {df.shape}")
print(f"  Shape after  removing outliers: {df_clean.shape}")
df_clean['Maths_norm'] = (
   (df_clean['Maths'] - df_clean['Maths'].min()) /
   (df_clean['Maths'].max() - df_clean['Maths'].min())
)
print(f"\n{DIVIDER}")
print("  STEP 6 — After Normalizing Maths column (0 to 1 scale)")
print(DIVIDER)
print(df_clean[['Student_ID', 'Maths', 'Maths_norm']].round(4).to_string(index=False))
df_clean['Science_log'] = np.log(df_clean['Science'])
print(f"\n{DIVIDER}")
print("  STEP 7 — After Log Transformation on Science column")
print("  (reduces skewness, good for right-skewed data)")
print(DIVIDER)
print(df_clean[['Student_ID', 'Science', 'Science_log']].round(4).to_string(index=False))
print(f"\n{DIVIDER}")
print("  FINAL — Complete Cleaned & Transformed Dataset")
print(f"  Shape: {df_clean.shape}")
print(DIVIDER)
print(df_clean.round(4).to_string(index=False))

print(f"\n{'='*60}")
print("  PRACTICAL 2 COMPLETE")
print(f"{'='*60}\n")
```

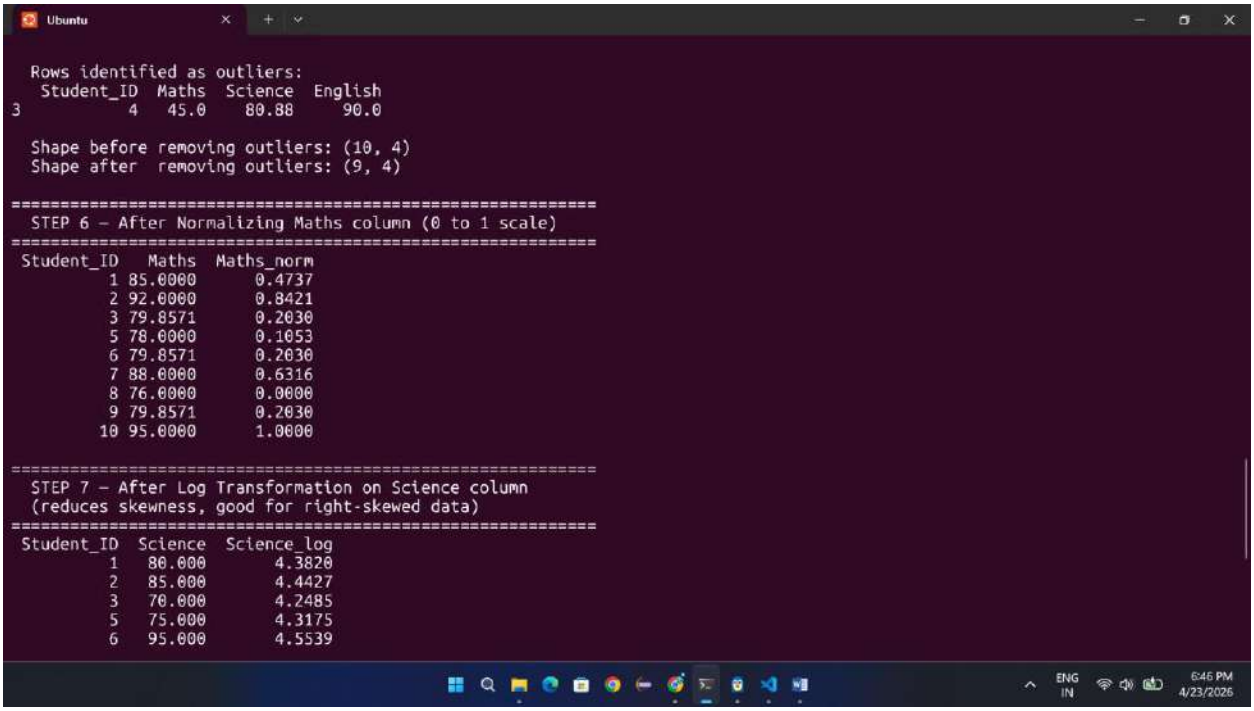
➢ Output

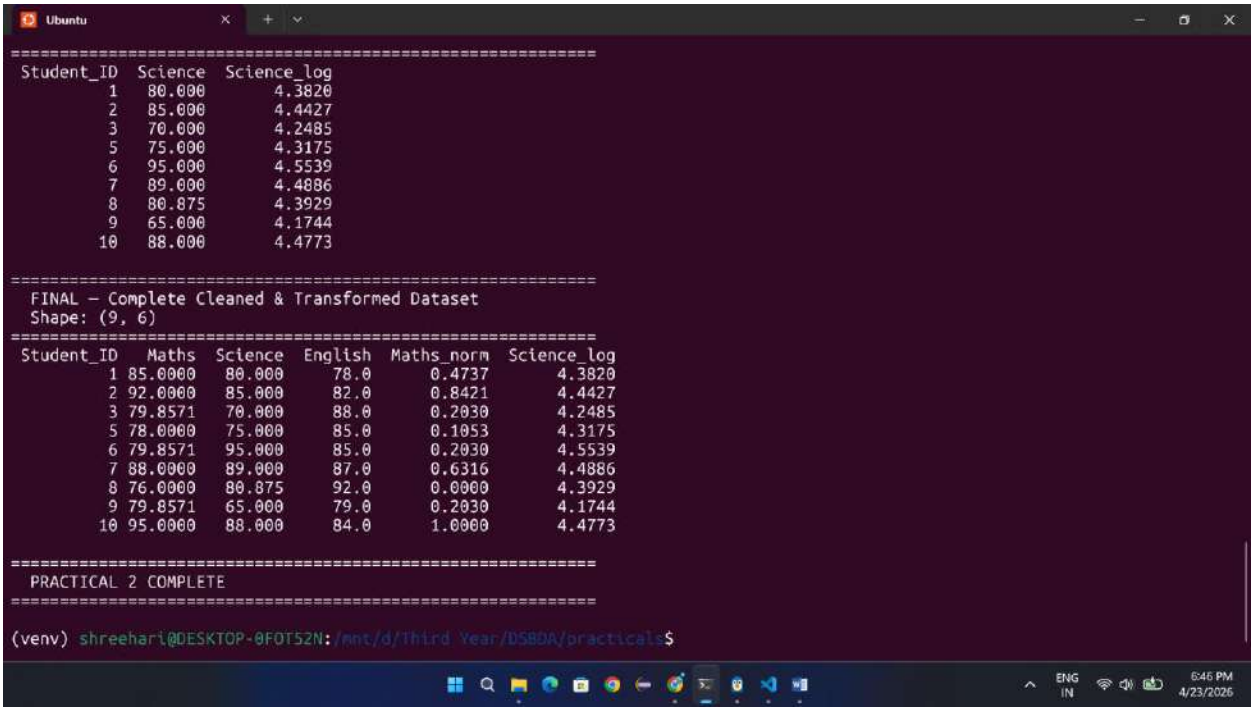

Ubuntu
(venv) shreehari@DESKTOP-0FOT52N:/mnt/d/Third Year/DSBDA/practicals$ python3 prac_2.py
STEP 0 – Raw Dataset (contains bad values)
Shape: (10, 4)
Student_ID Maths Science English
1 85.0 80.0 78.0
2 92.0 85.0 82.0
3 NaN 70.0 88.0
4 45.0 NaN 90.0
5 78.0 75.0 NaN
6 150.0 95.0 85.0
7 88.0 89.0 87.0
8 76.0 300.0 92.0
9 -5.0 65.0 79.0
10 95.0 88.0 84.0
Notice: Maths has 150 and -5 (impossible marks)
Science has 300 (impossible)
Some cells are NaN (missing)
STEP 1 – Missing Value Count
Student_ID 0
Maths 1
Science 1
English 1
dtype: int64
6:44 PM
4/23/2026


Ubuntu
STEP 2 – Statistical Summary
Student_ID Maths Science English
count 10.00000 9.000000 9.000000 9.000000
mean 5.50000 78.222222 105.222222 85.000000
std 3.02765 41.532450 73.671191 4.769696
min 1.00000 -5.000000 65.000000 78.000000
25% 3.25000 76.000000 75.000000 82.000000
50% 5.50000 85.000000 85.000000 85.000000
75% 7.75000 92.000000 89.000000 88.000000
max 10.00000 150.000000 300.000000 92.000000
STEP 3 – After Fixing Inconsistencies
(marks < 0 or > 100 replaced with NaN)
Student_ID Maths Science English
1 85.0 80.0 78.0
2 92.0 85.0 82.0
3 NaN 70.0 88.0
4 45.0 NaN 90.0
5 78.0 75.0 NaN
6 NaN 95.0 85.0
7 88.0 89.0 87.0
8 76.0 NaN 92.0
9 NaN 65.0 79.0
10 95.0 88.0 84.0
Missing values now:
Student_ID 0
6:45 PM
4/23/2026


Ubuntu
Missing values now:
Student_ID 0
Maths 3
Science 2
English 1
dtype: int64
STEP 4 – After Filling NaN with Column Mean
Student_ID Maths Science English
1 85.00 80.00 78.0
2 92.00 85.00 82.0
3 79.86 70.00 88.0
4 45.00 80.88 90.0
5 78.00 75.00 85.0
6 79.86 95.00 85.0
7 88.00 89.00 87.0
8 76.00 80.88 92.0
9 79.86 65.00 79.0
10 95.00 88.00 84.0
Missing values now:
Student_ID 0
Maths 0
Science 0
English 0
dtype: int64
STEP 5 – Outlier Detection (IQR Method)
ENG
IN
6:45 PM
4/23/2026


Ubuntu
STEP 5 – Outlier Detection (IQR Method)
Q1:
Maths 78.46
Science 76.25
English 82.50
Name: 0.25, dtype: float64
Q3:
Maths 87.25
Science 87.25
English 87.75
Name: 0.75, dtype: float64
IQR:
Maths 8.79
Science 11.00
English 5.25
dtype: float64
Lower bound:
Maths 65.29
Science 59.75
English 74.62
dtype: float64
Upper bound:
Maths 100.43
Science 103.75
English 95.62
dtype: float64
ENG
IN
6:45 PM
4/23/2026


Ubuntu
Rows identified as outliers:
Student_ID Maths Science English
3 4 45.0 80.88 90.0
Shape before removing outliers: (10, 4)
Shape after removing outliers: (9, 4)
STEP 6 — After Normalizing Maths column (0 to 1 scale)
Student_ID Maths Maths_norm
1 85.0000 0.4737
2 92.0000 0.8421
3 79.8571 0.2030
5 78.0000 0.1053
6 79.8571 0.2030
7 88.0000 0.6316
8 76.0000 0.0000
9 79.8571 0.2030
10 95.0000 1.0000
STEP 7 — After Log Transformation on Science column
(reduces skewness, good for right-skewed data)
Student_ID Science Science_log
1 80.000 4.3820
2 85.000 4.4427
3 70.000 4.2485
5 75.000 4.3175
6 95.000 4.5539
ENG
IN
6:46 PM
4/23/2026


Ubuntu
Student_ID Science Science_log
1 80.000 4.3820
2 85.000 4.4427
3 70.000 4.2485
5 75.000 4.3175
6 95.000 4.5539
7 89.000 4.4886
8 80.875 4.3929
9 65.000 4.1744
10 88.000 4.4773
FINAL — Complete Cleaned & Transformed Dataset
Shape: (9, 6)
Student_ID Maths Science English Maths_norm Science_log
1 85.0000 80.000 78.0 0.4737 4.3820
2 92.0000 85.000 82.0 0.8421 4.4427
3 79.8571 70.000 88.0 0.2030 4.2485
5 78.0000 75.000 85.0 0.1053 4.3175
6 79.8571 95.000 85.0 0.2030 4.5539
7 88.0000 89.000 87.0 0.6316 4.4886
8 76.0000 80.875 92.0 0.0000 4.3929
9 79.8571 65.000 79.0 0.2030 4.1744
10 95.0000 88.000 84.0 1.0000 4.4773
PRACTICAL 2 COMPLETE
(venv) shreehari@DESKTOP-0FOT52N:/mnt/d/Third Year/DSBDA/practicals$
ENG
IN
6:46 PM
4/23/2026

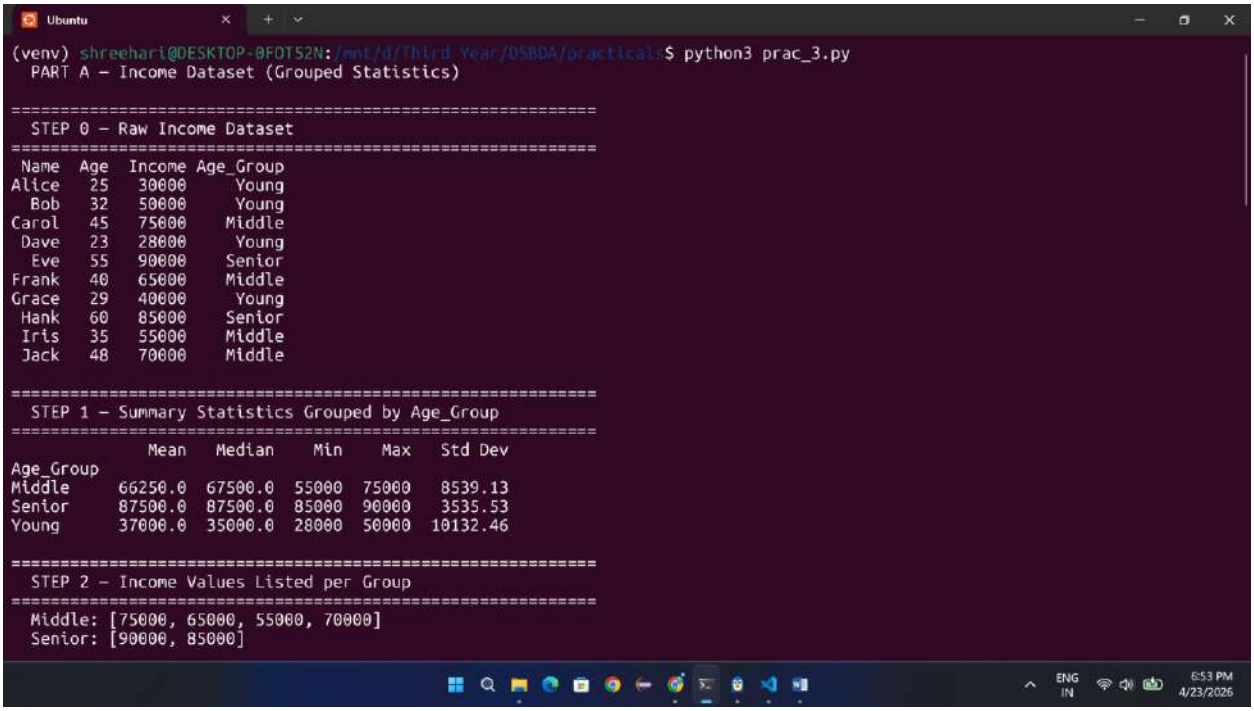
PRACTICAL 3

- Source Code

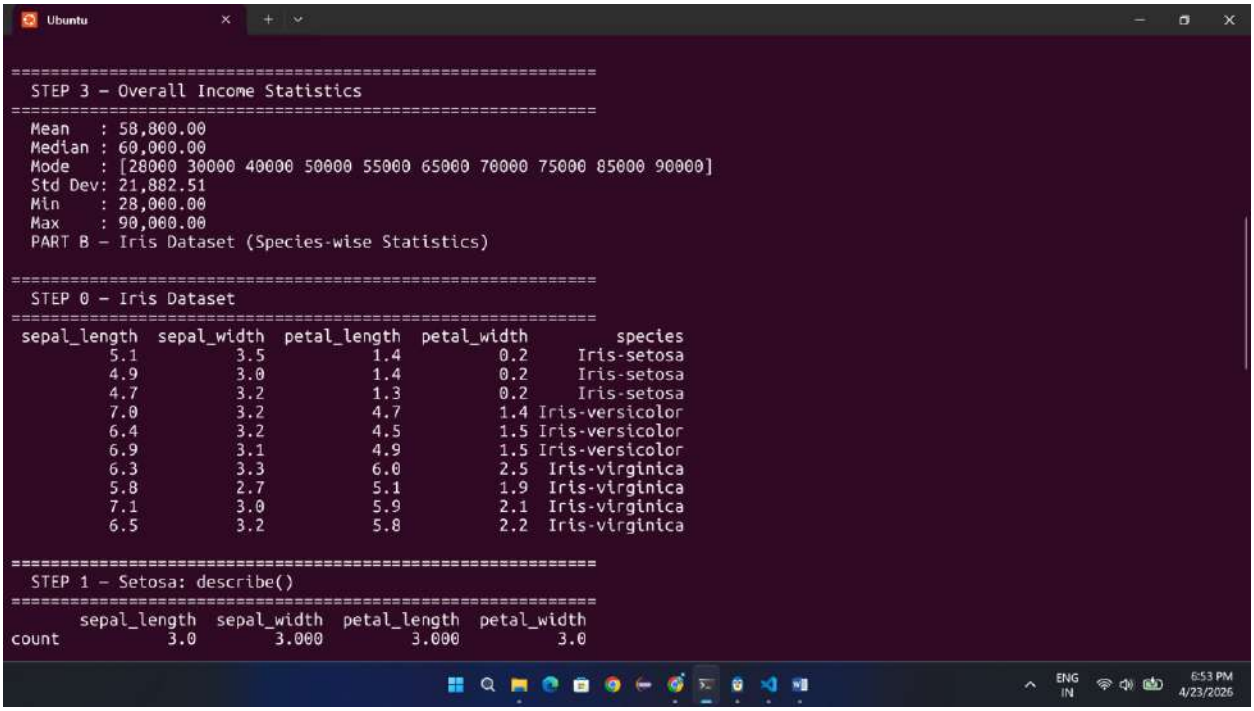
```
import pandas as pd
import numpy as np
DIVIDER = "=" * 60
print(f"\n{'#'*60}")
print("  PART A — Income Dataset (Grouped Statistics)")
print(f"{'#'*60}")
df = pd.read_csv("income_data.csv")

print(f"\n{DIVIDER}")
print("  STEP 0 — Raw Income Dataset")
print(DIVIDER)
print(df.to_string(index=False))

print(f"\n{DIVIDER}")
print("  STEP 1 — Summary Statistics Grouped by Age_Group")
print(DIVIDER)
summary = df.groupby('Age_Group')['Income'].agg(['mean', 'median', 'min', 'max', 'std'])
summary.columns = ['Mean', 'Median', 'Min', 'Max', 'Std Dev']
print(summary.round(2))

print(f"\n{DIVIDER}")
print("  STEP 2 — Income Values Listed per Group")
print(DIVIDER)
grouped_list = df.groupby('Age_Group')['Income'].apply(list)
for group, values in grouped_list.items():
   print(f"  {group}: {values}")
print(f"\n{DIVIDER}")
print("  STEP 3 — Overall Income Statistics")
print(DIVIDER)
print(f"  Mean   : {df['Income'].mean():,.2f}")
print(f"  Median : {df['Income'].median():,.2f}")
print(f"  Mode   : {df['Income'].mode().values}")
print(f"  Std Dev: {df['Income'].std():,.2f}")
print(f"  Min    : {df['Income'].min():,.2f}")
print(f"  Max    : {df['Income'].max():,.2f}")

print(f"\n\n{'#'*60}")
```

```
print("  PART B — Iris Dataset (Species-wise Statistics)")
print(f"{'#'*60}")
iris = pd.read_csv("iris_data.csv")

print(f"\n{DIVIDER}")
print("  STEP 0 — Iris Dataset")
print(DIVIDER)
print(iris.to_string(index=False))

setosa     = iris[iris['species'] == 'Iris-setosa']
versicolor = iris[iris['species'] == 'Iris-versicolor']
virginica  = iris[iris['species'] == 'Iris-virginica']
print(f"\n{DIVIDER}")
print("  STEP 1 — Setosa: describe()")
print(DIVIDER)
print(setosa.describe().round(3))
print(f"\n{DIVIDER}")
print("  STEP 2 — Versicolor: describe()")
print(DIVIDER)
print(versicolor.describe().round(3))

print(f"\n{DIVIDER}")
print("  STEP 3 — Virginica: describe()")
print(DIVIDER)
print(virginica.describe().round(3))

print(f"\n{DIVIDER}")
print("  STEP 4 — Percentiles for Sepal Length (all species)")
print(DIVIDER)
for name, grp in [('Setosa', setosa), ('Versicolor', versicolor), ('Virginica', virginica)]:
    p = np.percentile(grp['sepal_length'], [25, 50, 75])
    print(f"  {name:12s} → 25th: {p[0]:.2f}  |  50th (Median): {p[1]:.2f}  |  75th: {p[2]:.2f}")
print(f"\n{DIVIDER}")
print("  STEP 5 — Mean of All Features by Species")
print(DIVIDER)
print(iris.groupby('species').mean().round(3))

print(f"\n{'='*60}")
print("  PRACTICAL 3 COMPLETE")
print(f"{'='*60}\n")
```

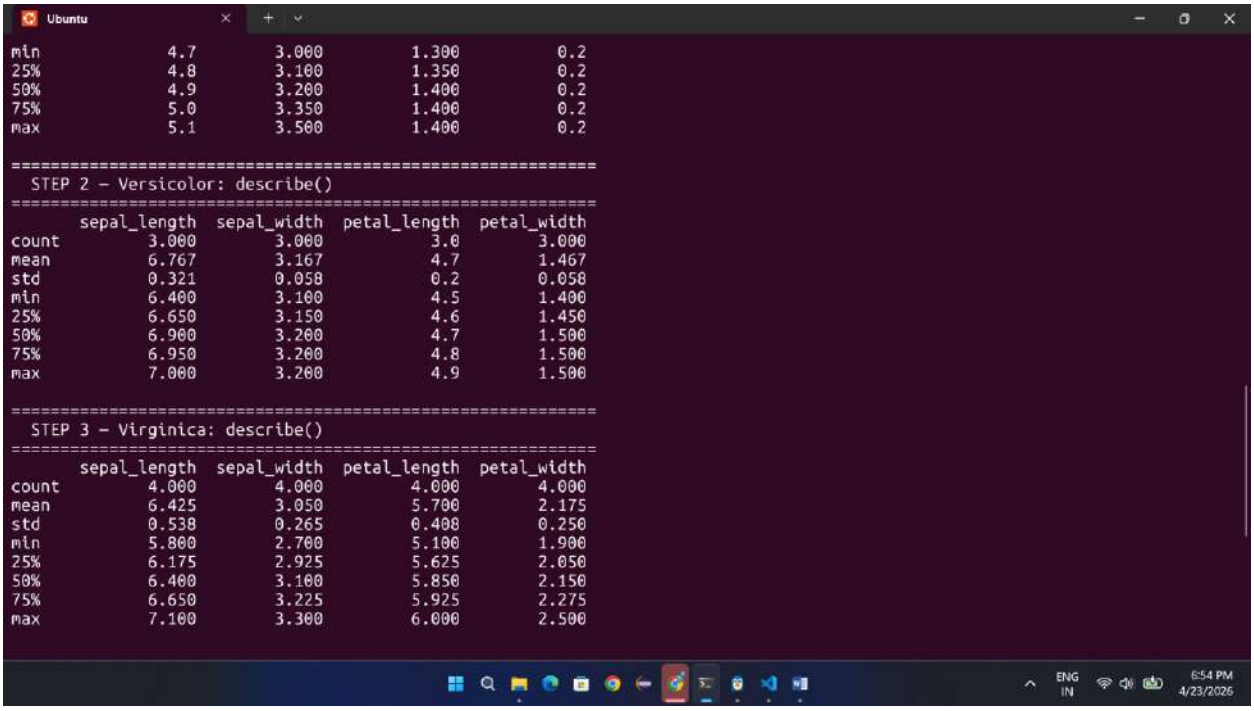
➢ Output

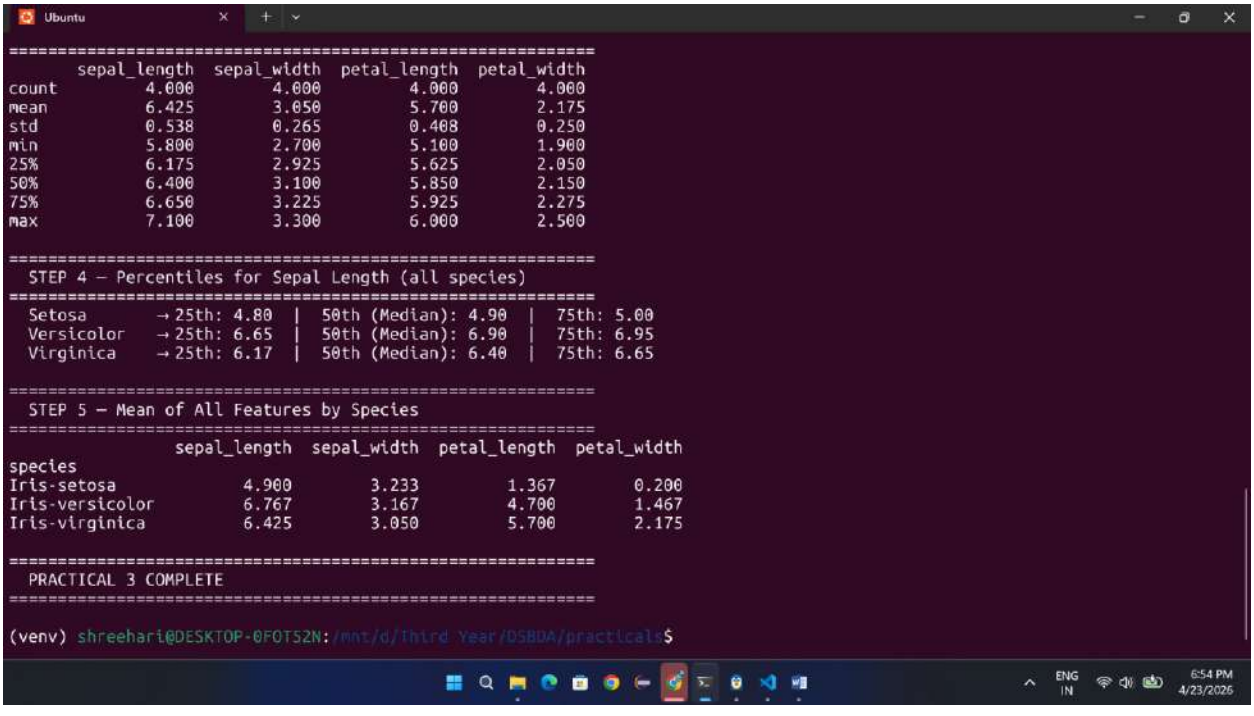


```
(venv) shreehari@DESKTOP-0FOT52N:/mnt/d/Third Year/DSBDA/practicals$ python3 prac_3.py
  PART A – Income Dataset (Grouped Statistics)

============================================================
  STEP 0 – Raw Income Dataset
============================================================
 Name  Age  Income Age_Group
Alice   25   30000     Young
  Bob   32   50000     Young
Carol   45   75000    Middle
 Dave   23   28000     Young
  Eve   55   90000    Senior
Frank   40   65000    Middle
Grace   29   40000     Young
 Hank   60   85000    Senior
 Iris   35   55000    Middle
 Jack   48   70000    Middle

============================================================
  STEP 1 – Summary Statistics Grouped by Age_Group
============================================================
              Mean    Median    Min    Max   Std Dev
Age_Group
Middle     66250.0  67500.0  55000  75000   8539.13
Senior     87500.0  87500.0  85000  90000   3535.53
Young      37000.0  35000.0  28000  50000  10132.46

============================================================
  STEP 2 – Income Values Listed per Group
============================================================
  Middle: [75000, 65000, 55000, 70000]
  Senior: [90000, 85000]
```



```
============================================================
  STEP 3 – Overall Income Statistics
============================================================
  Mean   : 58,800.00
  Median : 60,000.00
  Mode   : [28000 30000 40000 50000 55000 65000 70000 75000 85000 90000]
  Std Dev: 21,882.51
  Min    : 28,000.00
  Max    : 90,000.00
  PART B – Iris Dataset (Species-wise Statistics)

============================================================
  STEP 0 – Iris Dataset
============================================================
 sepal_length  sepal_width  petal_length  petal_width         species
          5.1          3.5           1.4          0.2     Iris-setosa
          4.9          3.0           1.4          0.2     Iris-setosa
          4.7          3.2           1.3          0.2     Iris-setosa
          7.0          3.2           4.7          1.4 Iris-versicolor
          6.4          3.2           4.5          1.5 Iris-versicolor
          6.9          3.1           4.9          1.5 Iris-versicolor
          6.3          3.3           6.0          2.5  Iris-virginica
          5.8          2.7           5.1          1.9  Iris-virginica
          7.1          3.0           5.9          2.1  Iris-virginica
          6.5          3.2           5.8          2.2  Iris-virginica

============================================================
  STEP 1 – Setosa: describe()
============================================================
       sepal_length  sepal_width  petal_length  petal_width
count           3.0        3.000         3.000          3.0
```


Ubuntu
min 4.7 3.000 1.300 0.2
25% 4.8 3.100 1.350 0.2
50% 4.9 3.200 1.400 0.2
75% 5.0 3.350 1.400 0.2
max 5.1 3.500 1.400 0.2
STEP 2 — Versicolor: describe()
sepal_length sepal_width petal_length petal_width
count 3.000 3.000 3.0 3.000
mean 6.767 3.167 4.7 1.467
std 0.321 0.058 0.2 0.058
min 6.400 3.100 4.5 1.400
25% 6.650 3.150 4.6 1.450
50% 6.900 3.200 4.7 1.500
75% 6.950 3.200 4.8 1.500
max 7.000 3.200 4.9 1.500
STEP 3 — Virginica: describe()
sepal_length sepal_width petal_length petal_width
count 4.000 4.000 4.000 4.000
mean 6.425 3.050 5.700 2.175
std 0.538 0.265 0.408 0.250
min 5.800 2.700 5.100 1.900
25% 6.175 2.925 5.625 2.050
50% 6.400 3.100 5.850 2.150
75% 6.650 3.225 5.925 2.275
max 7.100 3.300 6.000 2.500
ENG IN
6:54 PM 4/23/2026


Ubuntu
sepal_length sepal_width petal_length petal_width
count 4.000 4.000 4.000 4.000
mean 6.425 3.050 5.700 2.175
std 0.538 0.265 0.408 0.250
min 5.800 2.700 5.100 1.900
25% 6.175 2.925 5.625 2.050
50% 6.400 3.100 5.850 2.150
75% 6.650 3.225 5.925 2.275
max 7.100 3.300 6.000 2.500
STEP 4 — Percentiles for Sepal Length (all species)
Setosa → 25th: 4.80 | 50th (Median): 4.90 | 75th: 5.00
Versicolor → 25th: 6.65 | 50th (Median): 6.90 | 75th: 6.95
Virginica → 25th: 6.17 | 50th (Median): 6.40 | 75th: 6.65
STEP 5 — Mean of All Features by Species
sepal_length sepal_width petal_length petal_width
species
Iris-setosa 4.900 3.233 1.367 0.200
Iris-versicolor 6.767 3.167 4.700 1.467
Iris-virginica 6.425 3.050 5.700 2.175
PRACTICAL 3 COMPLETE
(venv) shreehari@DESKTOP-0FOT52N:/mnt/d/Third Year/DSBDA/practicals$
ENG IN
6:54 PM 4/23/2026

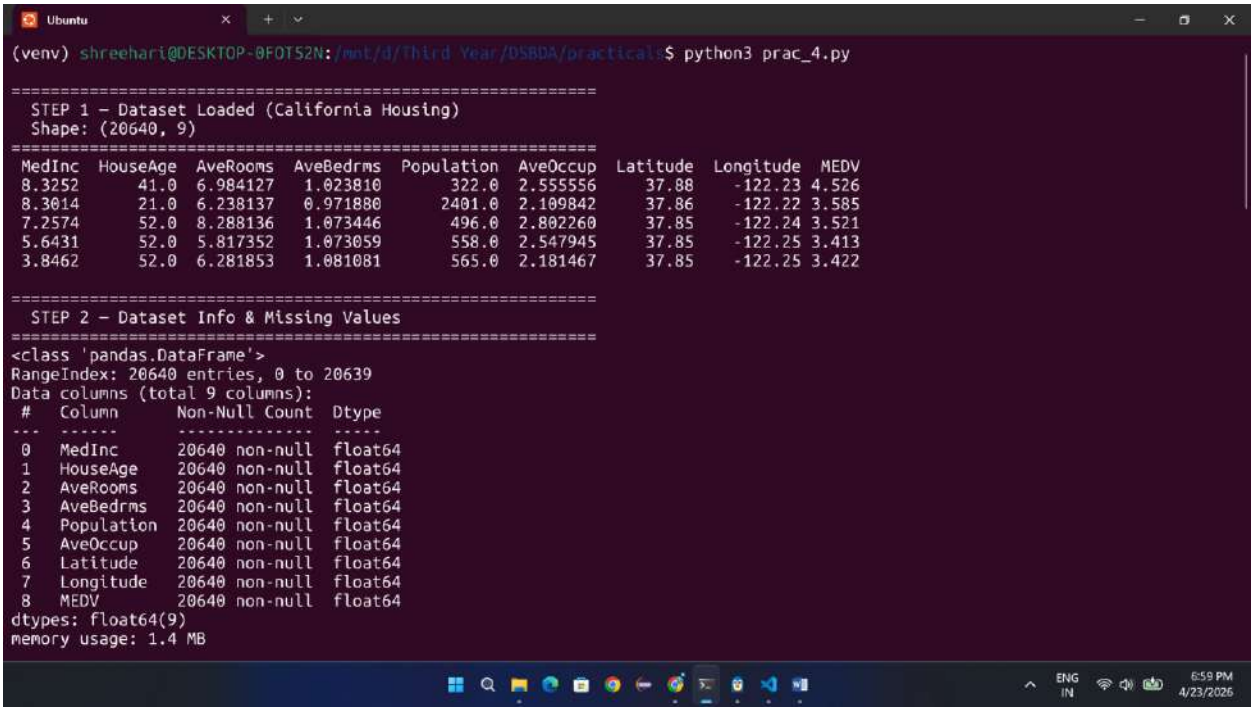
PRACTICAL 4

- Source Code

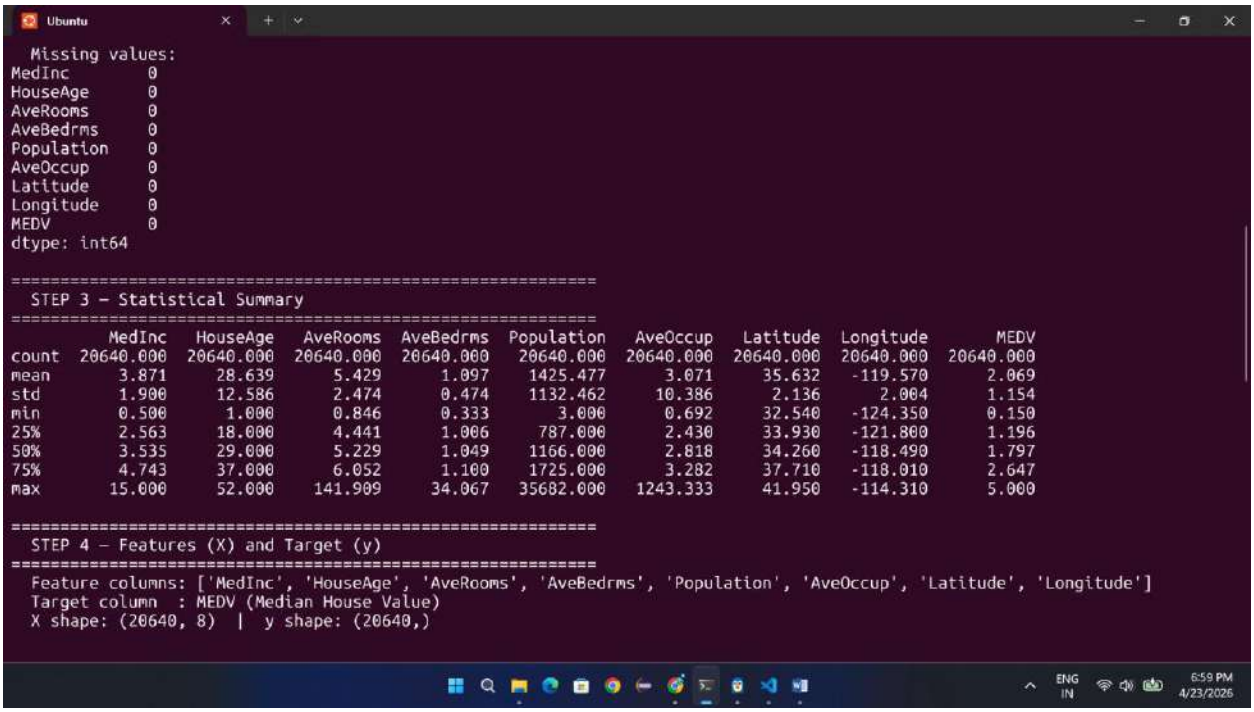
```
import pandas as pd
import numpy as np
from sklearn.model_selection import train_test_split
from sklearn.linear_model import LinearRegression
from sklearn.metrics import mean_squared_error, r2_score
DIVIDER = "=" * 60
from sklearn.datasets import fetch_california_housing
housing = fetch_california_housing(as_frame=True)
df = housing.frame
df = df.rename(columns={'MedHouseVal': 'MEDV'})
print(f"\n{DIVIDER}")
print("  STEP 1 — Dataset Loaded (California Housing)")
print(f"  Shape: {df.shape}")
print(DIVIDER)
print(df.head().to_string(index=False))
print(f"\n{DIVIDER}")
print("  STEP 2 — Dataset Info & Missing Values")
print(DIVIDER)
print(df.info())
print(f"\n  Missing values:\n{df.isnull().sum()}")

print(f"\n{DIVIDER}")
print("  STEP 3 — Statistical Summary")
print(DIVIDER)
print(df.describe().round(3))
X = df.drop('MEDV', axis=1)   # All columns except target
y = df['MEDV']                # Target: House price
print(f"\n{DIVIDER}")
print("  STEP 4 — Features (X) and Target (y)")
print(DIVIDER)
print(f"  Feature columns: {list(X.columns)}")
print(f"  Target column  : MEDV (Median House Value)")
print(f"  X shape: {X.shape}  |  y shape: {y.shape}")

X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42
)
```

```
print(f"\n{DIVIDER}")
print("  STEP 5 — Train-Test Split (80% train, 20% test)")
print(DIVIDER)
print(f"  Training set  : {X_train.shape[0]} rows")
print(f"  Testing set   : {X_test.shape[0]} rows")
model = LinearRegression()
model.fit(X_train, y_train)
print(f"\n{DIVIDER}")
print("  STEP 6 — Model Trained!")
print(DIVIDER)
print(f"  Intercept (b0): {model.intercept_:.4f}")
print(f"\n  Coefficients:")
for feat, coef in zip(X.columns, model.coef_):
    print(f"    {feat:20s}: {coef:.4f}")
y_pred = model.predict(X_test)
print(f"\n{DIVIDER}")
print("  STEP 7 — Sample Predictions vs Actual")
print(DIVIDER)
comparison = pd.DataFrame({
    'Actual Price':    y_test.values[:10],
    'Predicted Price': y_pred[:10].round(3)
})
print(comparison.to_string(index=False))

mse  = mean_squared_error(y_test, y_pred)
rmse = np.sqrt(mse)
r2   = r2_score(y_test, y_pred)

print(f"\n{DIVIDER}")
print("  STEP 8 — Model Evaluation Metrics")
print(DIVIDER)
print(f"  Mean Squared Error  (MSE) : {mse:.4f}")
print(f"  Root MSE            (RMSE): {rmse:.4f}")
print(f"  R² Score                  : {r2:.4f}")
print(f"\n  Interpretation:")
print(f"  → R² = {r2:.4f} means the model explains {r2*100:.1f}% of variance in price.")
print(f"  → Closer to 1.0 is better.")

print(f"\n{'='*60}")
print("  PRACTICAL 4 COMPLETE")
print(f"{'='*60}\n")
```

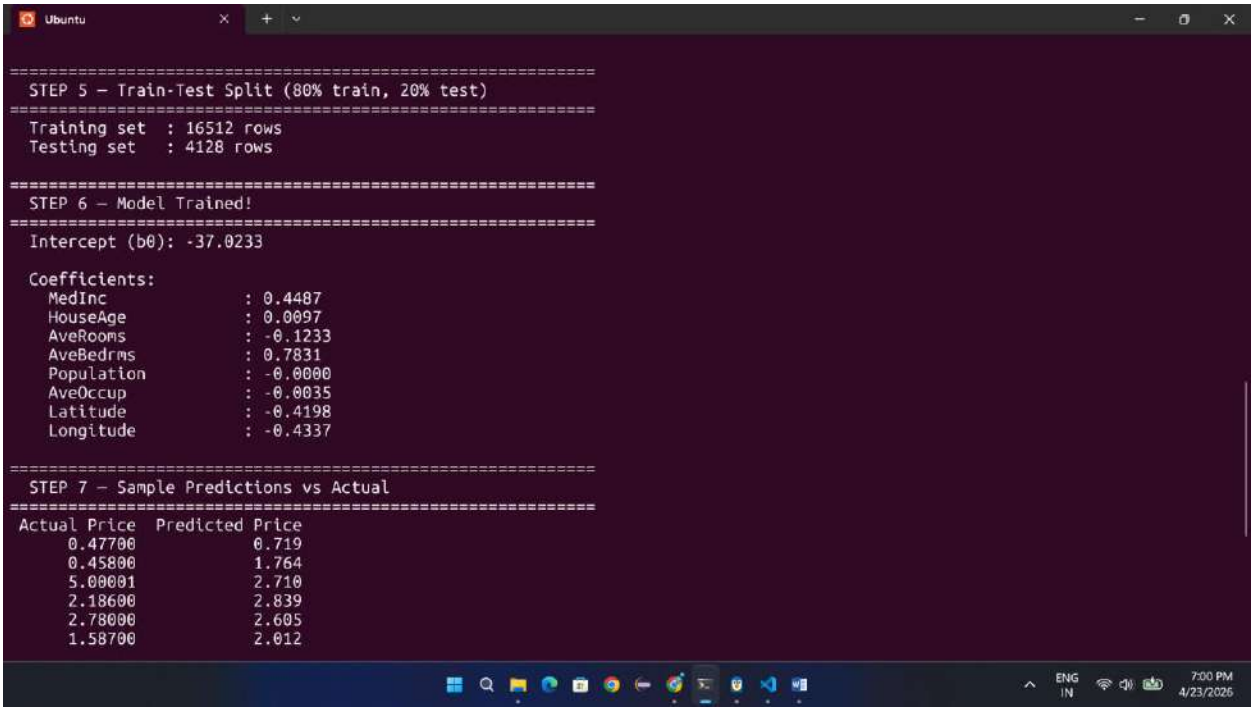
➢ Output

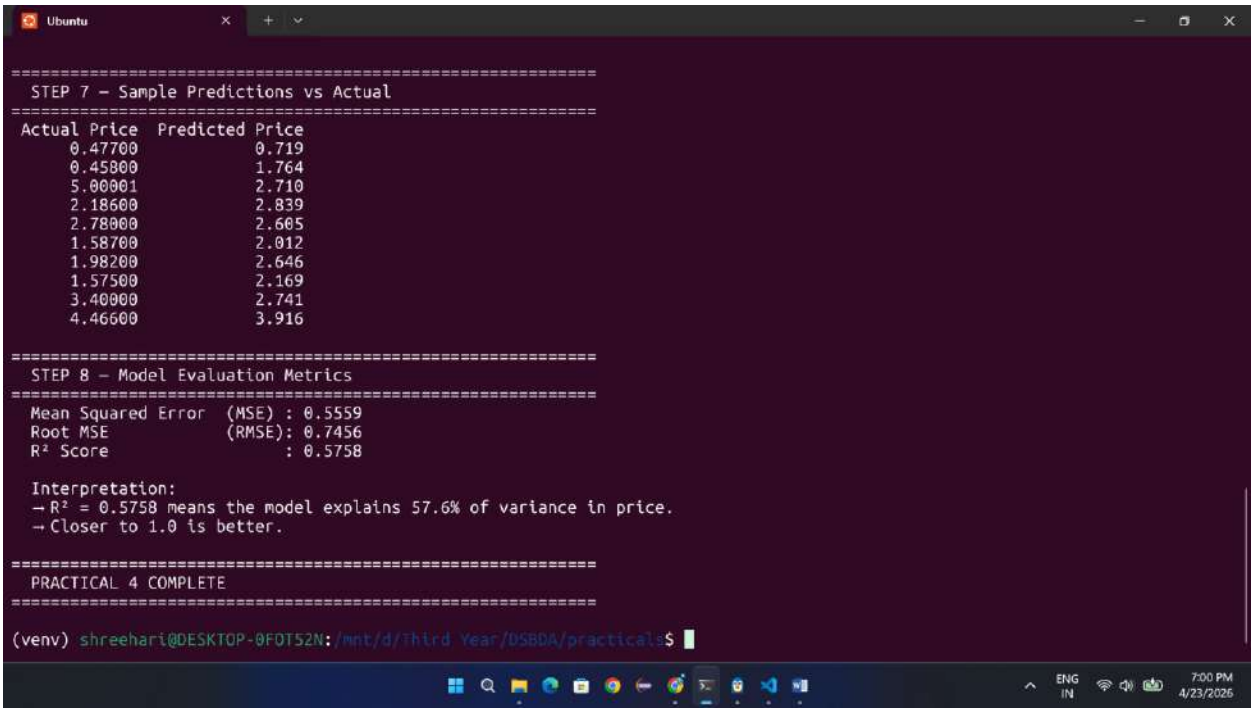


```
Ubuntu
(venv) shreehari@DESKTOP-0FOT52N:/mnt/d/Third Year/DSBDA/practicals$ python3 prac_4.py

============================================================
  STEP 1 – Dataset Loaded (California Housing)
  Shape: (20640, 9)
============================================================
 MedInc  HouseAge  AveRooms  AveBedrms  Population  AveOccup  Latitude  Longitude  MEDV
 8.3252      41.0  6.984127   1.023810       322.0  2.555556     37.88    -122.23 4.526
 8.3014      21.0  6.238137   0.971880      2401.0  2.109842     37.86    -122.22 3.585
 7.2574      52.0  8.288136   1.073446       496.0  2.802260     37.85    -122.24 3.521
 5.6431      52.0  5.817352   1.073059       558.0  2.547945     37.85    -122.25 3.413
 3.8462      52.0  6.281853   1.081081       565.0  2.181467     37.85    -122.25 3.422

============================================================
  STEP 2 – Dataset Info & Missing Values
============================================================
<class 'pandas.DataFrame'>
RangeIndex: 20640 entries, 0 to 20639
Data columns (total 9 columns):
 #   Column      Non-Null Count  Dtype
---  ------      --------------  -----
 0   MedInc      20640 non-null  float64
 1   HouseAge    20640 non-null  float64
 2   AveRooms    20640 non-null  float64
 3   AveBedrms   20640 non-null  float64
 4   Population  20640 non-null  float64
 5   AveOccup    20640 non-null  float64
 6   Latitude    20640 non-null  float64
 7   Longitude   20640 non-null  float64
 8   MEDV        20640 non-null  float64
dtypes: float64(9)
memory usage: 1.4 MB
```



```
Ubuntu
  Missing values:
MedInc        0
HouseAge      0
AveRooms      0
AveBedrms     0
Population    0
AveOccup      0
Latitude      0
Longitude     0
MEDV          0
dtype: int64

============================================================
  STEP 3 – Statistical Summary
============================================================
          MedInc   HouseAge   AveRooms  AveBedrms  Population   AveOccup   Latitude  Longitude       MEDV
count  20640.000  20640.000  20640.000  20640.000   20640.000  20640.000  20640.000  20640.000  20640.000
mean       3.871     28.639      5.429      1.097    1425.477      3.071     35.632   -119.570      2.069
std        1.900     12.586      2.474      0.474    1132.462     10.386      2.136      2.004      1.154
min        0.500      1.000      0.846      0.333       3.000      0.692     32.540   -124.350      0.150
25%        2.563     18.000      4.441      1.006     787.000      2.430     33.930   -121.800      1.196
50%        3.535     29.000      5.229      1.049    1166.000      2.818     34.260   -118.490      1.797
75%        4.743     37.000      6.052      1.100    1725.000      3.282     37.710   -118.010      2.647
max       15.000     52.000    141.909     34.067   35682.000   1243.333     41.950   -114.310      5.000

============================================================
  STEP 4 – Features (X) and Target (y)
============================================================
  Feature columns: ['MedInc', 'HouseAge', 'AveRooms', 'AveBedrms', 'Population', 'AveOccup', 'Latitude', 'Longitude']
  Target column  : MEDV (Median House Value)
  X shape: (20640, 8)  |  y shape: (20640,)
```


Ubuntu
STEP 5 — Train-Test Split (80% train, 20% test)
Training set : 16512 rows
Testing set : 4128 rows
STEP 6 — Model Trained!
Intercept (b0): -37.0233
Coefficients:
MedInc : 0.4487
HouseAge : 0.0097
AveRooms : -0.1233
AveBedrms : 0.7831
Population : -0.0000
AveOccup : -0.0035
Latitude : -0.4198
Longitude : -0.4337
STEP 7 — Sample Predictions vs Actual
Actual Price Predicted Price
0.47700 0.719
0.45800 1.764
5.00001 2.710
2.18600 2.839
2.78000 2.605
1.58700 2.012
ENG
IN
7:00 PM
4/23/2026


Ubuntu
STEP 7 — Sample Predictions vs Actual
Actual Price Predicted Price
0.47700 0.719
0.45800 1.764
5.00001 2.710
2.18600 2.839
2.78000 2.605
1.58700 2.012
1.98200 2.646
1.57500 2.169
3.40000 2.741
4.46600 3.916
STEP 8 — Model Evaluation Metrics
Mean Squared Error (MSE) : 0.5559
Root MSE (RMSE): 0.7456
R² Score : 0.5758
Interpretation:
→ R² = 0.5758 means the model explains 57.6% of variance in price.
→ Closer to 1.0 is better.
PRACTICAL 4 COMPLETE
(venv) shreehari@DESKTOP-0FOT52N:/mnt/d/Third Year/DSBDA/practicals$
ENG
IN
7:00 PM
4/23/2026

PRACTICAL 5

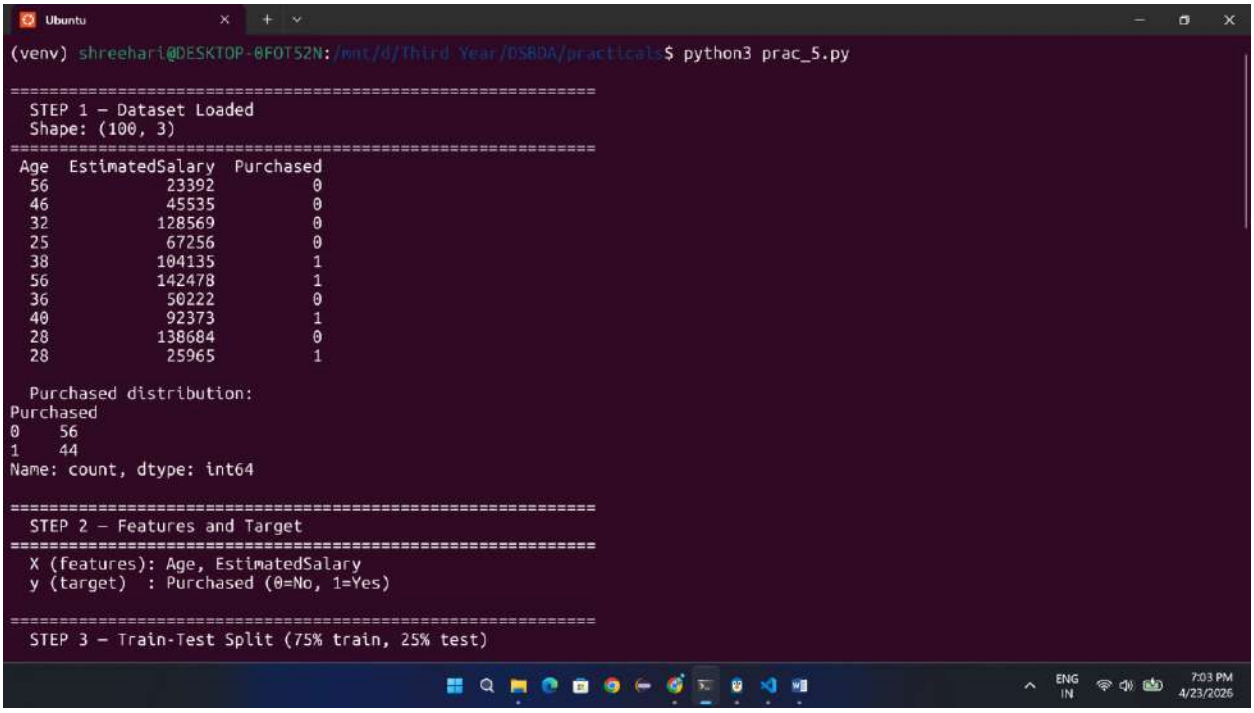
- Source Code

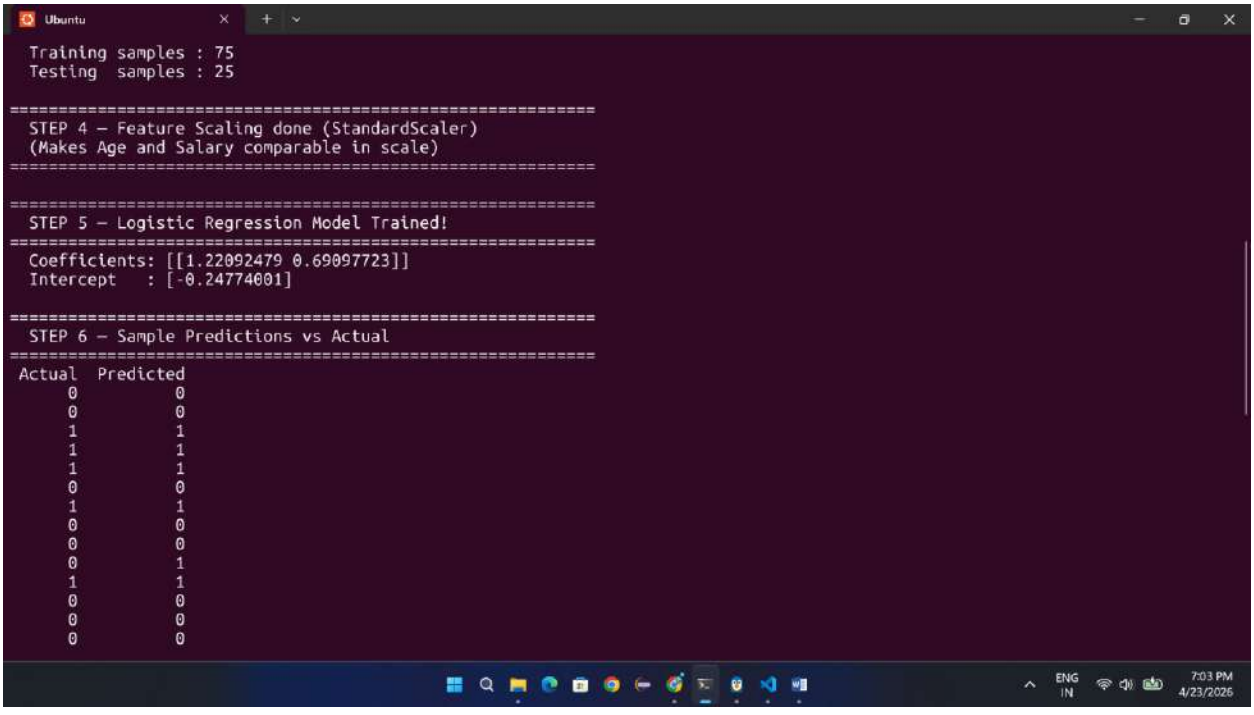
```
import pandas as pd
import numpy as np
from sklearn.model_selection import train_test_split
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import confusion_matrix, classification_report
from sklearn.preprocessing import StandardScaler

DIVIDER = "=" * 60

np.random.seed(42)
n = 100

age    = np.random.randint(18, 60, n)
salary = np.random.randint(15000, 150000, n)
# Older + higher salary → more likely to buy
purchased = ((age > 35) & (salary > 70000)).astype(int)
# Add some noise
noise_idx = np.random.choice(n, 10, replace=False)
purchased[noise_idx] = 1 - purchased[noise_idx]

df = pd.DataFrame({'Age': age, 'EstimatedSalary': salary, 'Purchased': purchased})

print(f"\n{DIVIDER}")
print("  STEP 1 — Dataset Loaded")
print(f"  Shape: {df.shape}")
print(DIVIDER)
print(df.head(10).to_string(index=False))
print(f"\n  Purchased distribution:\n{df['Purchased'].value_counts()}")

X = df[['Age', 'EstimatedSalary']]
y = df['Purchased']

print(f"\n{DIVIDER}")
print("  STEP 2 — Features and Target")
print(DIVIDER)
print(f"  X (features): Age, EstimatedSalary")
print(f"  y (target)  : Purchased (0=No, 1=Yes)")
```

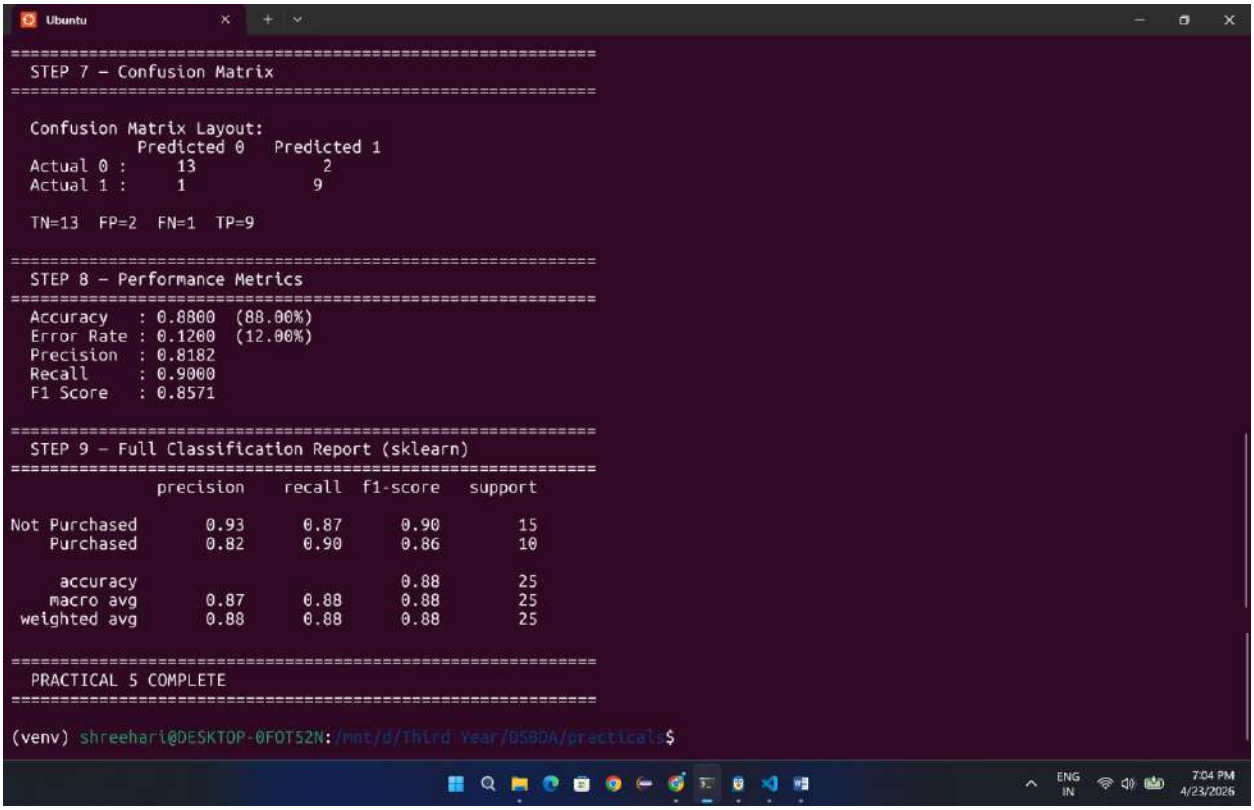
```
X_train, X_test, y_train, y_test = train_test_split(
   X, y, test_size=0.25, random_state=42
)

print(f"\n{DIVIDER}")
print("  STEP 3 — Train-Test Split (75% train, 25% test)")
print(DIVIDER)
print(f"  Training samples : {X_train.shape[0]}")
print(f"  Testing  samples : {X_test.shape[0]}")

scaler  = StandardScaler()
X_train = scaler.fit_transform(X_train)
X_test  = scaler.transform(X_test)

print(f"\n{DIVIDER}")
print("  STEP 4 — Feature Scaling done (StandardScaler)")
print("  (Makes Age and Salary comparable in scale)")
print(DIVIDER)

model = LogisticRegression()
model.fit(X_train, y_train)

print(f"\n{DIVIDER}")
print("  STEP 5 — Logistic Regression Model Trained!")
print(DIVIDER)
print(f"  Coefficients: {model.coef_}")
print(f"  Intercept   : {model.intercept_}")
y_pred = model.predict(X_test)

print(f"\n{DIVIDER}")
print("  STEP 6 — Sample Predictions vs Actual")
print(DIVIDER)
result = pd.DataFrame({'Actual': y_test.values, 'Predicted': y_pred})
print(result.head(15).to_string(index=False))

cm = confusion_matrix(y_test, y_pred)

print(f"\n{DIVIDER}")
print("  STEP 7 — Confusion Matrix")
print(DIVIDER)
```

```
print(f"\n  Confusion Matrix Layout:")
print(f"            Predicted 0   Predicted 1")
print(f"  Actual 0 :    {cm[0,0]}          {cm[0,1]}")
print(f"  Actual 1 :    {cm[1,0]}          {cm[1,1]}")
print(f"\n  TN={cm[0,0]}  FP={cm[0,1]}  FN={cm[1,0]}  TP={cm[1,1]}")

TP = cm[1, 1]
TN = cm[0, 0]
FP = cm[0, 1]
FN = cm[1, 0]

accuracy   = (TP + TN) / (TP + TN + FP + FN)
error_rate = (FP + FN) / (TP + TN + FP + FN)
precision  = TP / (TP + FP) if (TP + FP) != 0 else 0
recall     = TP / (TP + FN) if (TP + FN) != 0 else 0
f1         = 2 * precision * recall / (precision + recall) if (precision + recall) != 0 else 0

print(f"\n{DIVIDER}")
print("  STEP 8 — Performance Metrics")
print(DIVIDER)
print(f"  Accuracy   : {accuracy:.4f}  ({accuracy*100:.2f}%)")
print(f"  Error Rate : {error_rate:.4f}  ({error_rate*100:.2f}%)")
print(f"  Precision  : {precision:.4f}")
print(f"  Recall     : {recall:.4f}")
print(f"  F1 Score   : {f1:.4f}")

print(f"\n{DIVIDER}")
print("  STEP 9 — Full Classification Report (sklearn)")
print(DIVIDER)
print(classification_report(y_test, y_pred, target_names=['Not Purchased', 'Purchased']))

print(f"\n{'='*60}")
print("  PRACTICAL 5 COMPLETE")
print(f"{'='*60}\n")
```

➢ Output


Ubuntu
(venv) shreehari@DESKTOP-0FOT52N:/mnt/d/Third Year/DSBDA/practicals$ python3 prac_5.py
==
STEP 1 – Dataset Loaded
Shape: (100, 3)
==
Age EstimatedSalary Purchased
56 23392 0
46 45535 0
32 128569 0
25 67256 0
38 104135 1
56 142478 1
36 50222 0
40 92373 1
28 138684 0
28 25965 1
Purchased distribution:
Purchased
0 56
1 44
Name: count, dtype: int64
==
STEP 2 – Features and Target
==
X (features): Age, EstimatedSalary
y (target) : Purchased (0=No, 1=Yes)
==
STEP 3 – Train-Test Split (75% train, 25% test)
ENG
IN
7:03 PM
4/23/2026


Ubuntu
Training samples : 75
Testing samples : 25
==
STEP 4 – Feature Scaling done (StandardScaler)
(Makes Age and Salary comparable in scale)
==
==
STEP 5 – Logistic Regression Model Trained!
==
Coefficients: [[1.22092479 0.69097723]]
Intercept : [-0.24774001]
==
STEP 6 – Sample Predictions vs Actual
==
Actual Predicted
0 0
0 0
1 1
1 1
1 1
0 0
1 1
0 0
0 0
0 1
1 1
0 0
0 0
0 0
ENG
IN
7:03 PM
4/23/2026


Ubuntu
STEP 7 – Confusion Matrix
Confusion Matrix Layout:
Predicted 0 Predicted 1
Actual 0 : 13 2
Actual 1 : 1 9
TN=13 FP=2 FN=1 TP=9
STEP 8 – Performance Metrics
Accuracy : 0.8800 (88.00%)
Error Rate : 0.1200 (12.00%)
Precision : 0.8182
Recall : 0.9000
F1 Score : 0.8571
STEP 9 – Full Classification Report (sklearn)
precision recall f1-score support
Not Purchased 0.93 0.87 0.90 15
Purchased 0.82 0.90 0.86 10
accuracy 0.88 25
macro avg 0.87 0.88 0.88 25
weighted avg 0.88 0.88 0.88 25
PRACTICAL 5 COMPLETE
(venv) shreehari@DESKTOP-0FOT52N:/mnt/d/Third Year/DSBDA/practicals$
ENG
IN
7:04 PM
4/23/2026

PRACTICAL 6

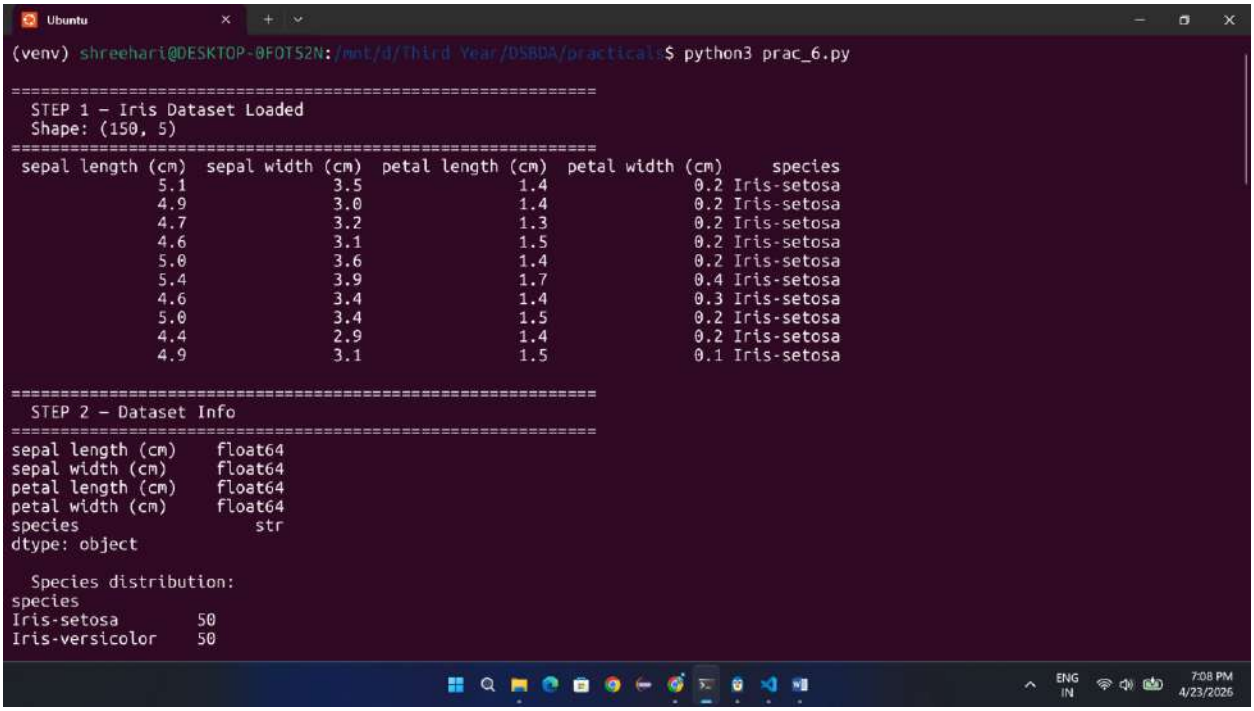
- Source Code

```
import pandas as pd
import numpy as np
from sklearn.model_selection import train_test_split
from sklearn.naive_bayes import GaussianNB
from sklearn.metrics import confusion_matrix, classification_report, accuracy_score

DIVIDER = "=" * 60

from sklearn.datasets import load_iris

iris_bunch = load_iris(as_frame=True)
df = iris_bunch.frame
df['species'] = df['target'].map({0: 'Iris-setosa', 1: 'Iris-versicolor', 2: 'Iris-virginica'})
df = df.drop('target', axis=1)

print(f"\n{DIVIDER}")
print("  STEP 1 — Iris Dataset Loaded")
print(f"  Shape: {df.shape}")
print(DIVIDER)
print(df.head(10).to_string(index=False))

print(f"\n{DIVIDER}")
print("  STEP 2 — Dataset Info")
print(DIVIDER)
print(df.dtypes)
print(f"\n  Species distribution:\n{df['species'].value_counts()}")

print(f"\n{DIVIDER}")
print("  STEP 3 — Statistical Summary")
print(DIVIDER)
print(df.describe().round(3))

X = df.drop('species', axis=1)
y = df['species']

print(f"\n{DIVIDER}")
print("  STEP 4 — Features (X) and Target (y)")
```

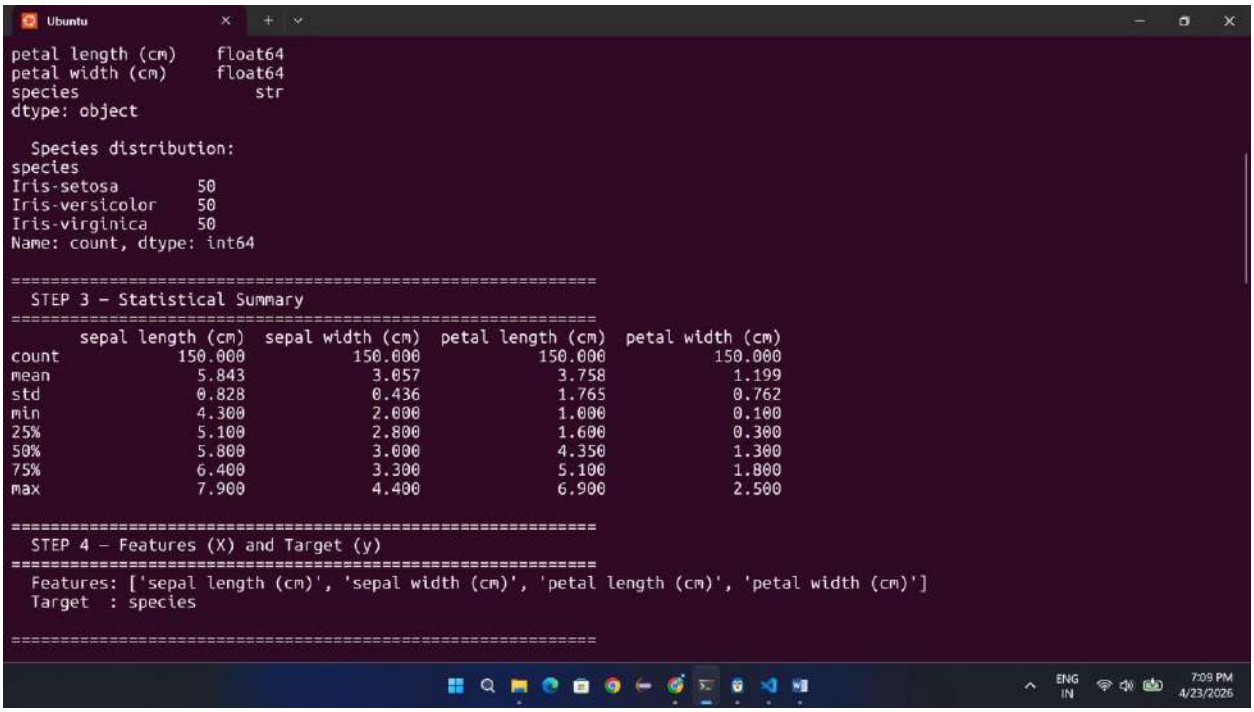
```
print(DIVIDER)
print(f"  Features: {list(X.columns)}")
print(f"  Target  : species")

X_train, X_test, y_train, y_test = train_test_split(
   X, y, test_size=0.3, random_state=42
)

print(f"\n{DIVIDER}")
print("  STEP 5 — Train-Test Split (70% train, 30% test)")
print(DIVIDER)
print(f"  Training samples: {X_train.shape[0]}")
print(f"  Testing  samples: {X_test.shape[0]}")

model = GaussianNB()
model.fit(X_train, y_train)

print(f"\n{DIVIDER}")
print("  STEP 6 — Gaussian Naïve Bayes Model Trained!")
print(DIVIDER)
print(f"  Classes learned: {list(model.classes_)}")

y_pred = model.predict(X_test)

print(f"\n{DIVIDER}")
print("  STEP 7 — Sample Predictions vs Actual")
print(DIVIDER)
result = pd.DataFrame({'Actual': y_test.values, 'Predicted': y_pred})
print(result.head(15).to_string(index=False))

cm = confusion_matrix(y_test, y_pred)
classes = ['Setosa', 'Versicolor', 'Virginica']

print(f"\n{DIVIDER}")
print("  STEP 8 — Confusion Matrix (3x3 — Multiclass)")
print(DIVIDER)
cm_df = pd.DataFrame(cm,
                index  =[f'Actual {c}' for c in classes],
                columns=[f'Pred {c}'   for c in classes])
print(cm_df)
```

```
print(f"\n{DIVIDER}")
print("  STEP 9 — Performance Metrics per Class")
print(DIVIDER)

for i, cls in enumerate(classes):
    TP = cm[i, i]
    FP = cm[:, i].sum() - TP
    FN = cm[i, :].sum() - TP
    TN = cm.sum() - TP - FP - FN

    accuracy  = (TP + TN) / cm.sum()
    precision = TP / (TP + FP) if (TP + FP) != 0 else 0
    recall    = TP / (TP + FN) if (TP + FN) != 0 else 0
    f1        = 2 * precision * recall / (precision + recall) if (precision + recall) != 0 else 0

    print(f"\n  [{cls}]")
    print(f"    TP={TP}  FP={FP}  FN={FN}  TN={TN}")
    print(f"    Accuracy  : {accuracy:.4f}")
    print(f"    Precision : {precision:.4f}")
    print(f"    Recall    : {recall:.4f}")
    print(f"    F1 Score  : {f1:.4f}")

overall = accuracy_score(y_test, y_pred)
print(f"\n{DIVIDER}")
print("  STEP 10 — Overall Accuracy & Full Report")
print(DIVIDER)
print(f"  Overall Accuracy: {overall:.4f} ({overall*100:.2f}%)")
print(f"\n{classification_report(y_test, y_pred)}")

print(f"\n{'='*60}")
print("  PRACTICAL 6 COMPLETE")
print(f"{'='*60}\n")
```

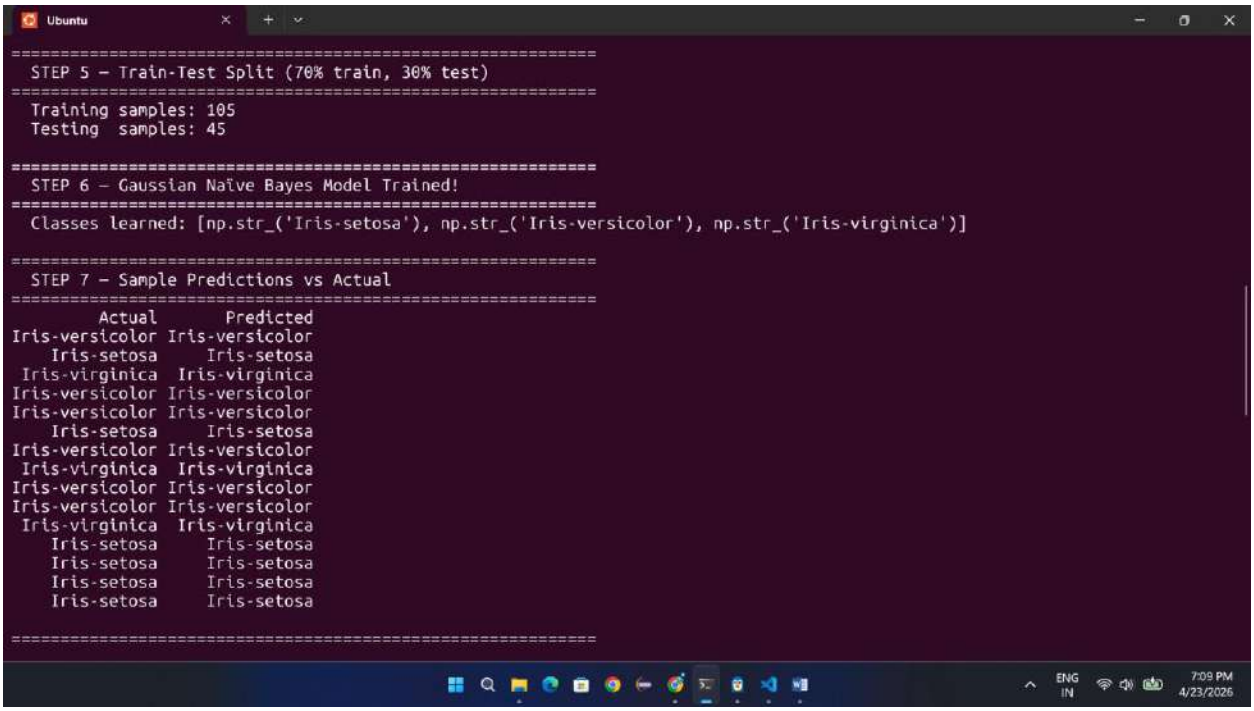
➢ Output

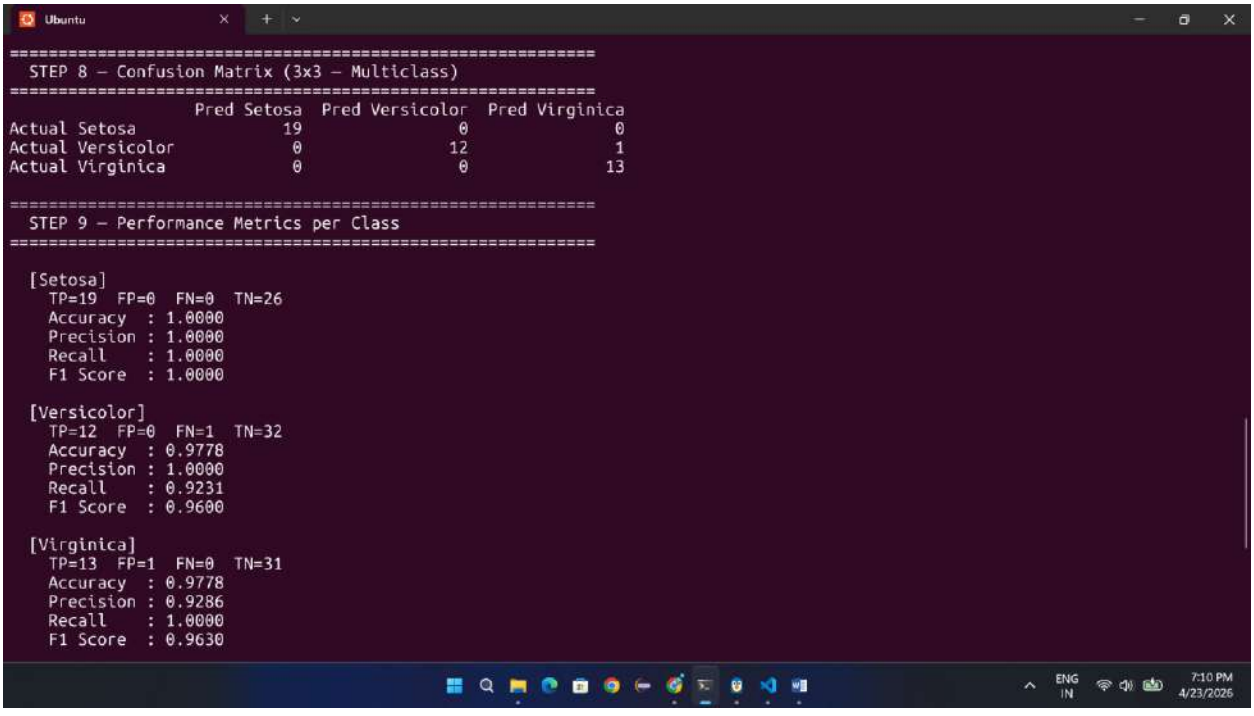


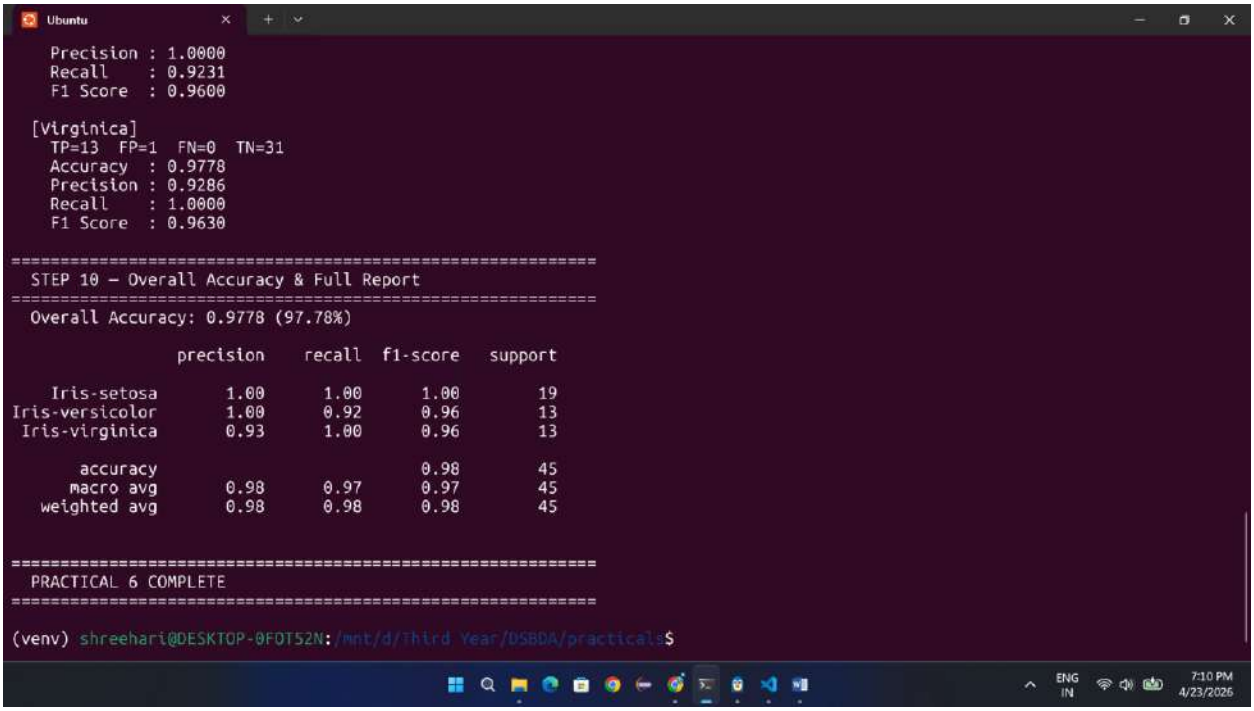
```
Ubuntu
(venv) shreehari@DESKTOP-0FOT52N:/mnt/d/Third Year/DSBDA/practicals$ python3 prac_6.py

============================================================
  STEP 1 – Iris Dataset Loaded
  Shape: (150, 5)
============================================================
 sepal length (cm)  sepal width (cm)  petal length (cm)  petal width (cm)     species
               5.1               3.5                1.4               0.2 Iris-setosa
               4.9               3.0                1.4               0.2 Iris-setosa
               4.7               3.2                1.3               0.2 Iris-setosa
               4.6               3.1                1.5               0.2 Iris-setosa
               5.0               3.6                1.4               0.2 Iris-setosa
               5.4               3.9                1.7               0.4 Iris-setosa
               4.6               3.4                1.4               0.3 Iris-setosa
               5.0               3.4                1.5               0.2 Iris-setosa
               4.4               2.9                1.4               0.2 Iris-setosa
               4.9               3.1                1.5               0.1 Iris-setosa

============================================================
  STEP 2 – Dataset Info
============================================================
sepal length (cm)    float64
sepal width (cm)     float64
petal length (cm)    float64
petal width (cm)     float64
species                  str
dtype: object

  Species distribution:
species
Iris-setosa        50
Iris-versicolor    50
```



```
Ubuntu
petal length (cm)    float64
petal width (cm)     float64
species                  str
dtype: object

  Species distribution:
species
Iris-setosa        50
Iris-versicolor    50
Iris-virginica     50
Name: count, dtype: int64

============================================================
  STEP 3 – Statistical Summary
============================================================
       sepal length (cm)  sepal width (cm)  petal length (cm)  petal width (cm)
count            150.000           150.000            150.000           150.000
mean               5.843             3.057              3.758             1.199
std                0.828             0.436              1.765             0.762
min                4.300             2.000              1.000             0.100
25%                5.100             2.800              1.600             0.300
50%                5.800             3.000              4.350             1.300
75%                6.400             3.300              5.100             1.800
max                7.900             4.400              6.900             2.500

============================================================
  STEP 4 – Features (X) and Target (y)
============================================================
  Features: ['sepal length (cm)', 'sepal width (cm)', 'petal length (cm)', 'petal width (cm)']
  Target  : species

============================================================
```


Ubuntu
==
STEP 5 — Train-Test Split (70% train, 30% test)
==
Training samples: 105
Testing samples: 45
==
STEP 6 — Gaussian Naïve Bayes Model Trained!
==
Classes learned: [np.str_('Iris-setosa'), np.str_('Iris-versicolor'), np.str_('Iris-virginica')]
==
STEP 7 — Sample Predictions vs Actual
==
Actual Predicted
Iris-versicolor Iris-versicolor
Iris-setosa Iris-setosa
Iris-virginica Iris-virginica
Iris-versicolor Iris-versicolor
Iris-versicolor Iris-versicolor
Iris-setosa Iris-setosa
Iris-versicolor Iris-versicolor
Iris-virginica Iris-virginica
Iris-versicolor Iris-versicolor
Iris-versicolor Iris-versicolor
Iris-virginica Iris-virginica
Iris-setosa Iris-setosa
Iris-setosa Iris-setosa
Iris-setosa Iris-setosa
Iris-setosa Iris-setosa
==
ENG
IN
7:09 PM
4/23/2026


Ubuntu
==
STEP 8 — Confusion Matrix (3x3 — Multiclass)
==
Pred Setosa Pred Versicolor Pred Virginica
Actual Setosa 19 0 0
Actual Versicolor 0 12 1
Actual Virginica 0 0 13
==
STEP 9 — Performance Metrics per Class
==
[Setosa]
TP=19 FP=0 FN=0 TN=26
Accuracy : 1.0000
Precision : 1.0000
Recall : 1.0000
F1 Score : 1.0000
[Versicolor]
TP=12 FP=0 FN=1 TN=32
Accuracy : 0.9778
Precision : 1.0000
Recall : 0.9231
F1 Score : 0.9600
[Virginica]
TP=13 FP=1 FN=0 TN=31
Accuracy : 0.9778
Precision : 0.9286
Recall : 1.0000
F1 Score : 0.9630
ENG
IN
7:10 PM
4/23/2026


Ubuntu
Precision : 1.0000
Recall : 0.9231
F1 Score : 0.9600
[Virginica]
TP=13 FP=1 FN=0 TN=31
Accuracy : 0.9778
Precision : 0.9286
Recall : 1.0000
F1 Score : 0.9630
STEP 10 — Overall Accuracy & Full Report
Overall Accuracy: 0.9778 (97.78%)
precision recall f1-score support
Iris-setosa 1.00 1.00 1.00 19
Iris-versicolor 1.00 0.92 0.96 13
Iris-virginica 0.93 1.00 0.96 13
accuracy 0.98 45
macro avg 0.98 0.97 0.97 45
weighted avg 0.98 0.98 0.98 45
PRACTICAL 6 COMPLETE
(venv) shreehari@DESKTOP-0FOT52N:/mnt/d/Third Year/DSBDA/practicals$
ENG
IN
7:10 PM
4/23/2026

PRACTICAL 7

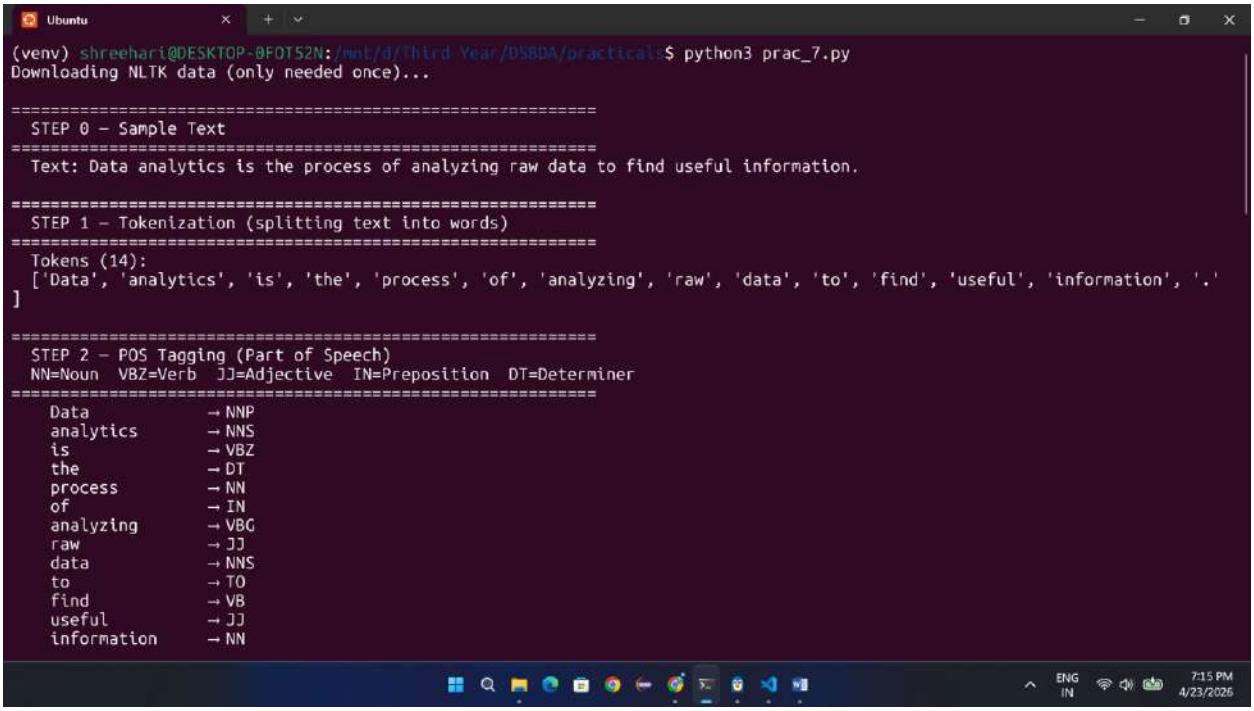
- Source Code

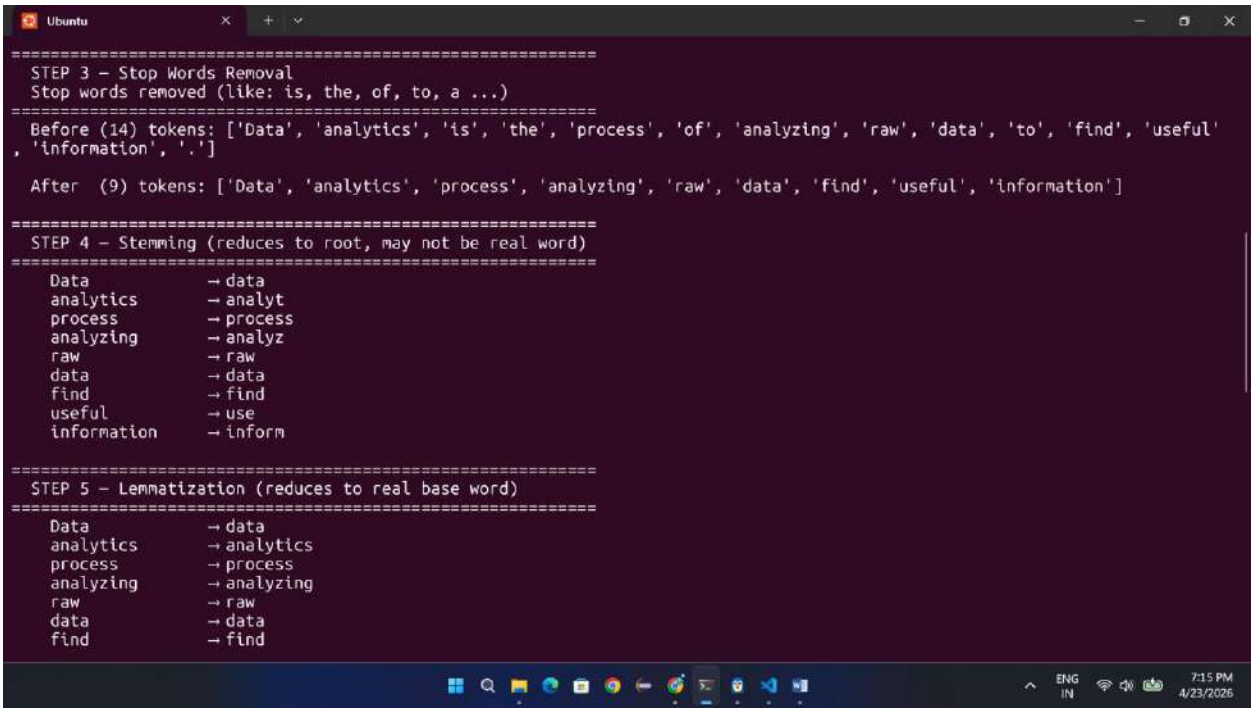
```
import nltk
import pandas as pd
from sklearn.feature_extraction.text import TfidfVectorizer
print("Downloading NLTK data (only needed once)...")
nltk.download('punkt',          quiet=True)
nltk.download('punkt_tab',      quiet=True)
nltk.download('averaged_perceptron_tagger', quiet=True)
nltk.download('averaged_perceptron_tagger_eng', quiet=True)
nltk.download('stopwords',      quiet=True)
nltk.download('wordnet',        quiet=True)
from nltk.tokenize import word_tokenize
from nltk import pos_tag
from nltk.corpus import stopwords
from nltk.stem import PorterStemmer, WordNetLemmatizer
DIVIDER = "=" * 60
text = "Data analytics is the process of analyzing raw data to find useful information."

docs = [
   "Data analytics is the process of analyzing raw data to find useful information.",
   "Machine learning algorithms learn from data and improve over time.",
   "Python is a popular programming language for data science and analytics.",
]

print(f"\n{DIVIDER}")
print("  STEP 0 — Sample Text")
print(DIVIDER)
print(f"  Text: {text}")
tokens = word_tokenize(text)
print(f"\n{DIVIDER}")
print("  STEP 1 — Tokenization (splitting text into words)")
print(DIVIDER)
print(f"  Tokens ({len(tokens)}):\n  {tokens}")
pos_tags = pos_tag(tokens)
print(f"\n{DIVIDER}")
print("  STEP 2 — POS Tagging (Part of Speech)")
print("  NN=Noun  VBZ=Verb  JJ=Adjective  IN=Preposition  DT=Determiner")
print(DIVIDER)
```

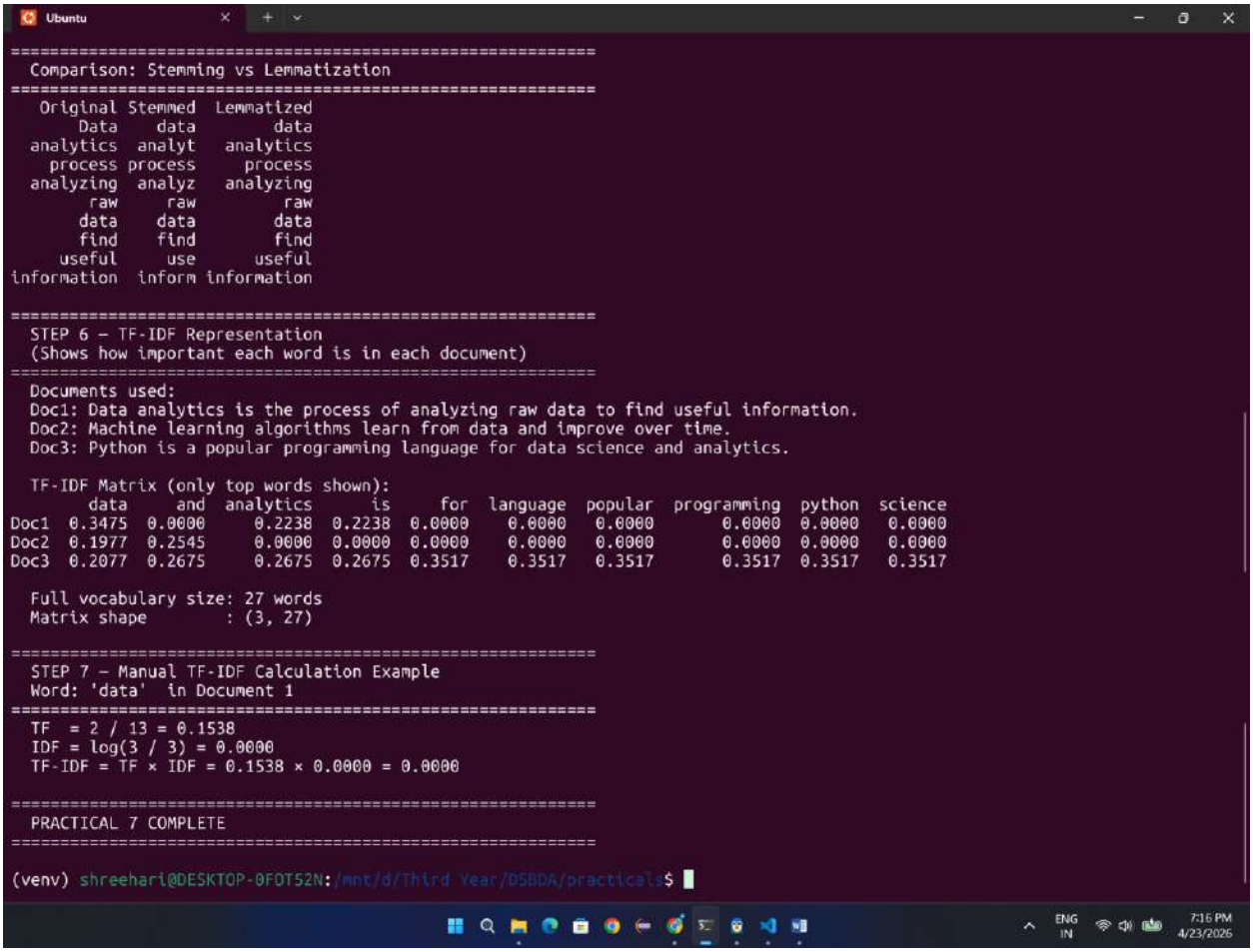
```
for word, tag in pos_tags:
    print(f"    {word:15s} → {tag}")

stop_words = set(stopwords.words('english'))
filtered_tokens = [w for w in tokens if w.lower() not in stop_words and w.isalpha()]

print(f"\n{DIVIDER}")
print("  STEP 3 — Stop Words Removal")
print(f"  Stop words removed (like: is, the, of, to, a ...)")
print(DIVIDER)
print(f"  Before ({len(tokens)}) tokens: {tokens}")
print(f"\n  After  ({len(filtered_tokens)}) tokens: {filtered_tokens}")

ps = PorterStemmer()
stemmed = [ps.stem(w) for w in filtered_tokens]

print(f"\n{DIVIDER}")
print("  STEP 4 — Stemming (reduces to root, may not be real word)")
print(DIVIDER)
for orig, stem in zip(filtered_tokens, stemmed):
    print(f"    {orig:15s} → {stem}")
lm = WordNetLemmatizer()
lemmatized = [lm.lemmatize(w.lower()) for w in filtered_tokens]

print(f"\n{DIVIDER}")
print("  STEP 5 — Lemmatization (reduces to real base word)")
print(DIVIDER)
for orig, lemma in zip(filtered_tokens, lemmatized):
    print(f"    {orig:15s} → {lemma}")
print(f"\n{DIVIDER}")
print("  Comparison: Stemming vs Lemmatization")
print(DIVIDER)
comp = pd.DataFrame({
    'Original':    filtered_tokens,
    'Stemmed':     stemmed,
    'Lemmatized':  lemmatized,
})
print(comp.to_string(index=False))

print(f"\n{DIVIDER}")
print("  STEP 6 — TF-IDF Representation")
```

```
print("  (Shows how important each word is in each document)")
print(DIVIDER)
print("  Documents used:")
for i, d in enumerate(docs):
    print(f"  Doc{i+1}: {d}")

vectorizer = TfidfVectorizer()
tfidf_matrix = vectorizer.fit_transform(docs)

feature_names = vectorizer.get_feature_names_out()
tfidf_df = pd.DataFrame(
    tfidf_matrix.toarray(),
    columns=feature_names,
    index=[f'Doc{i+1}' for i in range(len(docs))]
)

print(f"\n  TF-IDF Matrix (only top words shown):")
# Show top 8 words by total TF-IDF score
top_cols = tfidf_df.sum().nlargest(10).index
print(tfidf_df[top_cols].round(4))

print(f"\n  Full vocabulary size: {len(feature_names)} words")
print(f"  Matrix shape       : {tfidf_matrix.shape}")

print(f"\n{DIVIDER}")
print("  STEP 7 — Manual TF-IDF Calculation Example")
print("  Word: 'data'  in Document 1")
print(DIVIDER)
word = 'data'
doc1_words  = docs[0].lower().split()
tf = doc1_words.count(word) / len(doc1_words)
docs_with_word = sum(1 for d in docs if word in d.lower())
import math
idf = math.log(len(docs) / docs_with_word)
print(f"  TF  = {doc1_words.count(word)} / {len(doc1_words)} = {tf:.4f}")
print(f"  IDF = log({len(docs)} / {docs_with_word}) = {idf:.4f}")
print(f"  TF-IDF = TF × IDF = {tf:.4f} × {idf:.4f} = {tf*idf:.4f}")

print(f"\n{'='*60}")
print("  PRACTICAL 7 COMPLETE")
print(f"{'='*60}\n")
```

➢ Output


Ubuntu
(venv) shreehari@DESKTOP-0FOT52N:/mnt/d/Third Year/DSBDA/practicals$ python3 prac_7.py
Downloading NLTK data (only needed once)...
STEP 0 – Sample Text
Text: Data analytics is the process of analyzing raw data to find useful information.
STEP 1 – Tokenization (splitting text into words)
Tokens (14):
['Data', 'analytics', 'is', 'the', 'process', 'of', 'analyzing', 'raw', 'data', 'to', 'find', 'useful', 'information', '.'
]
STEP 2 – POS Tagging (Part of Speech)
NN=Noun VBZ=Verb JJ=Adjective IN=Preposition DT=Determiner
Data → NNP
analytics → NNS
is → VBZ
the → DT
process → NN
of → IN
analyzing → VBG
raw → JJ
data → NNS
to → TO
find → VB
useful → JJ
information → NN
ENG
IN
7:15 PM
4/23/2026


Ubuntu
STEP 3 – Stop Words Removal
Stop words removed (like: is, the, of, to, a ...)
Before (14) tokens: ['Data', 'analytics', 'is', 'the', 'process', 'of', 'analyzing', 'raw', 'data', 'to', 'find', 'useful'
, 'information', '.']
After (9) tokens: ['Data', 'analytics', 'process', 'analyzing', 'raw', 'data', 'find', 'useful', 'information']
STEP 4 – Stemming (reduces to root, may not be real word)
Data → data
analytics → analyt
process → process
analyzing → analyz
raw → raw
data → data
find → find
useful → use
information → inform
STEP 5 – Lemmatization (reduces to real base word)
Data → data
analytics → analytics
process → process
analyzing → analyzing
raw → raw
data → data
find → find
ENG
IN
7:15 PM
4/23/2026


Ubuntu
Comparison: Stemming vs Lemmatization
Original Stemmed Lemmatized
Data data data
analytics analyt analytics
process process process
analyzing analyz analyzing
raw raw raw
data data data
find find find
useful use useful
information inform information
STEP 6 – TF-IDF Representation
(Shows how important each word is in each document)
Documents used:
Doc1: Data analytics is the process of analyzing raw data to find useful information.
Doc2: Machine learning algorithms learn from data and improve over time.
Doc3: Python is a popular programming language for data science and analytics.
TF-IDF Matrix (only top words shown):
data and analytics is for language popular programming python science
Doc1 0.3475 0.0000 0.2238 0.2238 0.0000 0.0000 0.0000 0.0000 0.0000 0.0000
Doc2 0.1977 0.2545 0.0000 0.0000 0.0000 0.0000 0.0000 0.0000 0.0000 0.0000
Doc3 0.2077 0.2675 0.2675 0.2675 0.3517 0.3517 0.3517 0.3517 0.3517 0.3517
Full vocabulary size: 27 words
Matrix shape : (3, 27)
STEP 7 – Manual TF-IDF Calculation Example
Word: 'data' in Document 1
TF = 2 / 13 = 0.1538
IDF = log(3 / 3) = 0.0000
TF-IDF = TF × IDF = 0.1538 × 0.0000 = 0.0000
PRACTICAL 7 COMPLETE
(venv) shreehari@DESKTOP-0FOT52N:/mnt/d/Third Year/DSBDA/practicals$
ENG
IN
7:16 PM
4/23/2026

PRACTICAL 8

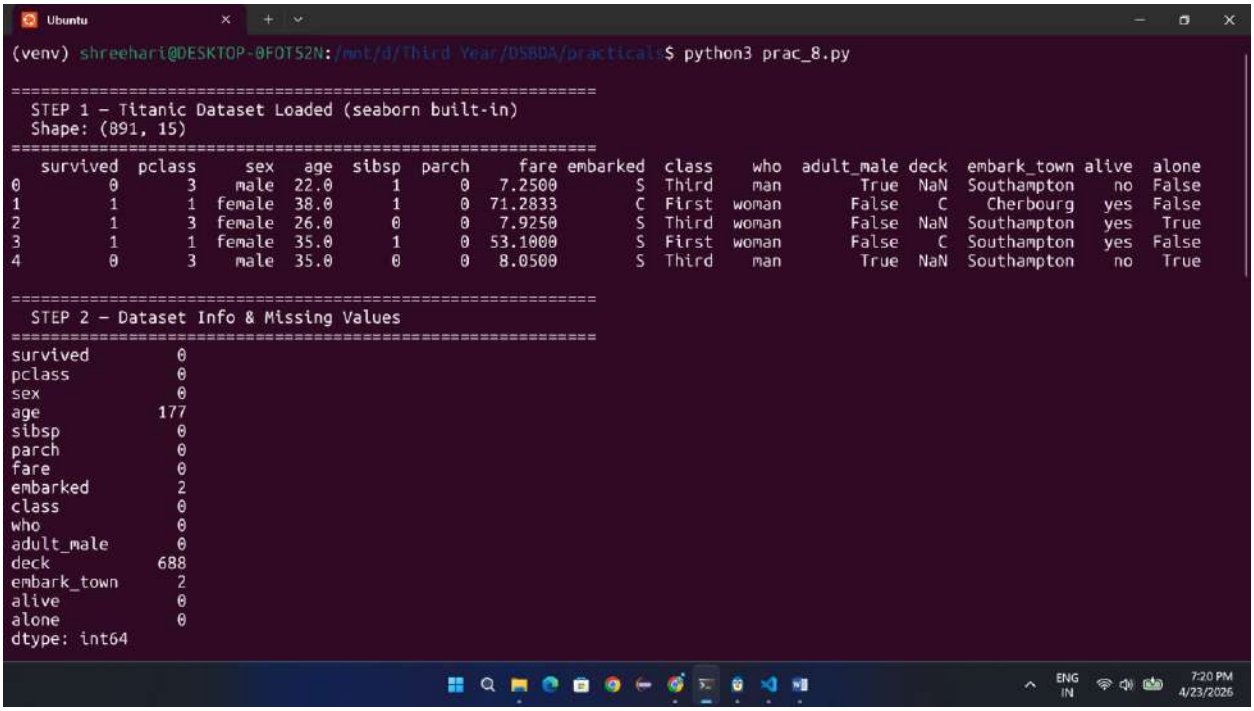
- Source Code

```
import seaborn as sns
import matplotlib.pyplot as plt
import pandas as pd

DIVIDER = "=" * 60

df = sns.load_dataset('titanic')

print(f"\n{DIVIDER}")
print("  STEP 1 — Titanic Dataset Loaded (seaborn built-in)")
print(f"  Shape: {df.shape}")
print(DIVIDER)
print(df.head())

print(f"\n{DIVIDER}")
print("  STEP 2 — Dataset Info & Missing Values")
print(DIVIDER)
print(df.isnull().sum())

print(f"\n{DIVIDER}")
print("  STEP 3 — Statistical Summary")
print(DIVIDER)
print(df.describe())

plt.figure(figsize=(6, 4))
sns.countplot(x='survived', data=df, palette='Set2')
plt.title("Plot 1: Survival Count (0=No, 1=Yes)")
plt.xlabel("Survived")
plt.ylabel("Count")
plt.tight_layout()
plt.savefig("plot1_survival_count.png")
plt.close()
print(f"\n  [Saved] Plot 1 → plot1_survival_count.png")
print("  Shows: How many passengers survived vs did not survive.")

plt.figure(figsize=(6, 4))
sns.countplot(x='survived', hue='sex', data=df, palette='Set1')
```

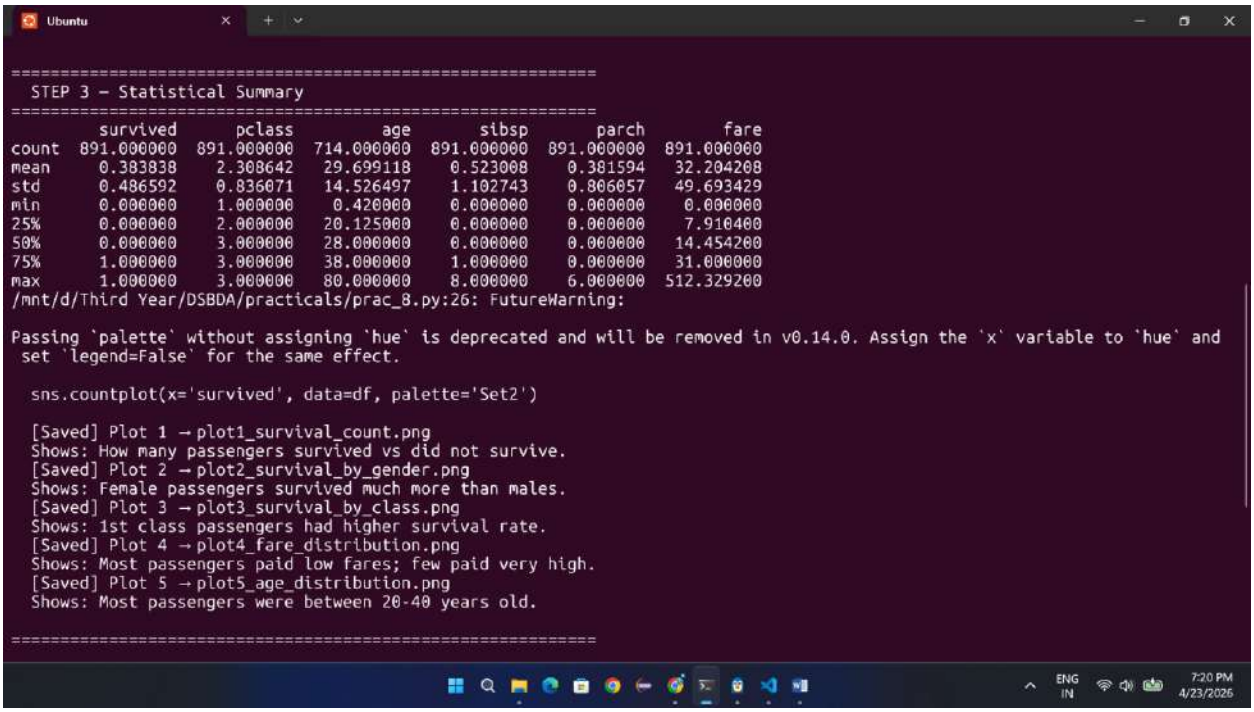
```
plt.title("Plot 2: Survival by Gender")
plt.xlabel("Survived")
plt.ylabel("Count")
plt.legend(title='Gender')
plt.tight_layout()
plt.savefig("plot2_survival_by_gender.png")
plt.close()
print(f"  [Saved] Plot 2 → plot2_survival_by_gender.png")
print("  Shows: Female passengers survived much more than males.")

plt.figure(figsize=(6, 4))
sns.countplot(x='survived', hue='pclass', data=df, palette='Set3')
plt.title("Plot 3: Survival by Passenger Class")
plt.xlabel("Survived")
plt.ylabel("Count")
plt.legend(title='Pclass')
plt.tight_layout()
plt.savefig("plot3_survival_by_class.png")
plt.close()
print(f"  [Saved] Plot 3 → plot3_survival_by_class.png")
print("  Shows: 1st class passengers had higher survival rate.")

plt.figure(figsize=(7, 4))
sns.histplot(df['fare'].dropna(), bins=30, kde=True, color='steelblue')
plt.title("Plot 4: Fare Distribution")
plt.xlabel("Fare (ticket price)")
plt.ylabel("Frequency")
plt.tight_layout()
plt.savefig("plot4_fare_distribution.png")
plt.close()
print(f"  [Saved] Plot 4 → plot4_fare_distribution.png")
print("  Shows: Most passengers paid low fares; few paid very high.")

plt.figure(figsize=(7, 4))
sns.histplot(df['age'].dropna(), bins=20, kde=True, color='coral')
plt.title("Plot 5: Age Distribution of Passengers")
plt.xlabel("Age")
plt.ylabel("Frequency")
plt.tight_layout()
plt.savefig("plot5_age_distribution.png")
plt.close()
```

```
print(f"  [Saved] Plot 5 → plot5_age_distribution.png")
print("  Shows: Most passengers were between 20-40 years old.")

print(f"\n{DIVIDER}")
print("  OBSERVATIONS")
print(DIVIDER)
survived     = df['survived'].value_counts()
gender_surv  = df.groupby('sex')['survived'].mean()
class_surv   = df.groupby('pclass')['survived'].mean()

print(f"  Total Survived     : {survived[1]}  |  Not Survived: {survived[0]}")
print(f"\n  Survival Rate by Gender:")
for g, r in gender_surv.items():
   print(f"    {g:8s}: {r*100:.1f}%")
print(f"\n  Survival Rate by Class:")
for c, r in class_surv.items():
   print(f"    Class {c}: {r*100:.1f}%")

print(f"\n{'='*60}")
print("  PRACTICAL 8 COMPLETE")
print("  5 plots saved as PNG in current directory")
print(f"{'='*60}\n")
```

➢ Output

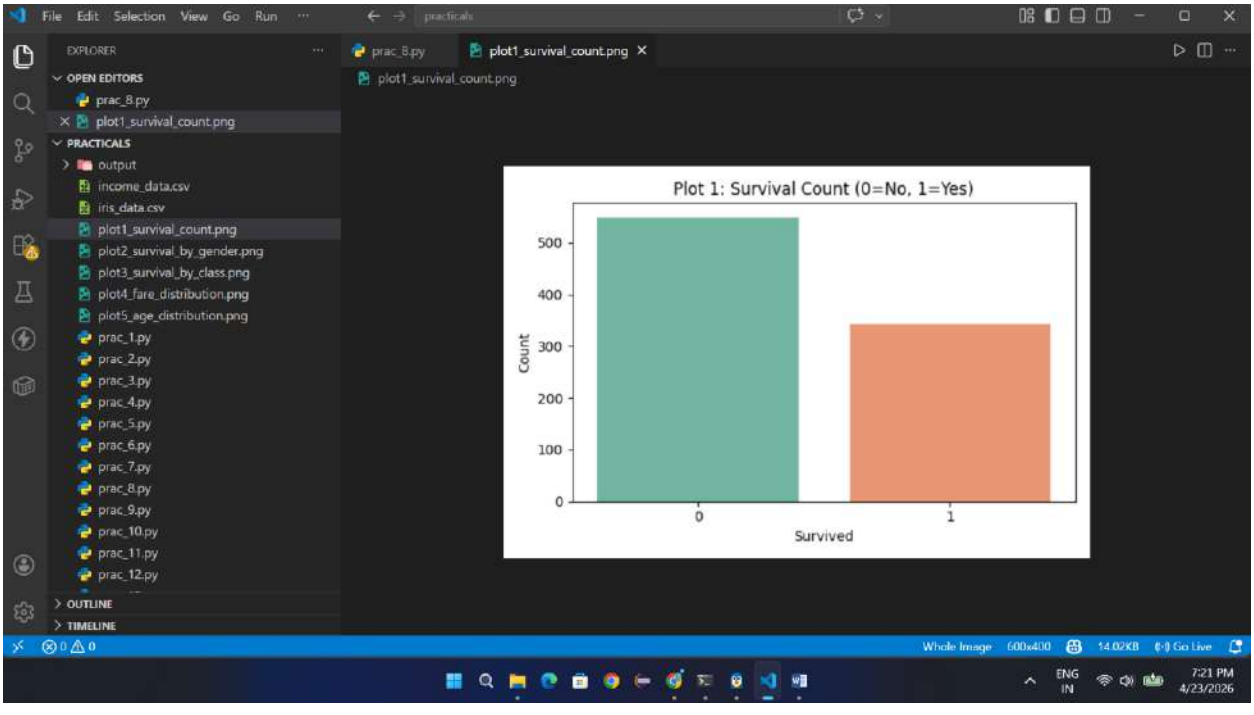


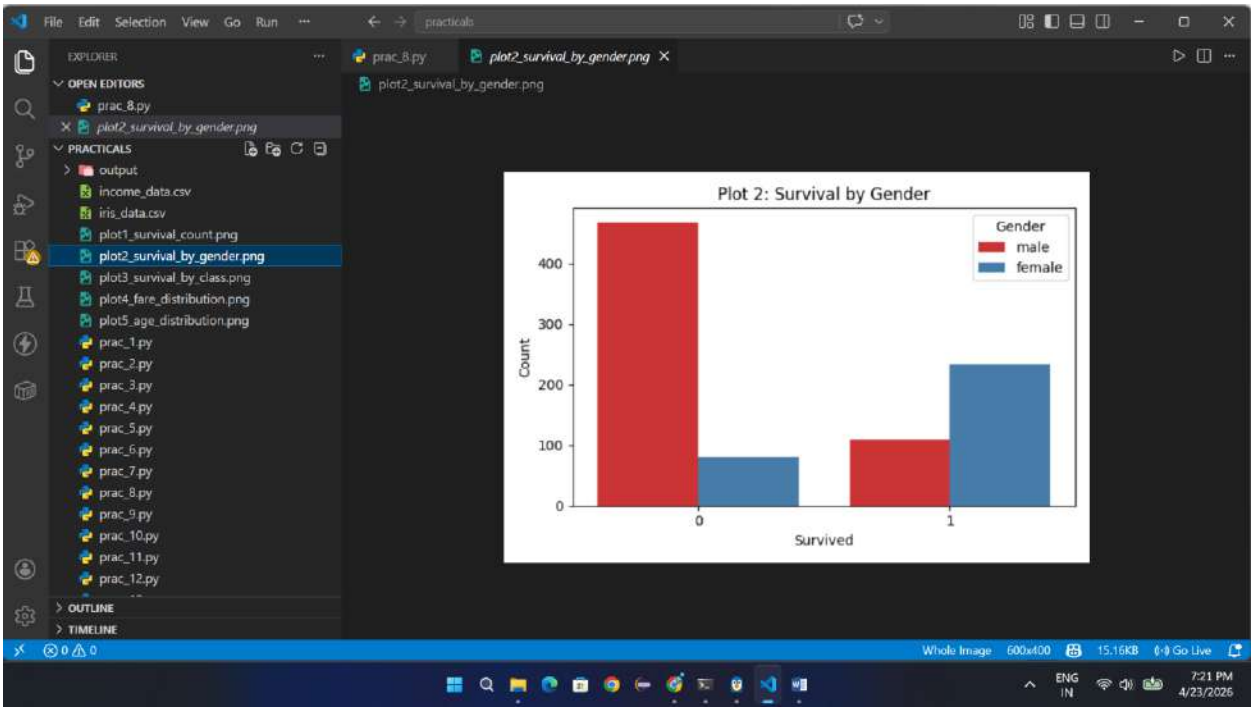
```
Ubuntu
(venv) shreehari@DESKTOP-0FOT52N:/mnt/d/Third Year/DSBDA/practicals$ python3 prac_8.py

============================================================
  STEP 1 — Titanic Dataset Loaded (seaborn built-in)
  Shape: (891, 15)
============================================================
   survived  pclass     sex   age  sibsp  parch     fare embarked  class    who  adult_male deck  embark_town alive  alone
0         0       3    male  22.0      1      0   7.2500        S  Third    man        True  NaN  Southampton    no  False
1         1       1  female  38.0      1      0  71.2833        C  First  woman       False    C    Cherbourg   yes  False
2         1       3  female  26.0      0      0   7.9250        S  Third  woman       False  NaN  Southampton   yes   True
3         1       1  female  35.0      1      0  53.1000        S  First  woman       False    C  Southampton   yes  False
4         0       3    male  35.0      0      0   8.0500        S  Third    man        True  NaN  Southampton    no   True

============================================================
  STEP 2 — Dataset Info & Missing Values
============================================================
survived         0
pclass           0
sex              0
age            177
sibsp            0
parch            0
fare             0
embarked         2
class            0
who              0
adult_male       0
deck           688
embark_town      2
alive            0
alone            0
dtype: int64
```

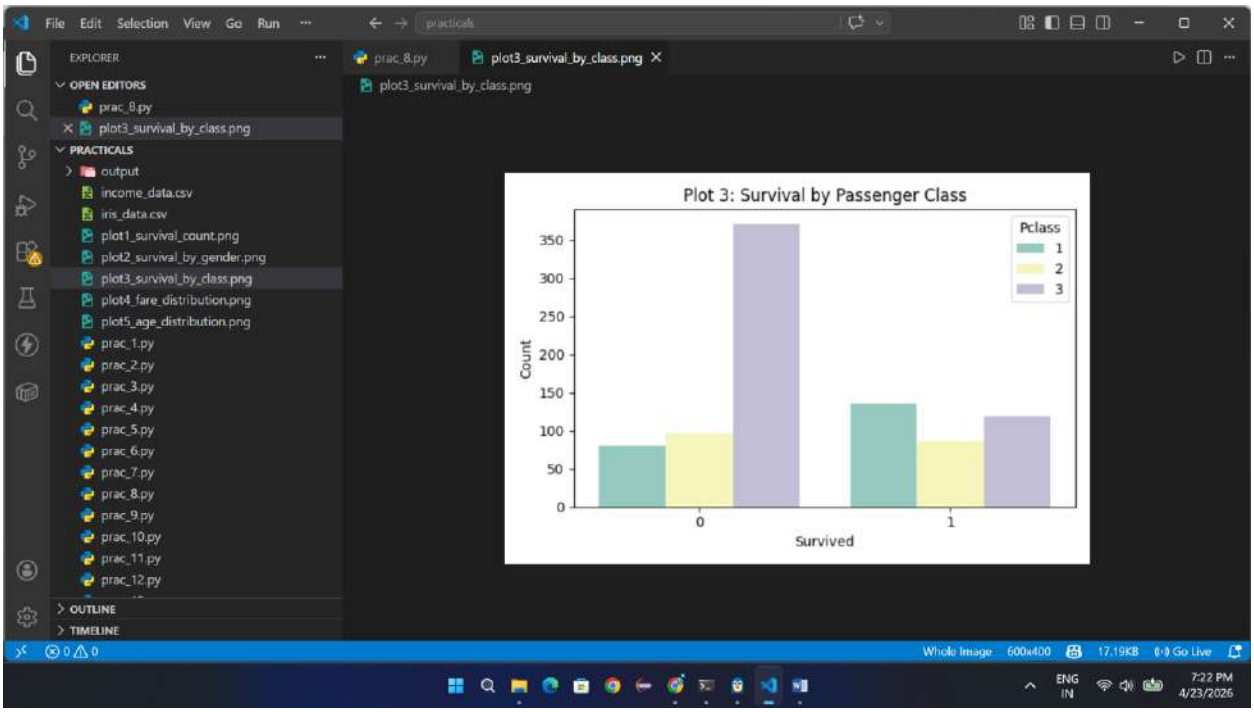


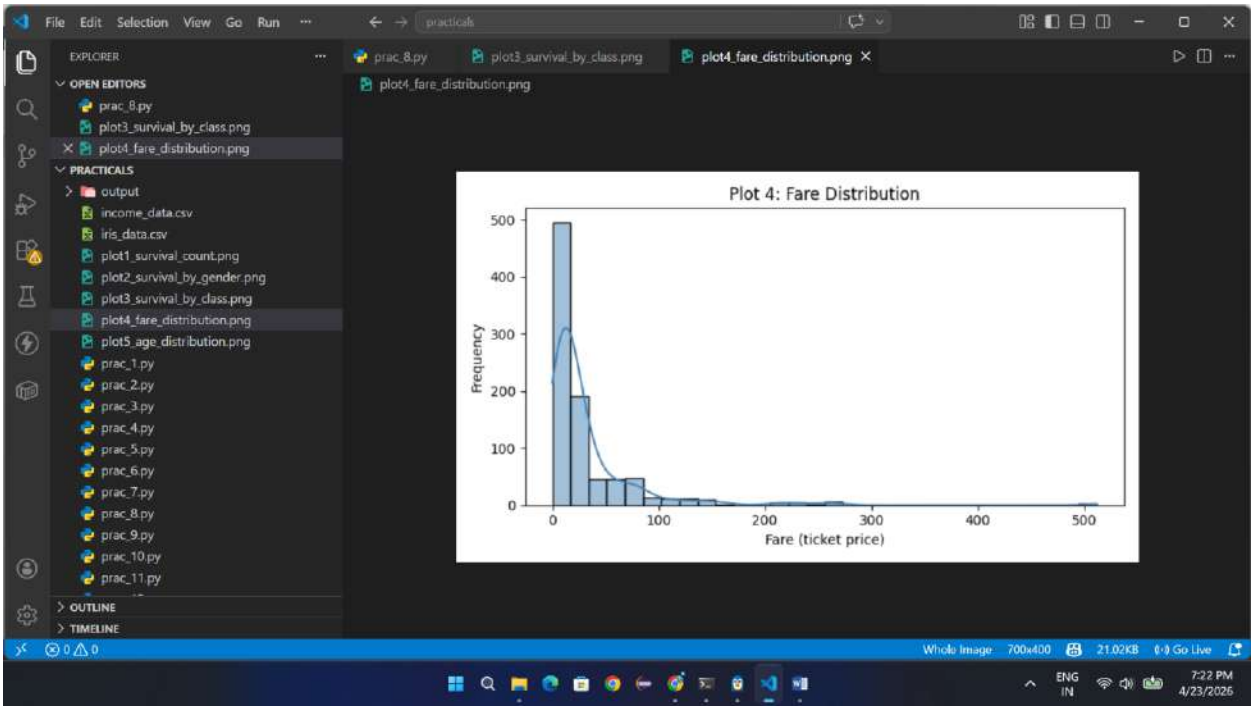
```
Ubuntu

============================================================
  STEP 3 — Statistical Summary
============================================================
         survived      pclass         age       sibsp       parch        fare
count  891.000000  891.000000  714.000000  891.000000  891.000000  891.000000
mean     0.383838    2.308642   29.699118    0.523008    0.381594   32.204208
std      0.486592    0.836071   14.526497    1.102743    0.806057   49.693429
min      0.000000    1.000000    0.420000    0.000000    0.000000    0.000000
25%      0.000000    2.000000   20.125000    0.000000    0.000000    7.910400
50%      0.000000    3.000000   28.000000    0.000000    0.000000   14.454200
75%      1.000000    3.000000   38.000000    1.000000    0.000000   31.000000
max      1.000000    3.000000   80.000000    8.000000    6.000000  512.329200
/mnt/d/Third Year/DSBDA/practicals/prac_8.py:26: FutureWarning:

Passing `palette` without assigning `hue` is deprecated and will be removed in v0.14.0. Assign the `x` variable to `hue` and
 set `legend=False` for the same effect.

  sns.countplot(x='survived', data=df, palette='Set2')

  [Saved] Plot 1 → plot1_survival_count.png
  Shows: How many passengers survived vs did not survive.
  [Saved] Plot 2 → plot2_survival_by_gender.png
  Shows: Female passengers survived much more than males.
  [Saved] Plot 3 → plot3_survival_by_class.png
  Shows: 1st class passengers had higher survival rate.
  [Saved] Plot 4 → plot4_fare_distribution.png
  Shows: Most passengers paid low fares; few paid very high.
  [Saved] Plot 5 → plot5_age_distribution.png
  Shows: Most passengers were between 20-40 years old.

============================================================
```

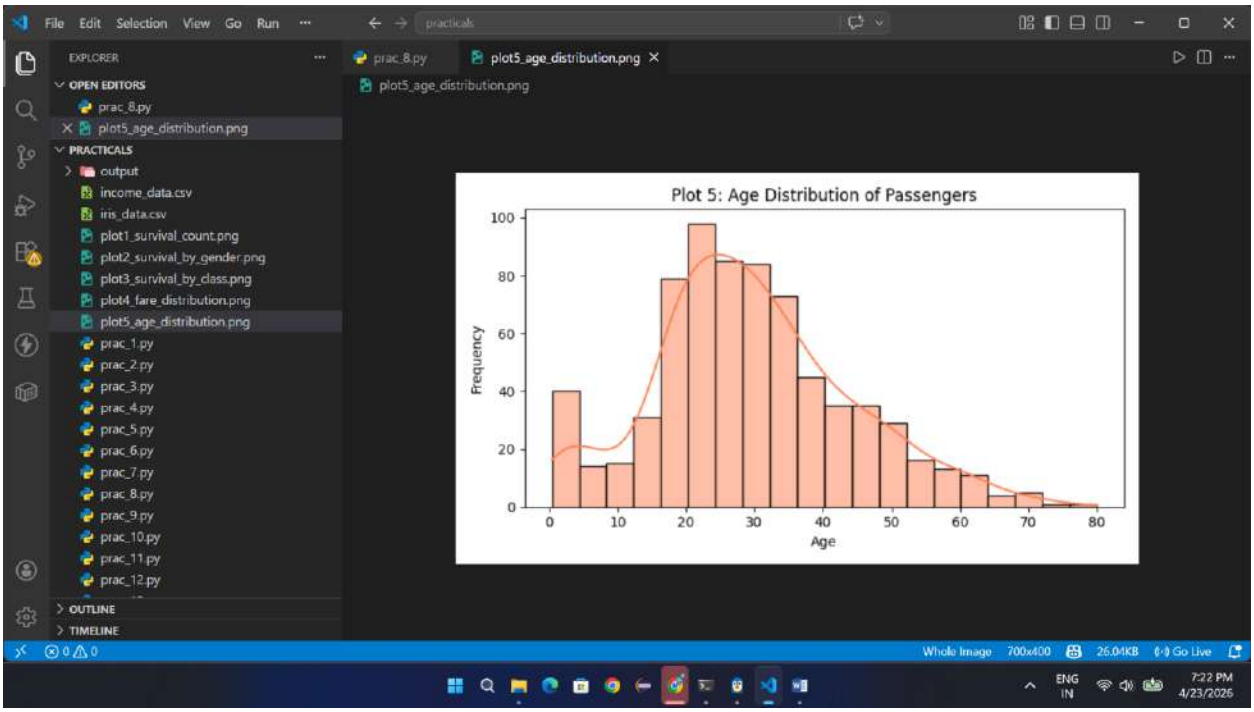
```
Ubuntu

  [Saved] Plot 1 → plot1_survival_count.png
  Shows: How many passengers survived vs did not survive.
  [Saved] Plot 2 → plot2_survival_by_gender.png
  Shows: Female passengers survived much more than males.
  [Saved] Plot 3 → plot3_survival_by_class.png
  Shows: 1st class passengers had higher survival rate.
  [Saved] Plot 4 → plot4_fare_distribution.png
  Shows: Most passengers paid low fares; few paid very high.
  [Saved] Plot 5 → plot5_age_distribution.png
  Shows: Most passengers were between 20-40 years old.

============================================================
  OBSERVATIONS
============================================================
  Total Survived     : 342  |  Not Survived: 549

  Survival Rate by Gender:
    female  : 74.2%
    male    : 18.9%

  Survival Rate by Class:
    Class 1: 63.0%
    Class 2: 47.3%
    Class 3: 24.2%

============================================================
  PRACTICAL 8 COMPLETE
  5 plots saved as PNG in current directory
============================================================

(venv) shreehari@DESKTOP-0FOT52N:/mnt/d/Third Year/DSBDA/practicals$
```




File Edit Selection View Go Run
practicals
EXPLORER
OPEN EDITORS
prac_8.py
plot2_survival_by_gender.png
PRACTICALS
output
income_data.csv
iris_data.csv
plot1_survival_count.png
plot2_survival_by_gender.png
plot3_survival_by_class.png
plot4_fare_distribution.png
plot5_age_distribution.png
prac_1.py
prac_2.py
prac_3.py
prac_4.py
prac_5.py
prac_6.py
prac_7.py
prac_8.py
prac_9.py
prac_10.py
prac_11.py
prac_12.py
OUTLINE
TIMELINE
prac_8.py
plot2_survival_by_gender.png
plot2_survival_by_gender.png
Plot 2: Survival by Gender
Gender
male
female
Count
Survived
0
100
200
300
400
0
1
Whole Image 600x400 15.16KB Go Live
ENG
IN
7:21 PM
4/23/2026


File Edit Selection View Go Run
practicals
EXPLORER
OPEN EDITORS
prac_8.py
plot3_survival_by_class.png
PRACTICALS
output
income_data.csv
iris_data.csv
plot1_survival_count.png
plot2_survival_by_gender.png
plot3_survival_by_class.png
plot4_fare_distribution.png
plot5_age_distribution.png
prac_1.py
prac_2.py
prac_3.py
prac_4.py
prac_5.py
prac_6.py
prac_7.py
prac_8.py
prac_9.py
prac_10.py
prac_11.py
prac_12.py
OUTLINE
TIMELINE
prac_8.py
plot3_survival_by_class.png
plot3_survival_by_class.png
Plot 3: Survival by Passenger Class
Pclass
1
2
3
Count
Survived
0
50
100
150
200
250
300
350
0
1
Whole Image 600x400 17.19KB Go Live
ENG
IN
7:22 PM
4/23/2026


File Edit Selection View Go Run
practicals
EXPLORER
OPEN EDITORS
prac_8.py
plot3_survival_by_class.png
plot4_fare_distribution.png
PRACTICALS
output
income_data.csv
iris_data.csv
plot1_survival_count.png
plot2_survival_by_gender.png
plot3_survival_by_class.png
plot4_fare_distribution.png
plot5_age_distribution.png
prac_1.py
prac_2.py
prac_3.py
prac_4.py
prac_5.py
prac_6.py
prac_7.py
prac_8.py
prac_9.py
prac_10.py
prac_11.py
OUTLINE
TIMELINE
Plot 4: Fare Distribution
Frequency
Fare (ticket price)
0
100
200
300
400
500
Whole Image 700x400 21.02KB Go Live
ENG IN
7:22 PM 4/23/2026


File Edit Selection View Go Run
practicals
EXPLORER
OPEN EDITORS
prac_8.py
plot5_age_distribution.png
PRACTICALS
output
income_data.csv
iris_data.csv
plot1_survival_count.png
plot2_survival_by_gender.png
plot3_survival_by_class.png
plot4_fare_distribution.png
plot5_age_distribution.png
prac_1.py
prac_2.py
prac_3.py
prac_4.py
prac_5.py
prac_6.py
prac_7.py
prac_8.py
prac_9.py
prac_10.py
prac_11.py
prac_12.py
OUTLINE
TIMELINE
Plot 5: Age Distribution of Passengers
Frequency
Age
0
20
40
60
80
100
0
10
20
30
40
50
60
70
80
Whole Image 700x400 26.04KB Go Live
ENG IN
7:22 PM 4/23/2026

PRACTICAL 9

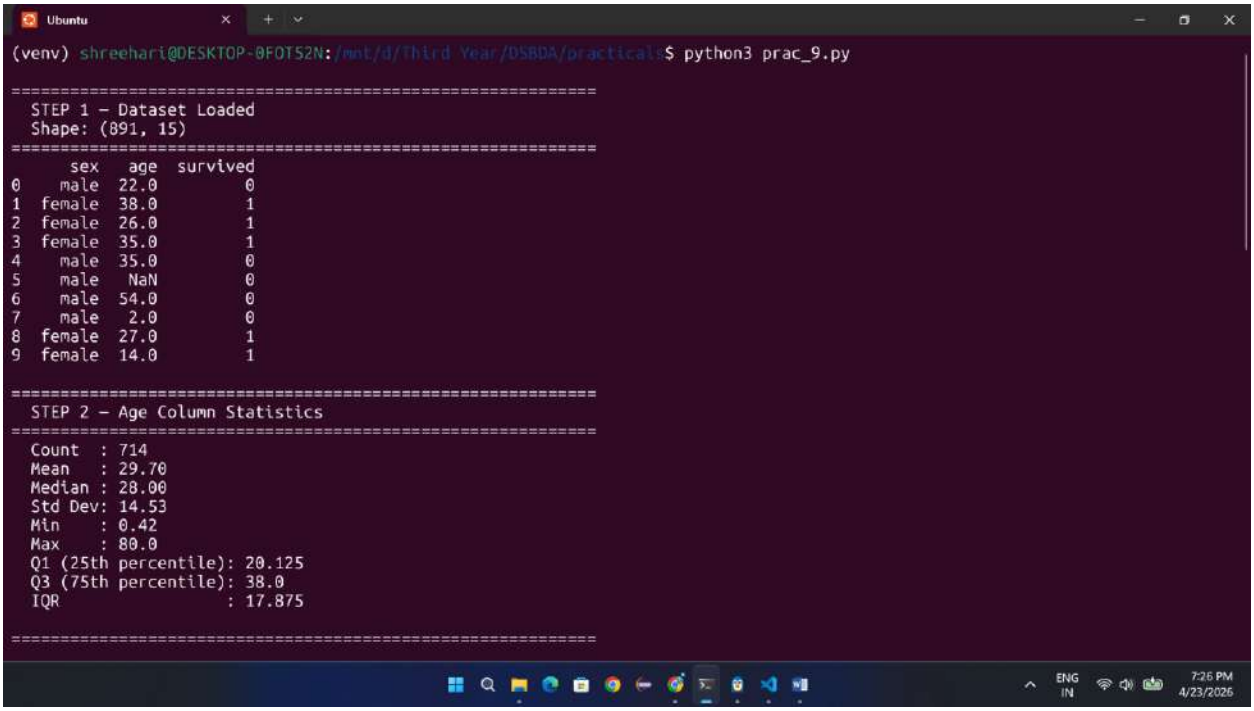
- Source Code

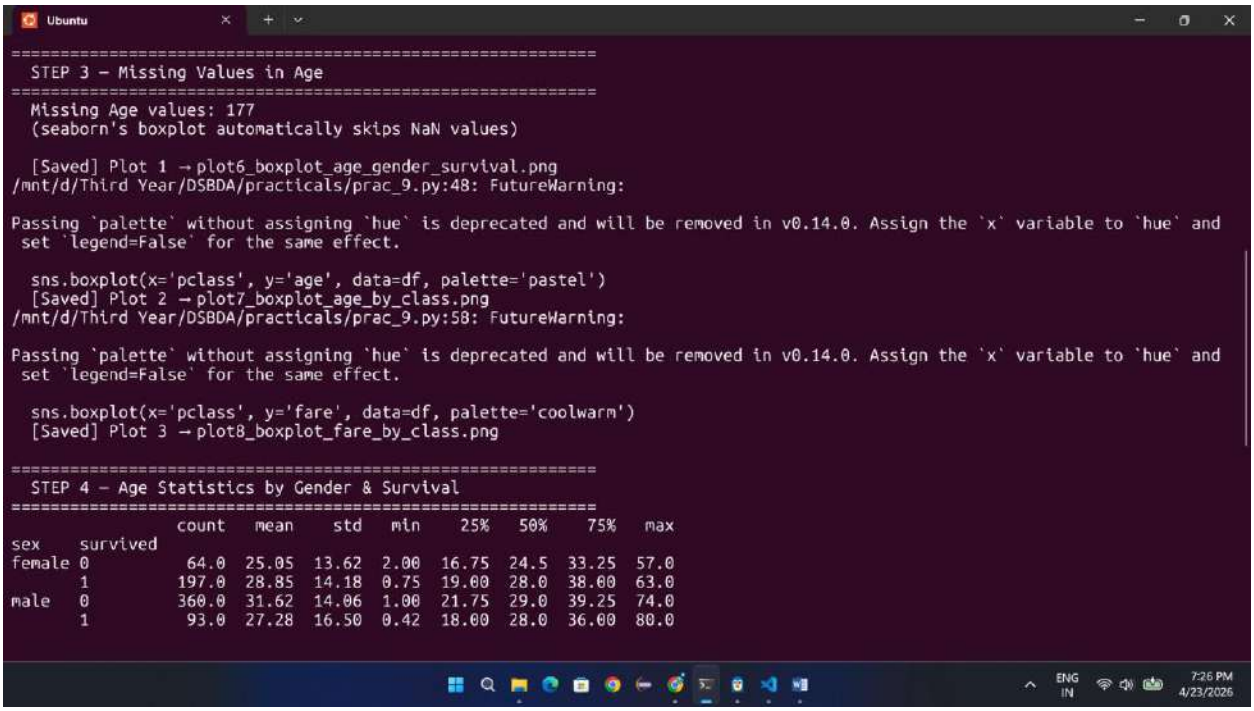
```
import seaborn as sns
import matplotlib.pyplot as plt
import pandas as pd

DIVIDER = "=" * 60

df = sns.load_dataset('titanic')

print(f"\n{DIVIDER}")
print("  STEP 1 — Dataset Loaded")
print(f"  Shape: {df.shape}")
print(DIVIDER)
print(df[['sex', 'age', 'survived']].head(10))

print(f"\n{DIVIDER}")
print("  STEP 2 — Age Column Statistics")
print(DIVIDER)
print(f"  Count  : {df['age'].count()}")
print(f"  Mean   : {df['age'].mean():.2f}")
print(f"  Median : {df['age'].median():.2f}")
print(f"  Std Dev: {df['age'].std():.2f}")
print(f"  Min    : {df['age'].min()}")
print(f"  Max    : {df['age'].max()}")
q1 = df['age'].quantile(0.25)
q3 = df['age'].quantile(0.75)
print(f"  Q1 (25th percentile): {q1}")
print(f"  Q3 (75th percentile): {q3}")
print(f"  IQR                 : {q3 - q1}")

print(f"\n{DIVIDER}")
print("  STEP 3 — Missing Values in Age")
print(DIVIDER)
print(f"  Missing Age values: {df['age'].isna().sum()}")
print("  (seaborn's boxplot automatically skips NaN values)")

plt.figure(figsize=(8, 5))
```

```
sns.boxplot(x='sex', y='age', hue='survived', data=df, palette='Set2')
plt.title("Plot 1: Age Distribution by Gender and Survival")
plt.xlabel("Gender")
plt.ylabel("Age")
plt.legend(title='Survived', labels=['No (0)', 'Yes (1)'])
plt.tight_layout()
plt.savefig("plot6_boxplot_age_gender_survival.png")
plt.close()
print(f"\n  [Saved] Plot 1 → plot6_boxplot_age_gender_survival.png")

plt.figure(figsize=(7, 5))
sns.boxplot(x='pclass', y='age', data=df, palette='pastel')
plt.title("Plot 2: Age Distribution by Passenger Class")
plt.xlabel("Passenger Class")
plt.ylabel("Age")
plt.tight_layout()
plt.savefig("plot7_boxplot_age_by_class.png")
plt.close()
print(f"  [Saved] Plot 2 → plot7_boxplot_age_by_class.png")

plt.figure(figsize=(7, 5))
sns.boxplot(x='pclass', y='fare', data=df, palette='coolwarm')
plt.title("Plot 3: Fare Distribution by Passenger Class")
plt.xlabel("Passenger Class")
plt.ylabel("Fare")
plt.tight_layout()
plt.savefig("plot8_boxplot_fare_by_class.png")
plt.close()
print(f"  [Saved] Plot 3 → plot8_boxplot_fare_by_class.png")

print(f"\n{DIVIDER}")
print("  STEP 4 — Age Statistics by Gender & Survival")
print(DIVIDER)
grouped = df.groupby(['sex', 'survived'])['age'].describe().round(2)
print(grouped)

print(f"\n{DIVIDER}")
print("  OBSERVATIONS (from Box Plots)")
print(DIVIDER)
print("""
  1. GENDER & AGE:
```

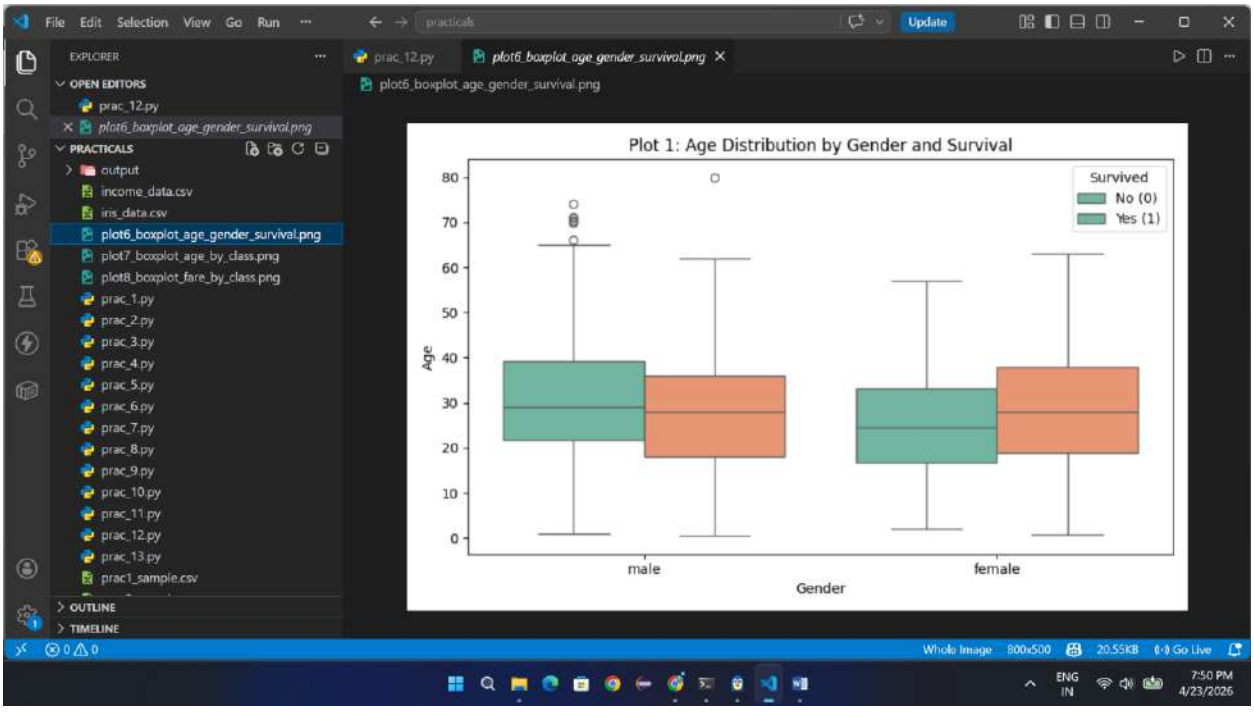
```
   - Male passengers have a wider age range than females.
   - Median age for males is slightly higher (~30) than females (~28).

 2. SURVIVAL & GENDER:
   - Among survivors, females tend to be slightly younger.
   - Non-surviving males have a higher median age.
   - This supports the 'women and children first' evacuation pattern.

 3. AGE & CLASS:
   - 1st class passengers are generally older (median ~37).
   - 3rd class passengers are younger (median ~25).

 4. FARE & CLASS:
   - 1st class fares are much higher and widely spread.
   - 2nd and 3rd class fares are low and tightly clustered.

 5. OUTLIERS:
   - Some passengers aged above 70 appear as outliers.
   - Very high fares in 1st class show as outliers.
""")

print(f"{'='*60}")
print("  PRACTICAL 9 COMPLETE")
print("  3 plots saved as PNG in current directory"
print(f"{'='*60}\n")
```

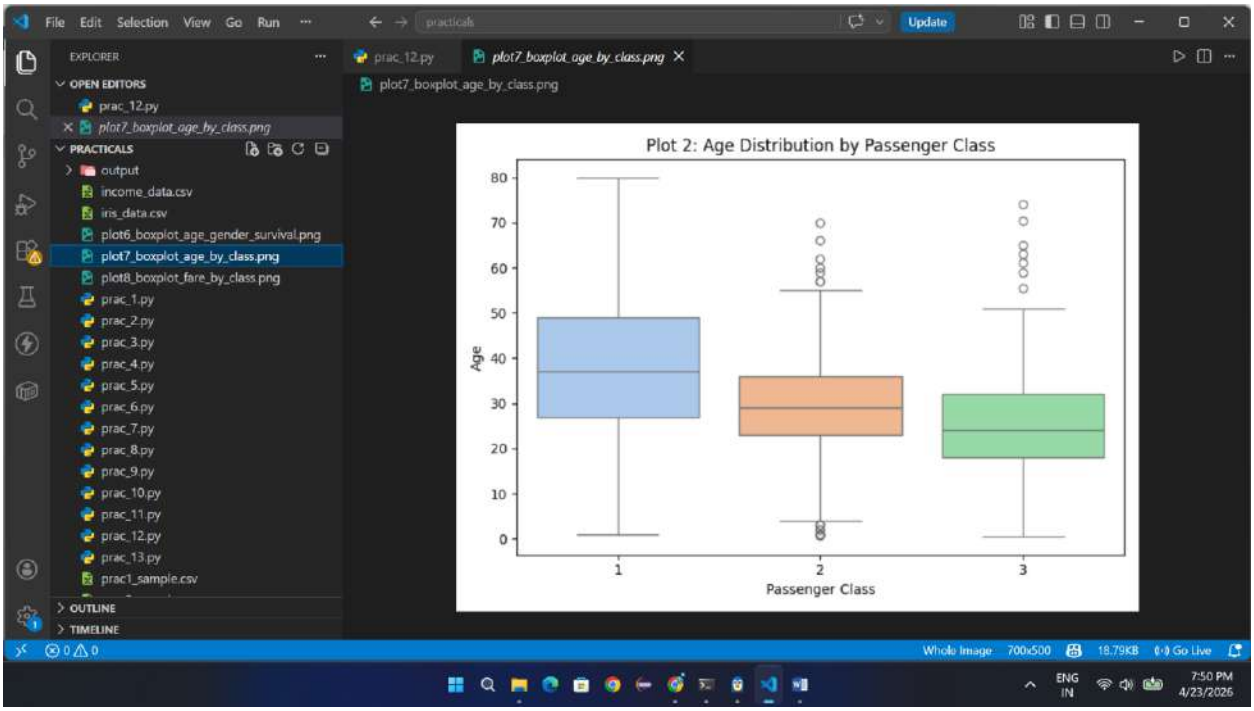
➢ Output

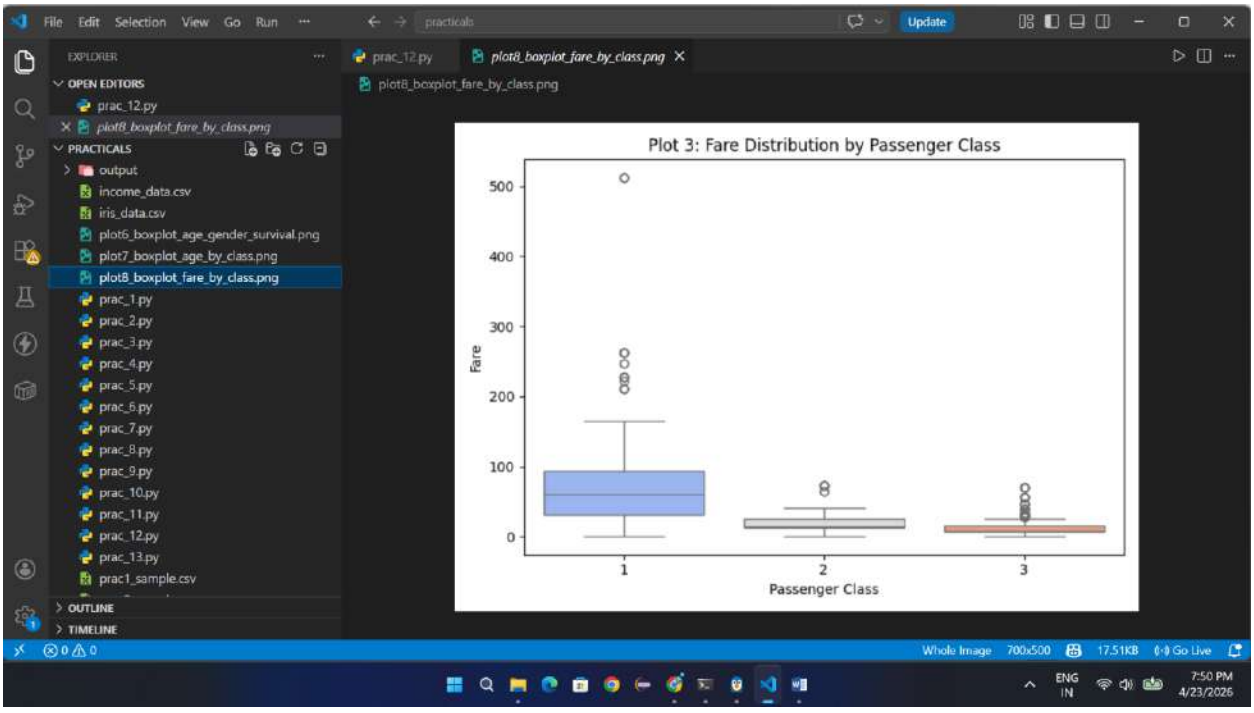

Ubuntu
(venv) shreehari@DESKTOP-0FOT52N:/mnt/d/Third Year/DSBDA/practicals$ python3 prac_9.py
==
STEP 1 — Dataset Loaded
Shape: (891, 15)
==
sex age survived
0 male 22.0 0
1 female 38.0 1
2 female 26.0 1
3 female 35.0 1
4 male 35.0 0
5 male NaN 0
6 male 54.0 0
7 male 2.0 0
8 female 27.0 1
9 female 14.0 1
==
STEP 2 — Age Column Statistics
==
Count : 714
Mean : 29.70
Median : 28.00
Std Dev: 14.53
Min : 0.42
Max : 80.0
Q1 (25th percentile): 20.125
Q3 (75th percentile): 38.0
IQR : 17.875
==
ENG IN
7:26 PM 4/23/2026


Ubuntu
==
STEP 3 — Missing Values in Age
==
Missing Age values: 177
(seaborn's boxplot automatically skips NaN values)
[Saved] Plot 1 → plot6_boxplot_age_gender_survival.png
/mnt/d/Third Year/DSBDA/practicals/prac_9.py:48: FutureWarning:
Passing `palette` without assigning `hue` is deprecated and will be removed in v0.14.0. Assign the `x` variable to `hue` and set `legend=False` for the same effect.
sns.boxplot(x='pclass', y='age', data=df, palette='pastel')
[Saved] Plot 2 → plot7_boxplot_age_by_class.png
/mnt/d/Third Year/DSBDA/practicals/prac_9.py:58: FutureWarning:
Passing `palette` without assigning `hue` is deprecated and will be removed in v0.14.0. Assign the `x` variable to `hue` and set `legend=False` for the same effect.
sns.boxplot(x='pclass', y='fare', data=df, palette='coolwarm')
[Saved] Plot 3 → plot8_boxplot_fare_by_class.png
==
STEP 4 — Age Statistics by Gender & Survival
==
count mean std min 25% 50% 75% max
sex survived
female 0 64.0 25.05 13.62 2.00 16.75 24.5 33.25 57.0
1 197.0 28.85 14.18 0.75 19.00 28.0 38.00 63.0
male 0 360.0 31.62 14.06 1.00 21.75 29.0 39.25 74.0
1 93.0 27.28 16.50 0.42 18.00 28.0 36.00 80.0
ENG IN
7:26 PM 4/23/2026

```
Ubuntu

============================================================
  OBSERVATIONS (from Box Plots)
============================================================

  1. GENDER & AGE:
     - Male passengers have a wider age range than females.
     - Median age for males is slightly higher (~30) than females (~28).

  2. SURVIVAL & GENDER:
     - Among survivors, females tend to be slightly younger.
     - Non-surviving males have a higher median age.
     - This supports the 'women and children first' evacuation pattern.

  3. AGE & CLASS:
     - 1st class passengers are generally older (median ~37).
     - 3rd class passengers are younger (median ~25).

  4. FARE & CLASS:
     - 1st class fares are much higher and widely spread.
     - 2nd and 3rd class fares are low and tightly clustered.

  5. OUTLIERS:
     - Some passengers aged above 70 appear as outliers.
     - Very high fares in 1st class show as outliers.

============================================================
  PRACTICAL 9 COMPLETE
  3 plots saved as PNG in current directory
============================================================

(venv) shreehari@DESKTOP-0FOT52N:/mnt/d/Third Year/DSBDA/practicals$
```




File Edit Selection View Go Run
practicals
Update
EXPLORER
OPEN EDITORS
prac_12.py
plot7_boxplot_age_by_class.png
PRACTICALS
output
income_data.csv
iris_data.csv
plot6_boxplot_age_gender_survival.png
plot7_boxplot_age_by_class.png
plot8_boxplot_fare_by_class.png
prac_1.py
prac_2.py
prac_3.py
prac_4.py
prac_5.py
prac_6.py
prac_7.py
prac_8.py
prac_9.py
prac_10.py
prac_11.py
prac_12.py
prac_13.py
prac1_sample.csv
OUTLINE
TIMELINE
plot7_boxplot_age_by_class.png
Plot 2: Age Distribution by Passenger Class
Age
Passenger Class
Whole Image 700x500 18.79KB Go Live
ENG IN
7:50 PM 4/23/2026


File Edit Selection View Go Run
practicals
Update
EXPLORER
OPEN EDITORS
prac_12.py
plot8_boxplot_fare_by_class.png
PRACTICALS
output
income_data.csv
iris_data.csv
plot6_boxplot_age_gender_survival.png
plot7_boxplot_age_by_class.png
plot8_boxplot_fare_by_class.png
prac_1.py
prac_2.py
prac_3.py
prac_4.py
prac_5.py
prac_6.py
prac_7.py
prac_8.py
prac_9.py
prac_10.py
prac_11.py
prac_12.py
prac_13.py
prac1_sample.csv
OUTLINE
TIMELINE
plot8_boxplot_fare_by_class.png
Plot 3: Fare Distribution by Passenger Class
Fare
Passenger Class
Whole Image 700x500 17.51KB Go Live
ENG IN
7:50 PM 4/23/2026

PRACTICAL 10

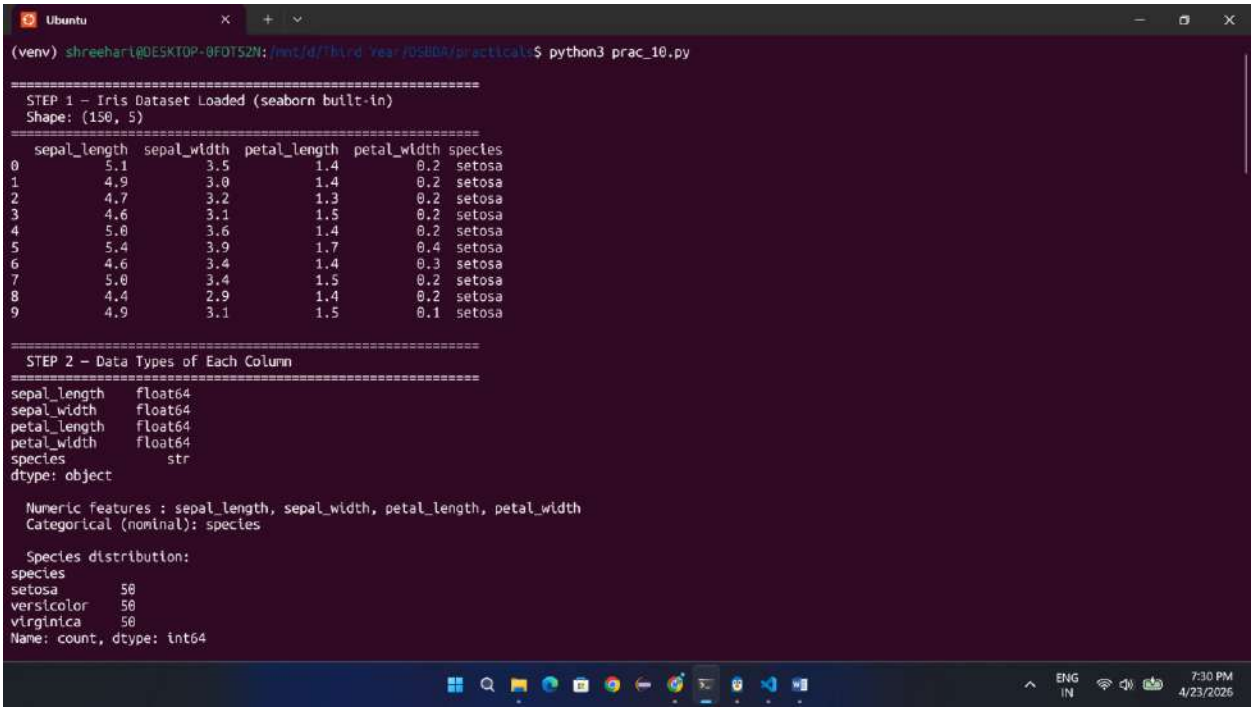
- Source Code

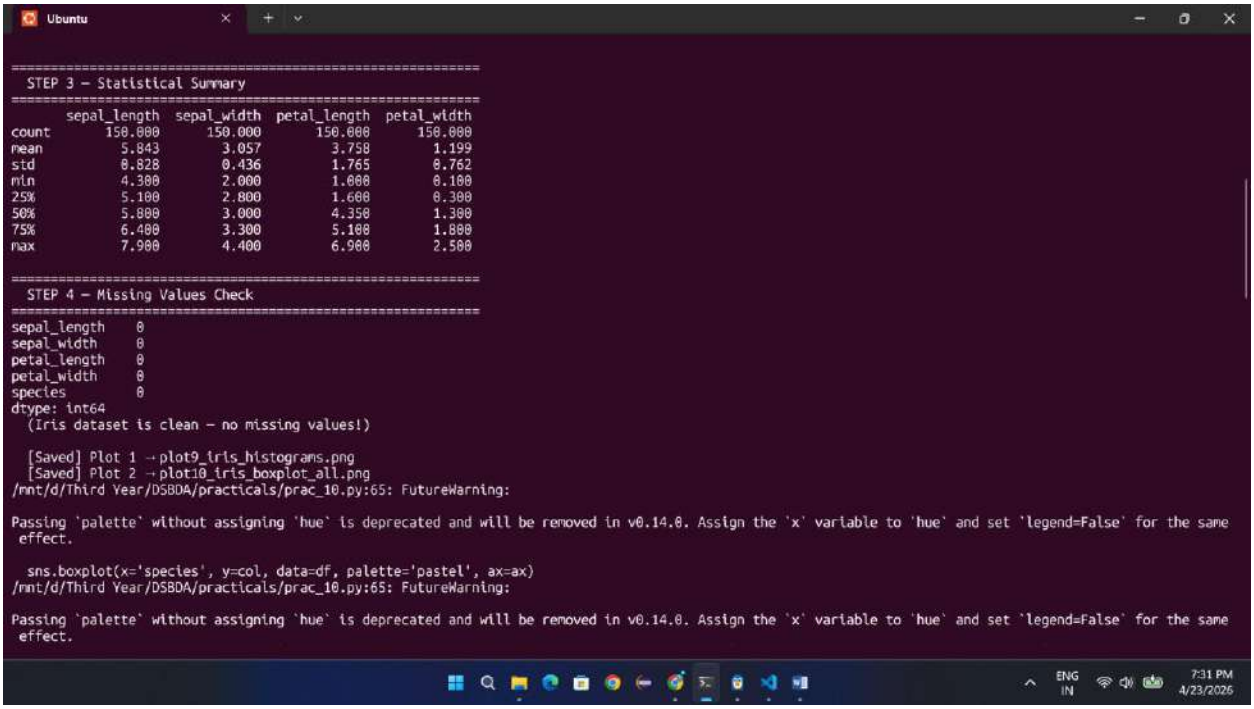
```
from collections import defaultdict
DIVIDER = "=" * 60
input_text = """

hadoop is simple
hadoop is powerful
map reduce in hadoop

data processing with hadoop is fast
map reduce is a programming model
"""
print(f"\n{DIVIDER}")
print("  PRACTICAL 11 — WordCount using MapReduce Logic")
print(DIVIDER)
print("  Input Text:")
print(input_text)

def mapper(text):
    """Tokenizes text and emits (word, 1) pairs."""
    intermediate = []
    for line in text.strip().split('\n'):
        for word in line.strip().split():
            word = word.lower()
            intermediate.append((word, 1))
    return intermediate

mapper_output = mapper(input_text)

print(f"{DIVIDER}")
print("  MAPPER OUTPUT — (word, 1) pairs emitted")
print(DIVIDER)
for pair in mapper_output:
    print(f"    {pair[0]:15s}  →  {pair[1]}")

def shuffle_and_sort(mapper_output):
    """Groups values by key."""
```

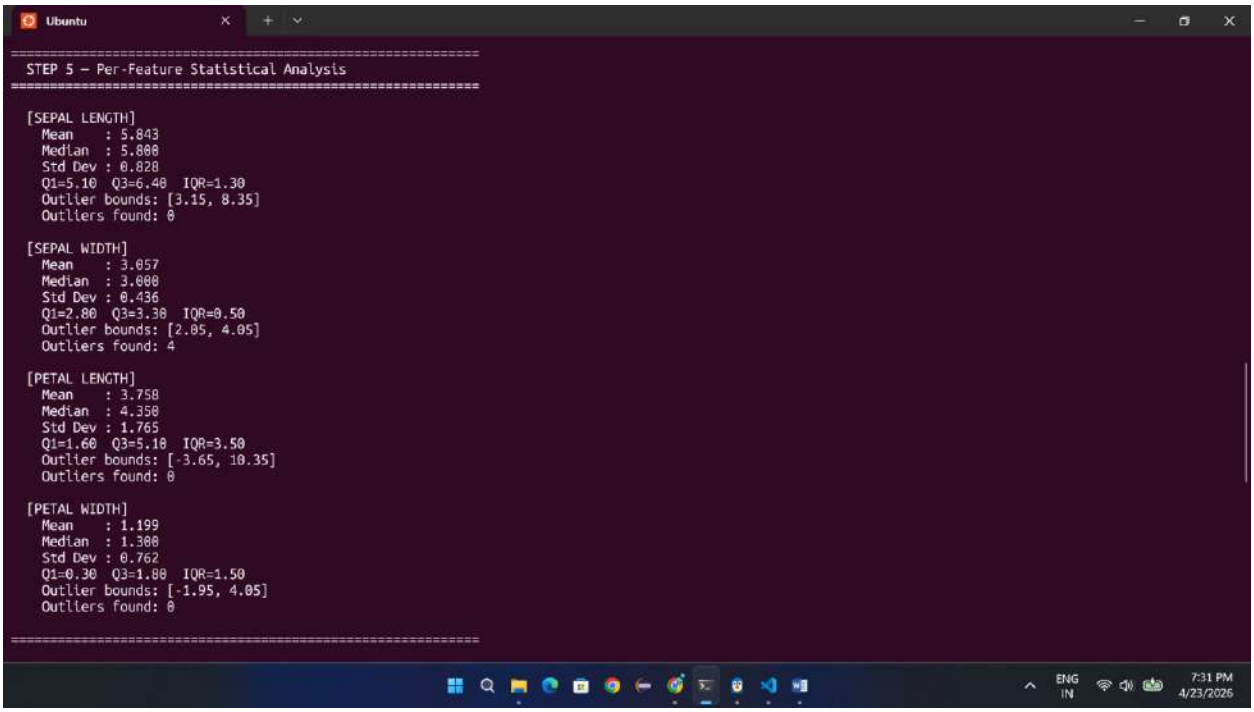
```
    grouped = defaultdict(list)
    for key, value in mapper_output:
        grouped[key].append(value)
    return dict(sorted(grouped.items()))

shuffled = shuffle_and_sort(mapper_output)

print(f"\n{DIVIDER}")
print("  SHUFFLE & SORT — Values grouped by key (word)")

print(DIVIDER)
for word, values in shuffled.items():
    print(f"    {word:15s}  →  {values}")


def reducer(shuffled):
    """Sums values for each key."""
    result = {}
    for key, values in shuffled.items():
        result[key] = sum(values)
    return result
final_output = reducer(shuffled)
print(f"\n{DIVIDER}")

print("  REDUCER OUTPUT — Final Word Count")
print(DIVIDER)
print(f"  {'Word':<20} Count")
print(f"  {'-'*30}")

for word, count in sorted(final_output.items(), key=lambda x: -x[1]):
    print(f"  {word:<20} {count}")



print(f"\n{DIVIDER}")

print("  SUMMARY")
print(DIVIDER)
print(f"  Total unique words : {len(final_output)}")
print(f"  Total word tokens  : {sum(final_output.values())}")
print(f"  Most frequent word : '{max(final_output, key=final_output.get)}'")
```

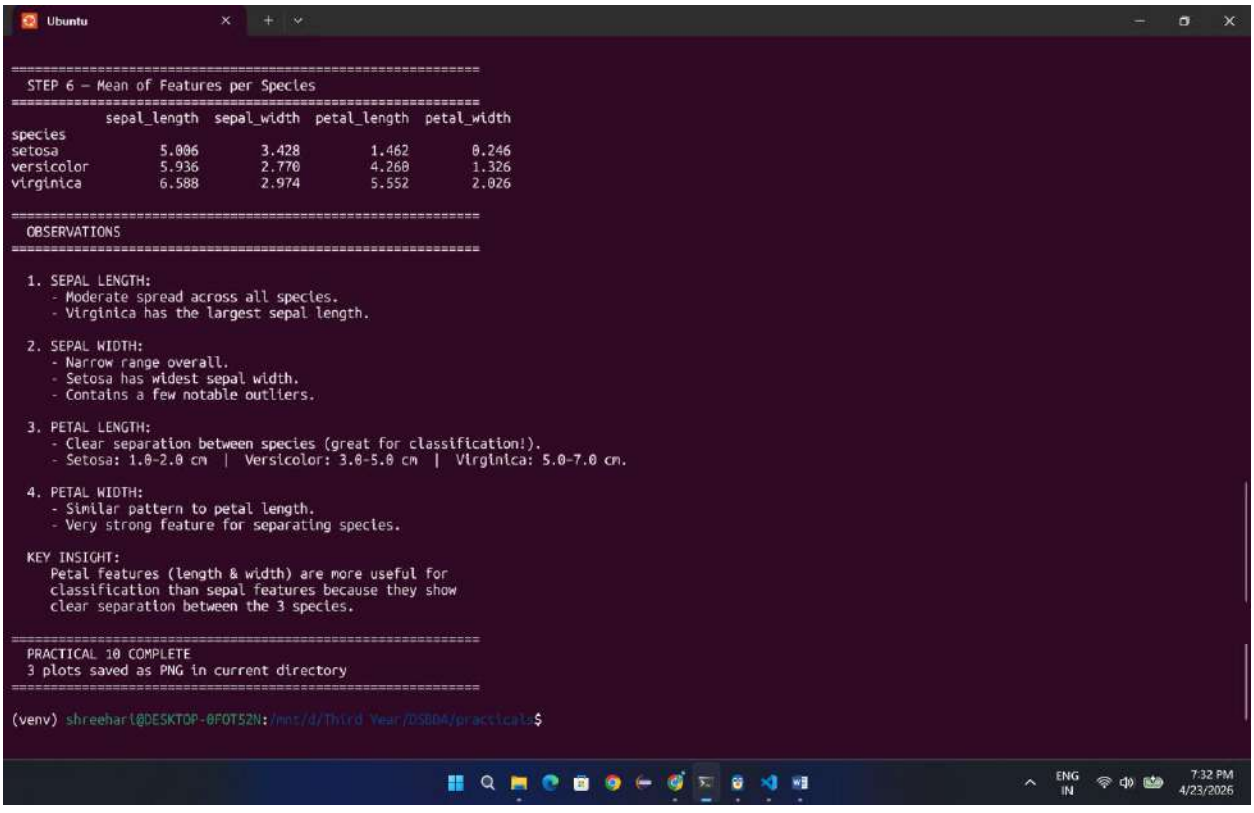
```
print(f"\n{DIVIDER}")
print("  MapReduce Flow (how Hadoop works)")
print(DIVIDER)
print("""
  INPUT FILE
     ↓
  [MAPPER]  — reads lines, emits (word, 1) for each word
     ↓
  [SHUFFLE & SORT]  — groups all (word, [1,1,1,...]) together
     ↓
  [REDUCER]  — sums the list → emits (word, total_count)
     ↓
  OUTPUT FILE
""")

print(f"{'='*60}")
print("  PRACTICAL 11 COMPLETE")
print(f"{'='*60}\n")
```

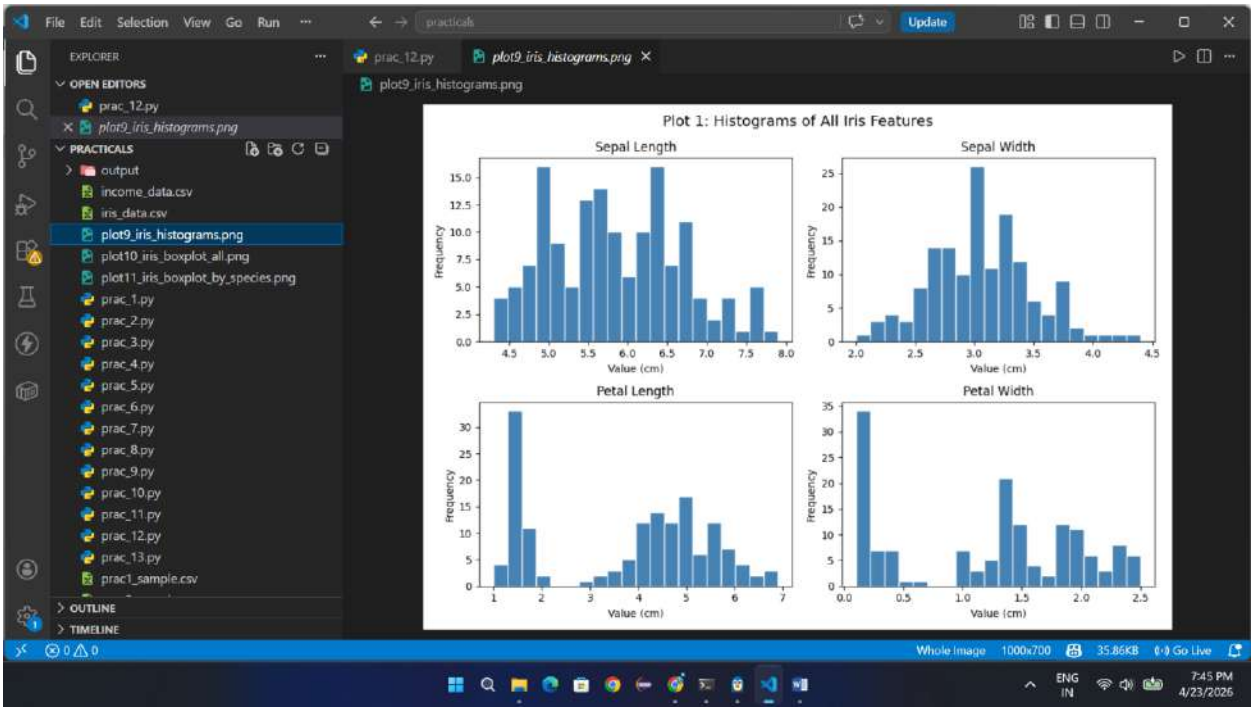
➢ Output

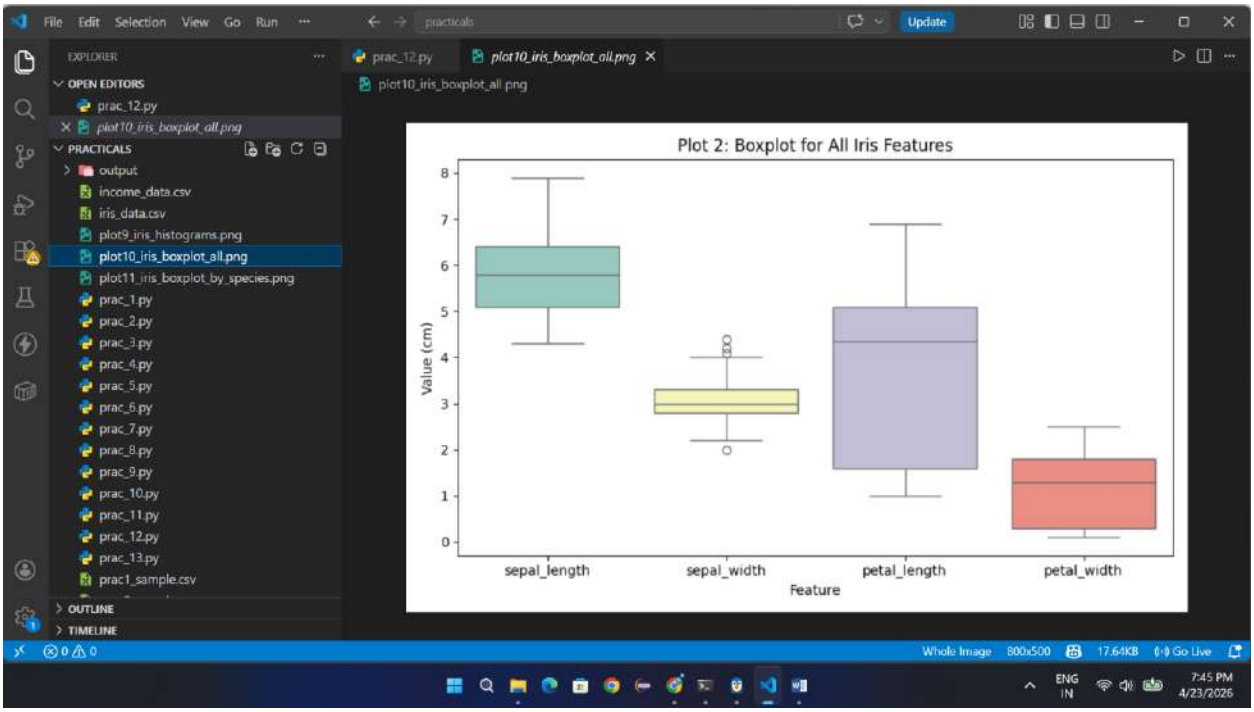

Ubuntu
(venv) shreehari@DESKTOP-0FOT52N:/mnt/d/Third Year/DSBDA/practicals$ python3 prac_10.py
==
STEP 1 — Iris Dataset Loaded (seaborn built-in)
Shape: (150, 5)
==
sepal_length sepal_width petal_length petal_width species
0 5.1 3.5 1.4 0.2 setosa
1 4.9 3.0 1.4 0.2 setosa
2 4.7 3.2 1.3 0.2 setosa
3 4.6 3.1 1.5 0.2 setosa
4 5.0 3.6 1.4 0.2 setosa
5 5.4 3.9 1.7 0.4 setosa
6 4.6 3.4 1.4 0.3 setosa
7 5.0 3.4 1.5 0.2 setosa
8 4.4 2.9 1.4 0.2 setosa
9 4.9 3.1 1.5 0.1 setosa
==
STEP 2 — Data Types of Each Column
==
sepal_length float64
sepal_width float64
petal_length float64
petal_width float64
species str
dtype: object
Numeric features : sepal_length, sepal_width, petal_length, petal_width
Categorical (nominal): species
Species distribution:
species
setosa 50
versicolor 50
virginica 50
Name: count, dtype: int64
ENG IN
7:30 PM 4/23/2026

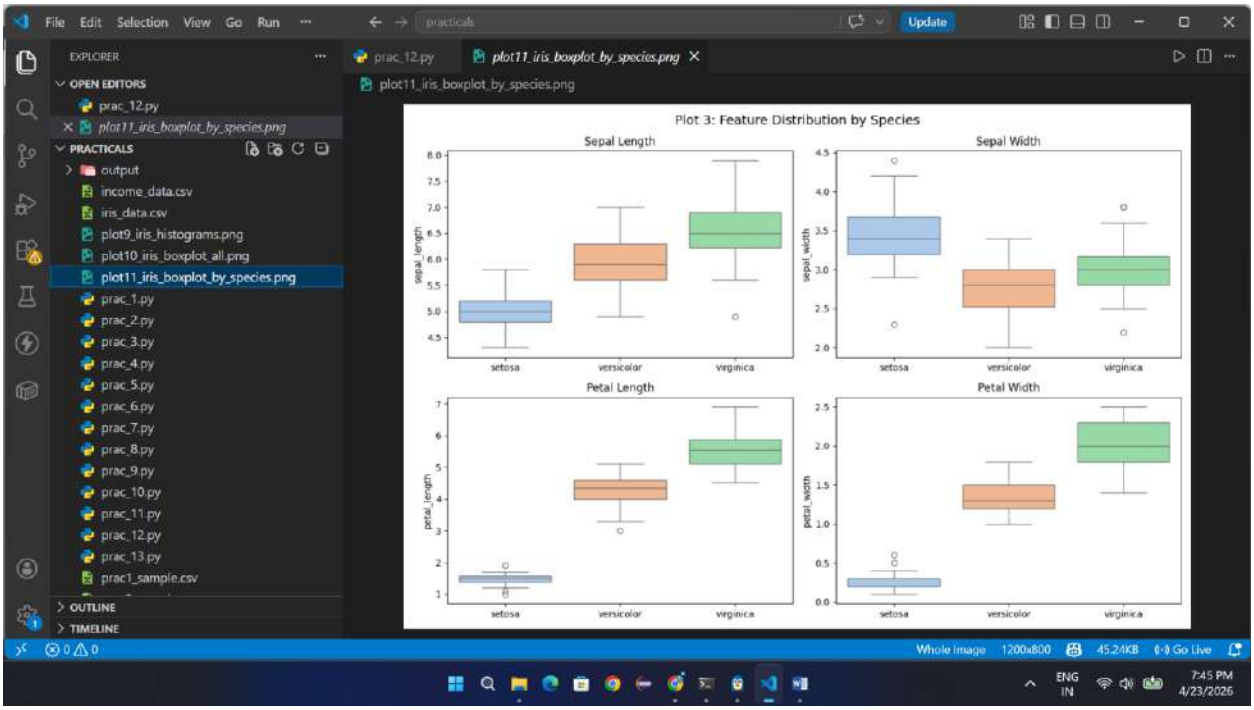

Ubuntu
==
STEP 3 — Statistical Summary
==
sepal_length sepal_width petal_length petal_width
count 150.000 150.000 150.000 150.000
mean 5.843 3.057 3.758 1.199
std 0.828 0.436 1.765 0.762
min 4.300 2.000 1.000 0.100
25% 5.100 2.800 1.600 0.300
50% 5.800 3.000 4.350 1.300
75% 6.400 3.300 5.100 1.800
max 7.900 4.400 6.900 2.500
==
STEP 4 — Missing Values Check
==
sepal_length 0
sepal_width 0
petal_length 0
petal_width 0
species 0
dtype: int64
(Iris dataset is clean — no missing values!)
[Saved] Plot 1 → plot9_iris_histograms.png
[Saved] Plot 2 → plot10_iris_boxplot_all.png
/mnt/d/Third Year/DSBDA/practicals/prac_10.py:65: FutureWarning:
Passing `palette` without assigning `hue` is deprecated and will be removed in v0.14.0. Assign the `x` variable to `hue` and set `legend=False` for the same effect.
sns.boxplot(x='species', y=col, data=df, palette='pastel', ax=ax)
/mnt/d/Third Year/DSBDA/practicals/prac_10.py:65: FutureWarning:
Passing `palette` without assigning `hue` is deprecated and will be removed in v0.14.0. Assign the `x` variable to `hue` and set `legend=False` for the same effect.
ENG IN
7:31 PM 4/23/2026


Ubuntu
STEP 5 — Per-Feature Statistical Analysis
[SEPAL LENGTH]
Mean : 5.843
Median : 5.800
Std Dev : 0.828
Q1=5.10 Q3=6.40 IQR=1.30
Outlier bounds: [3.15, 8.35]
Outliers found: 0
[SEPAL WIDTH]
Mean : 3.057
Median : 3.000
Std Dev : 0.436
Q1=2.80 Q3=3.30 IQR=0.50
Outlier bounds: [2.05, 4.05]
Outliers found: 4
[PETAL LENGTH]
Mean : 3.758
Median : 4.350
Std Dev : 1.765
Q1=1.60 Q3=5.10 IQR=3.50
Outlier bounds: [-3.65, 10.35]
Outliers found: 0
[PETAL WIDTH]
Mean : 1.199
Median : 1.300
Std Dev : 0.762
Q1=0.30 Q3=1.80 IQR=1.50
Outlier bounds: [-1.95, 4.05]
Outliers found: 0
ENG
IN
7:31 PM
4/23/2026


Ubuntu
STEP 6 — Mean of Features per Species
sepal_length sepal_width petal_length petal_width
species
setosa 5.006 3.428 1.462 0.246
versicolor 5.936 2.770 4.260 1.326
virginica 6.588 2.974 5.552 2.026
OBSERVATIONS
1. SEPAL LENGTH:
- Moderate spread across all species.
- Virginica has the largest sepal length.
2. SEPAL WIDTH:
- Narrow range overall.
- Setosa has widest sepal width.
- Contains a few notable outliers.
3. PETAL LENGTH:
- Clear separation between species (great for classification!).
- Setosa: 1.0-2.0 cm | Versicolor: 3.0-5.0 cm | Virginica: 5.0-7.0 cm.
4. PETAL WIDTH:
- Similar pattern to petal length.
- Very strong feature for separating species.
KEY INSIGHT:
Petal features (length & width) are more useful for
classification than sepal features because they show
clear separation between the 3 species.
PRACTICAL 10 COMPLETE
3 plots saved as PNG in current directory
(venv) shreehar1@DESKTOP-0FOT52N:/mnt/d/Third Year/DSBDA/practicals$
ENG
IN
7:32 PM
4/23/2026


File Edit Selection View Go Run
practicals
Update
EXPLORER
OPEN EDITORS
prac_12.py
plot9_iris_histograms.png
PRACTICALS
output
income_data.csv
iris_data.csv
plot9_iris_histograms.png
plot10_iris_boxplot_all.png
plot11_iris_boxplot_by_species.png
prac_1.py
prac_2.py
prac_3.py
prac_4.py
prac_5.py
prac_6.py
prac_7.py
prac_8.py
prac_9.py
prac_10.py
prac_11.py
prac_12.py
prac_13.py
prac1_sample.csv
OUTLINE
TIMELINE
prac_12.py
plot9_iris_histograms.png
plot9_iris_histograms.png
Plot 1: Histograms of All Iris Features
Sepal Length
Sepal Width
Petal Length
Petal Width
Frequency
Value (cm)
Whole Image 1000x700 35.86KB Go Live
ENG IN
7:45 PM 4/23/2026


File Edit Selection View Go Run
practicals
Update
EXPLORER
OPEN EDITORS
prac_12.py
plot10_iris_boxplot_all.png
PRACTICALS
output
income_data.csv
iris_data.csv
plot9_iris_histograms.png
plot10_iris_boxplot_all.png
plot11_iris_boxplot_by_species.png
prac_1.py
prac_2.py
prac_3.py
prac_4.py
prac_5.py
prac_6.py
prac_7.py
prac_8.py
prac_9.py
prac_10.py
prac_11.py
prac_12.py
prac_13.py
prac1_sample.csv
OUTLINE
TIMELINE
prac_12.py
plot10_iris_boxplot_all.png
plot10_iris_boxplot_all.png
Plot 2: Boxplot for All Iris Features
Value (cm)
sepal_length
sepal_width
petal_length
petal_width
Feature
Whole Image 800x500 17.64KB Go Live
ENG IN
7:45 PM 4/23/2026


File Edit Selection View Go Run
practicals
Update
EXPLORER
OPEN EDITORS
prac_12.py
plot11_iris_boxplot_by_species.png
PRACTICALS
output
income_data.csv
iris_data.csv
plot9_iris_histograms.png
plot10_iris_boxplot_all.png
plot11_iris_boxplot_by_species.png
prac_1.py
prac_2.py
prac_3.py
prac_4.py
prac_5.py
prac_6.py
prac_7.py
prac_8.py
prac_9.py
prac_10.py
prac_11.py
prac_12.py
prac_13.py
prac1_sample.csv
OUTLINE
TIMELINE
prac_12.py
plot11_iris_boxplot_by_species.png
Plot 3: Feature Distribution by Species
Sepal Length
Sepal Width
Petal Length
Petal Width
sepal_length
sepal_width
petal_length
petal_width
setosa
versicolor
virginica
Whole Image 1200x800 45.24KB Go Live
ENG IN
7:45 PM 4/23/2026

Practical No. 11

WordCount.java:

```
import java.io.IOException;
import java.util.StringTokenizer;
import org.apache.hadoop.conf.Configuration;
import org.apache.hadoop.fs.Path;
import org.apache.hadoop.io.IntWritable;
import org.apache.hadoop.io.Text;
import org.apache.hadoop.mapreduce.Job;
import org.apache.hadoop.mapreduce.Mapper;
import org.apache.hadoop.mapreduce.Reducer;
import org.apache.hadoop.mapreduce.lib.input.FileInputFormat;
import org.apache.hadoop.mapreduce.lib.output.FileOutputFormat;
public class WordCount {
  public static class TokenizerMapper
       extends Mapper<Object, Text, Text, IntWritable> {
    private final static IntWritable one = new IntWritable(1);
    private Text word = new Text();
    public void map(Object key, Text value, Context context)
        throws IOException, InterruptedException {
      StringTokenizer itr = new StringTokenizer(value.toString());
      while (itr.hasMoreTokens()) {
        word.set(itr.nextToken().toLowerCase());
        context.write(word, one);
      }
    }
  }
  public static class IntSumReducer
       extends Reducer<Text, IntWritable, Text, IntWritable> {
    private IntWritable result = new IntWritable();
    public void reduce(Text key, Iterable<IntWritable> values,
                       Context context)
        throws IOException, InterruptedException {
      int sum = 0;
      for (IntWritable val : values) {
        sum += val.get();
      }
```

```
            result.set(sum);
            context.write(key, result);
        }
    }
    public static void main(String[] args)
            throws Exception {
        if (args.length != 2) {
            System.err.println("Usage: WordCount <input path> <output path>");
            System.exit(-1);
        }
        Configuration conf = new Configuration();
        Job job = Job.getInstance(conf, "Word Count");
        job.setJarByClass(WordCount.class);
        job.setMapperClass(TokenizerMapper.class);
        job.setReducerClass(IntSumReducer.class);
        job.setOutputKeyClass(Text.class);
        job.setOutputValueClass(IntWritable.class);
        FileInputFormat.addInputPath(job, new Path(args[0]));
        FileOutputFormat.setOutputPath(job, new Path(args[1]));
        System.exit(job.waitForCompletion(true) ? 0 : 1);
    }
}
```

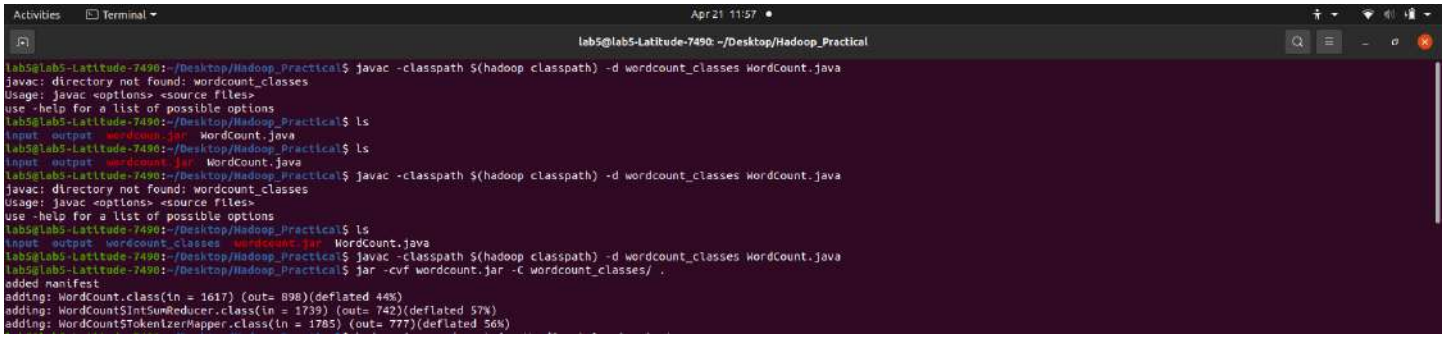
sample.txt:

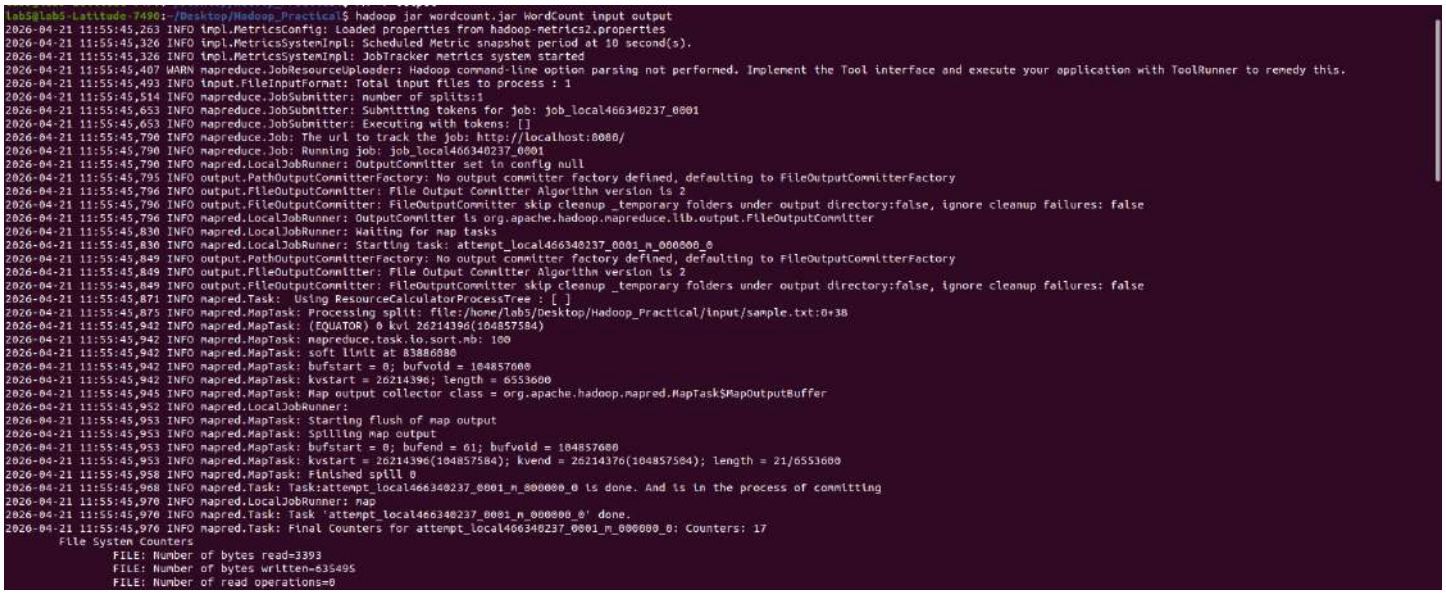
```
hello world
hello hadoop
hello world
```

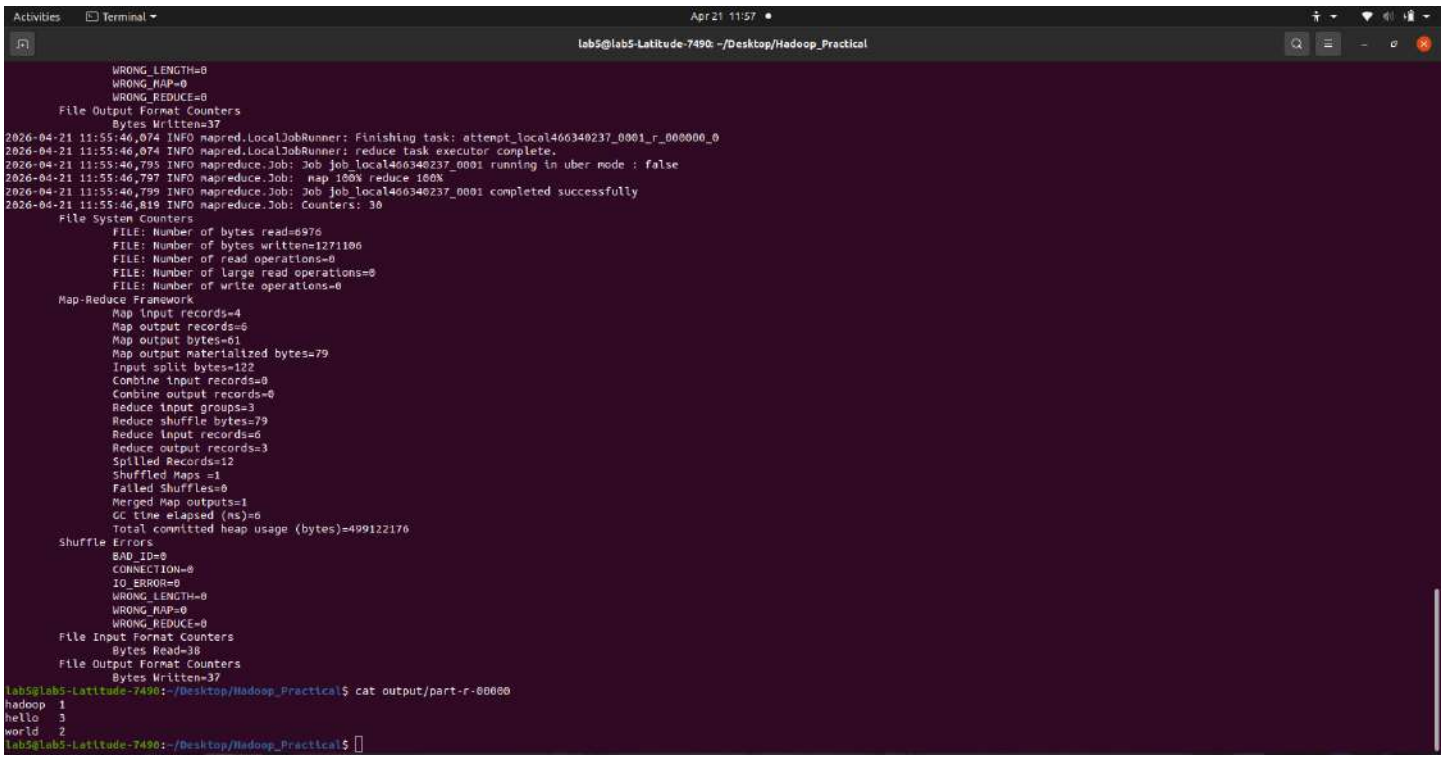
part-r-00000.:

```
hadoop  1
hello   3
world   2
```

OUTPUT:


Activities
Terminal
Apr 21 11:57
lab5@lab5-Latitude-7490: ~/Desktop/Hadoop_Practical
lab5@lab5-Latitude-7490:~/Desktop/Hadoop_Practical$ javac -classpath $(hadoop classpath) -d wordcount_classes WordCount.java
javac: directory not found: wordcount_classes
Usage: javac <options> <source files>
use -help for a list of possible options
lab5@lab5-Latitude-7490:~/Desktop/Hadoop_Practical$ ls
input output wordcoun.jar WordCount.java
lab5@lab5-Latitude-7490:~/Desktop/Hadoop_Practical$ ls
input output wordcount.jar WordCount.java
lab5@lab5-Latitude-7490:~/Desktop/Hadoop_Practical$ javac -classpath $(hadoop classpath) -d wordcount_classes WordCount.java
javac: directory not found: wordcount_classes
Usage: javac <options> <source files>
use -help for a list of possible options
lab5@lab5-Latitude-7490:~/Desktop/Hadoop_Practical$ ls
input output wordcount_classes wordcount.jar WordCount.java
lab5@lab5-Latitude-7490:~/Desktop/Hadoop_Practical$ javac -classpath $(hadoop classpath) -d wordcount_classes WordCount.java
lab5@lab5-Latitude-7490:~/Desktop/Hadoop_Practical$ jar -cvf wordcount.jar -C wordcount_classes/ .
added manifest
adding: WordCount.class(in = 1617) (out= 898)(deflated 44%)
adding: WordCount$IntSumReducer.class(in = 1739) (out= 742)(deflated 57%)
adding: WordCount$TokenizerMapper.class(in = 1785) (out= 777)(deflated 56%)


lab5@lab5-Latitude-7490:~/Desktop/Hadoop_Practical$ hadoop jar wordcount.jar WordCount input output
2026-04-21 11:55:45,263 INFO impl.MetricsConfig: Loaded properties from hadoop-metrics2.properties
2026-04-21 11:55:45,326 INFO impl.MetricsSystemImpl: Scheduled Metric snapshot period at 10 second(s).
2026-04-21 11:55:45,326 INFO impl.MetricsSystemImpl: JobTracker metrics system started
2026-04-21 11:55:45,407 WARN mapreduce.JobResourceUploader: Hadoop command-line option parsing not performed. Implement the Tool interface and execute your application with ToolRunner to remedy this.
2026-04-21 11:55:45,493 INFO input.FileInputFormat: Total input files to process : 1
2026-04-21 11:55:45,514 INFO mapreduce.JobSubmitter: number of splits:1
2026-04-21 11:55:45,653 INFO mapreduce.JobSubmitter: Submitting tokens for job: job_local466340237_0001
2026-04-21 11:55:45,653 INFO mapreduce.JobSubmitter: Executing with tokens: []
2026-04-21 11:55:45,790 INFO mapreduce.Job: The url to track the job: http://localhost:8080/
2026-04-21 11:55:45,790 INFO mapreduce.Job: Running job: job_local466340237_0001
2026-04-21 11:55:45,790 INFO mapred.LocalJobRunner: OutputCommitter set in config null
2026-04-21 11:55:45,795 INFO output.PathOutputCommitterFactory: No output committer factory defined, defaulting to FileOutputCommitterFactory
2026-04-21 11:55:45,796 INFO output.FileOutputCommitter: File Output Committer Algorithm version is 2
2026-04-21 11:55:45,796 INFO output.FileOutputCommitter: FileOutputCommitter skip cleanup _temporary folders under output directory:false, ignore cleanup failures: false
2026-04-21 11:55:45,796 INFO mapred.LocalJobRunner: OutputCommitter is org.apache.hadoop.mapreduce.lib.output.FileOutputCommitter
2026-04-21 11:55:45,830 INFO mapred.LocalJobRunner: Waiting for map tasks
2026-04-21 11:55:45,830 INFO mapred.LocalJobRunner: Starting task: attempt_local466340237_0001_m_000000_0
2026-04-21 11:55:45,849 INFO output.PathOutputCommitterFactory: No output committer factory defined, defaulting to FileOutputCommitterFactory
2026-04-21 11:55:45,849 INFO output.FileOutputCommitter: File Output Committer Algorithm version is 2
2026-04-21 11:55:45,849 INFO output.FileOutputCommitter: FileOutputCommitter skip cleanup _temporary folders under output directory:false, ignore cleanup failures: false
2026-04-21 11:55:45,871 INFO mapred.Task: Using ResourceCalculatorProcessTree : [ ]
2026-04-21 11:55:45,875 INFO mapred.MapTask: Processing split: file:/home/lab5/Desktop/Hadoop_Practical/input/sample.txt:0+38
2026-04-21 11:55:45,942 INFO mapred.MapTask: (EQUATOR) 0 kvi 26214396(104857584)
2026-04-21 11:55:45,942 INFO mapred.MapTask: mapreduce.task.io.sort.mb: 100
2026-04-21 11:55:45,942 INFO mapred.MapTask: soft limit at 83886080
2026-04-21 11:55:45,942 INFO mapred.MapTask: bufstart = 0; bufvoid = 104857600
2026-04-21 11:55:45,942 INFO mapred.MapTask: kvstart = 26214396; length = 6553600
2026-04-21 11:55:45,945 INFO mapred.MapTask: Map output collector class = org.apache.hadoop.mapred.MapTask$MapOutputBuffer
2026-04-21 11:55:45,952 INFO mapred.LocalJobRunner:
2026-04-21 11:55:45,953 INFO mapred.MapTask: Starting flush of map output
2026-04-21 11:55:45,953 INFO mapred.MapTask: Spilling map output
2026-04-21 11:55:45,953 INFO mapred.MapTask: bufstart = 0; bufend = 61; bufvoid = 104857600
2026-04-21 11:55:45,953 INFO mapred.MapTask: kvstart = 26214396(104857584); kvend = 26214376(104857504); length = 21/6553600
2026-04-21 11:55:45,958 INFO mapred.MapTask: Finished spill 0
2026-04-21 11:55:45,968 INFO mapred.Task: Task:attempt_local466340237_0001_m_000000_0 is done. And is in the process of committing
2026-04-21 11:55:45,970 INFO mapred.LocalJobRunner: map
2026-04-21 11:55:45,970 INFO mapred.Task: Task 'attempt_local466340237_0001_m_000000_0' done.
2026-04-21 11:55:45,976 INFO mapred.Task: Final Counters for attempt_local466340237_0001_m_000000_0: Counters: 17
File System Counters
FILE: Number of bytes read=3393
FILE: Number of bytes written=633495
FILE: Number of read operations=0


Activities
Terminal
Apr 21 11:57
lab5@lab5-Latitude-7490: ~/Desktop/Hadoop_Practical
WRONG_LENGTH=0
WRONG_MAP=0
WRONG_REDUCE=0
File Output Format Counters
Bytes Written=37
2026-04-21 11:55:46,074 INFO mapred.LocalJobRunner: Finishing task: attempt_local466340237_0001_r_000000_0
2026-04-21 11:55:46,074 INFO mapred.LocalJobRunner: reduce task executor complete.
2026-04-21 11:55:46,795 INFO mapreduce.Job: Job job_local466340237_0001 running in uber mode : false
2026-04-21 11:55:46,797 INFO mapreduce.Job: map 100% reduce 100%
2026-04-21 11:55:46,799 INFO mapreduce.Job: Job job_local466340237_0001 completed successfully
2026-04-21 11:55:46,819 INFO mapreduce.Job: Counters: 30
File System Counters
FILE: Number of bytes read=6976
FILE: Number of bytes written=1271106
FILE: Number of read operations=0
FILE: Number of large read operations=0
FILE: Number of write operations=0
Map-Reduce Framework
Map input records=4
Map output records=6
Map output bytes=61
Map output materialized bytes=79
Input split bytes=122
Combine input records=0
Combine output records=0
Reduce input groups=3
Reduce shuffle bytes=79
Reduce input records=6
Reduce output records=3
Spilled Records=12
Shuffled Maps =1
Failed Shuffles=0
Merged Map outputs=1
GC time elapsed (ms)=6
Total committed heap usage (bytes)=499122176
Shuffle Errors
BAD_ID=0
CONNECTION=0
IO_ERROR=0
WRONG_LENGTH=0
WRONG_MAP=0
WRONG_REDUCE=0
File Input Format Counters
Bytes Read=38
File Output Format Counters
Bytes Written=37
lab5@lab5-Latitude-7490:~/Desktop/Hadoop_Practical$ cat output/part-r-00000
hadoop 1
hello 3
world 2
lab5@lab5-Latitude-7490:~/Desktop/Hadoop_Practical$

Practical No. 12

LogAnalysis.java:

```
import java.io.IOException;
import org.apache.hadoop.conf.Configuration;
import org.apache.hadoop.fs.Path;
import org.apache.hadoop.io.IntWritable;
import org.apache.hadoop.io.Text;
import org.apache.hadoop.mapreduce.Job;
import org.apache.hadoop.mapreduce.Mapper;
import org.apache.hadoop.mapreduce.Reducer;
import org.apache.hadoop.mapreduce.lib.input.FileInputFormat;
import org.apache.hadoop.mapreduce.lib.output.FileOutputFormat;
public class LogAnalysis {
   public static class LogMapper
      extends Mapper<Object, Text, Text, IntWritable> {
      private final static IntWritable one = new IntWritable(1);
      private Text word = new Text();
      public void map(Object key, Text value, Context context)
            throws IOException, InterruptedException {
         String line = value.toString();
         if(line.contains("INFO")) {
            word.set("INFO");
            context.write(word, one);
         }
         if(line.contains("ERROR")) {
            word.set("ERROR");
            context.write(word, one);
         }
         if(line.contains("WARN")) {
            word.set("WARN");
            context.write(word, one);
         }
      }
   }
   public static class LogReducer
      extends Reducer<Text, IntWritable, Text, IntWritable> {
```

```
    public void reduce(Text key, Iterable<IntWritable> values,
                       Context context)
        throws IOException, InterruptedException {
      int sum = 0;
      for(IntWritable val : values) {
        sum += val.get();
      }
      context.write(key, new IntWritable(sum));
    }
  }
  public static void main(String[] args) throws Exception {
    Configuration conf = new Configuration();
    Job job = Job.getInstance(conf, "log analysis");
    job.setJarByClass(LogAnalysis.class);
    job.setMapperClass(LogMapper.class);
    job.setReducerClass(LogReducer.class);
    job.setOutputKeyClass(Text.class);
    job.setOutputValueClass(IntWritable.class);
    FileInputFormat.addInputPath(job, new Path(args[0]));
    FileOutputFormat.setOutputPath(job, new Path(args[1]));
    System.exit(job.waitForCompletion(true) ? 0 : 1);
  }
}
```

logs.txt:

2026 INFO User Login

2026 ERROR Database Failed

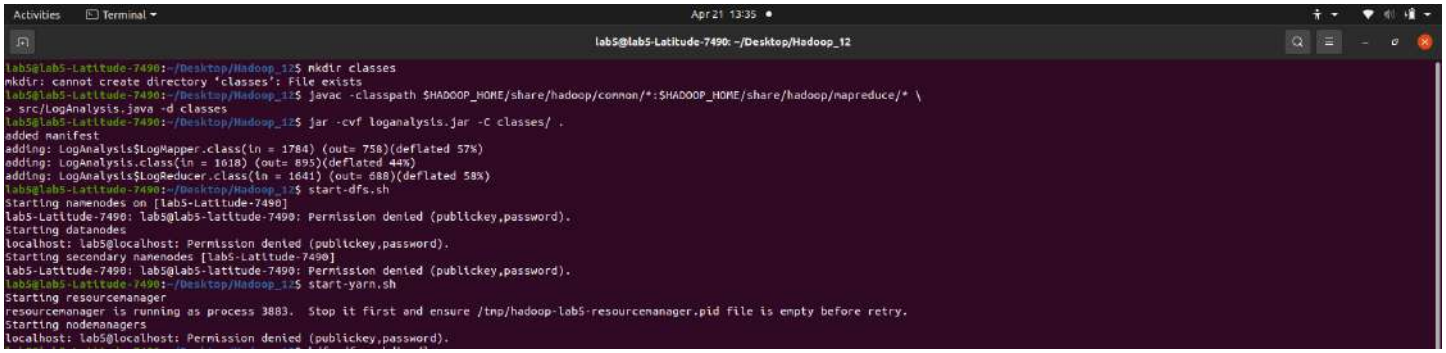
2026 INFO File Opened

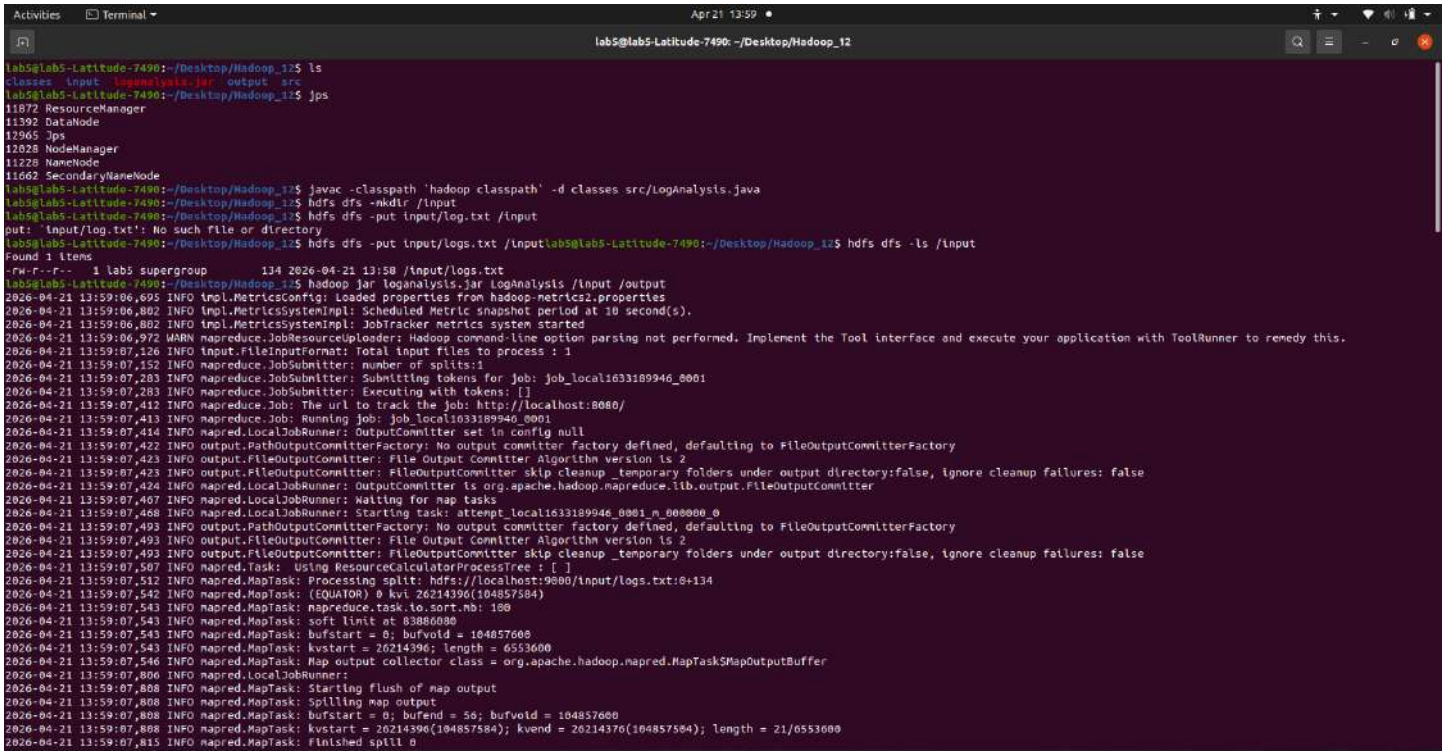
2026 WARN Memory Low

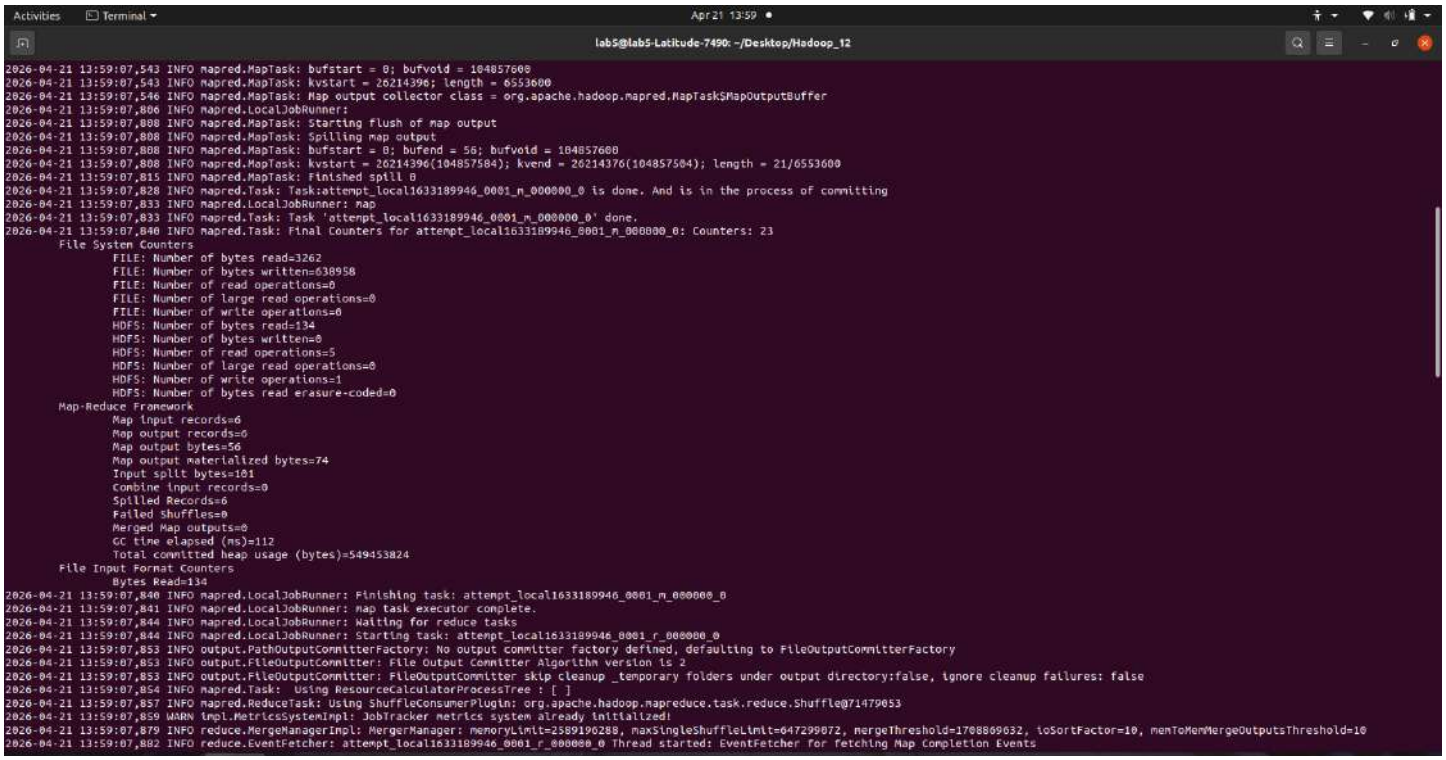
2026 ERROR Network Failed

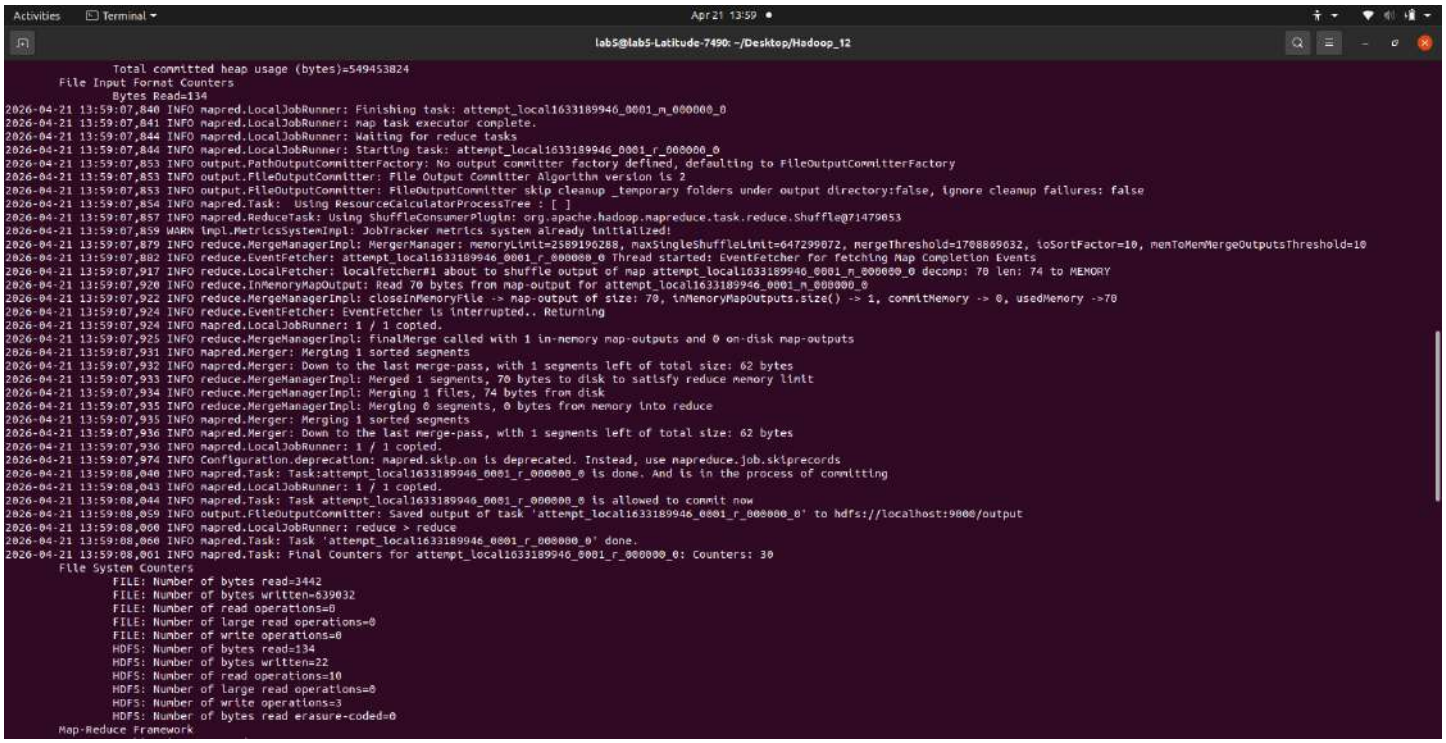
2026 INFO Logout

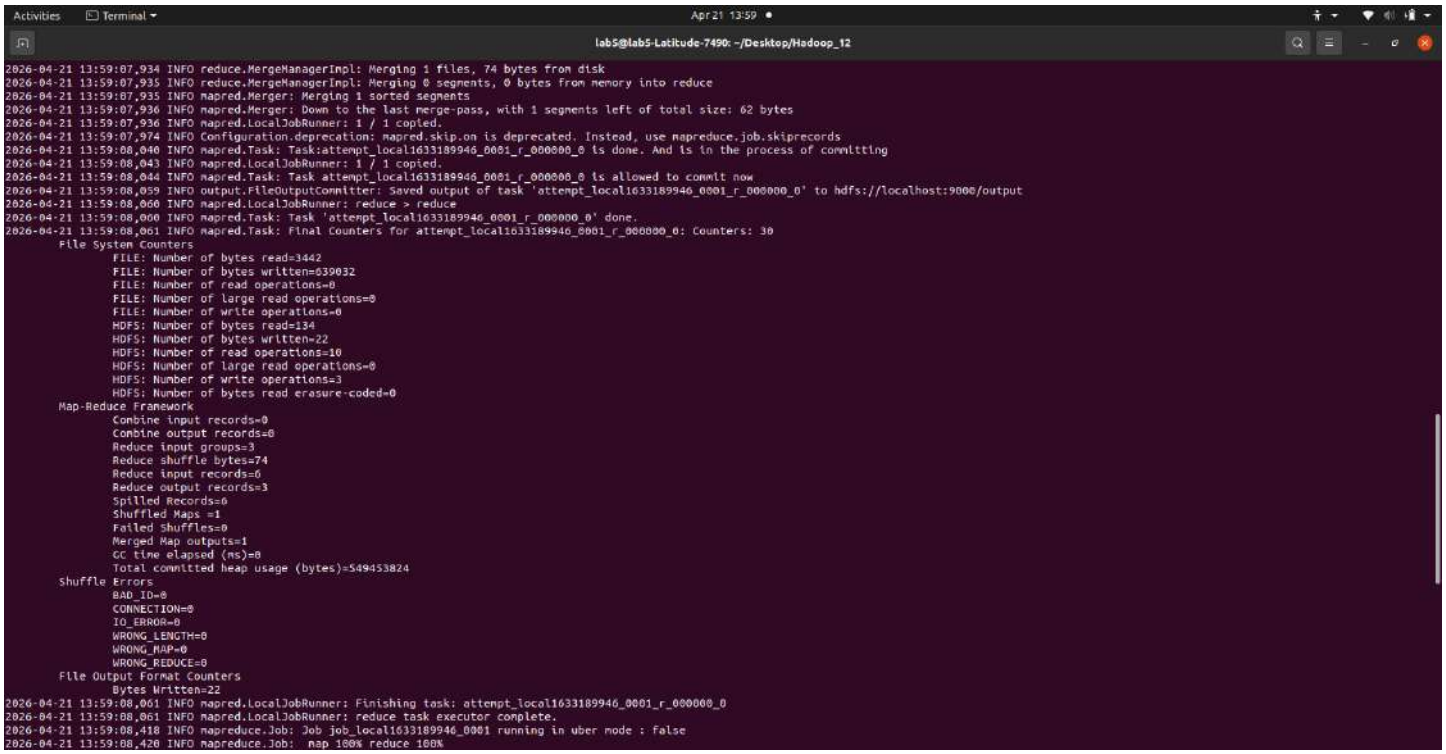
OUTPUT:

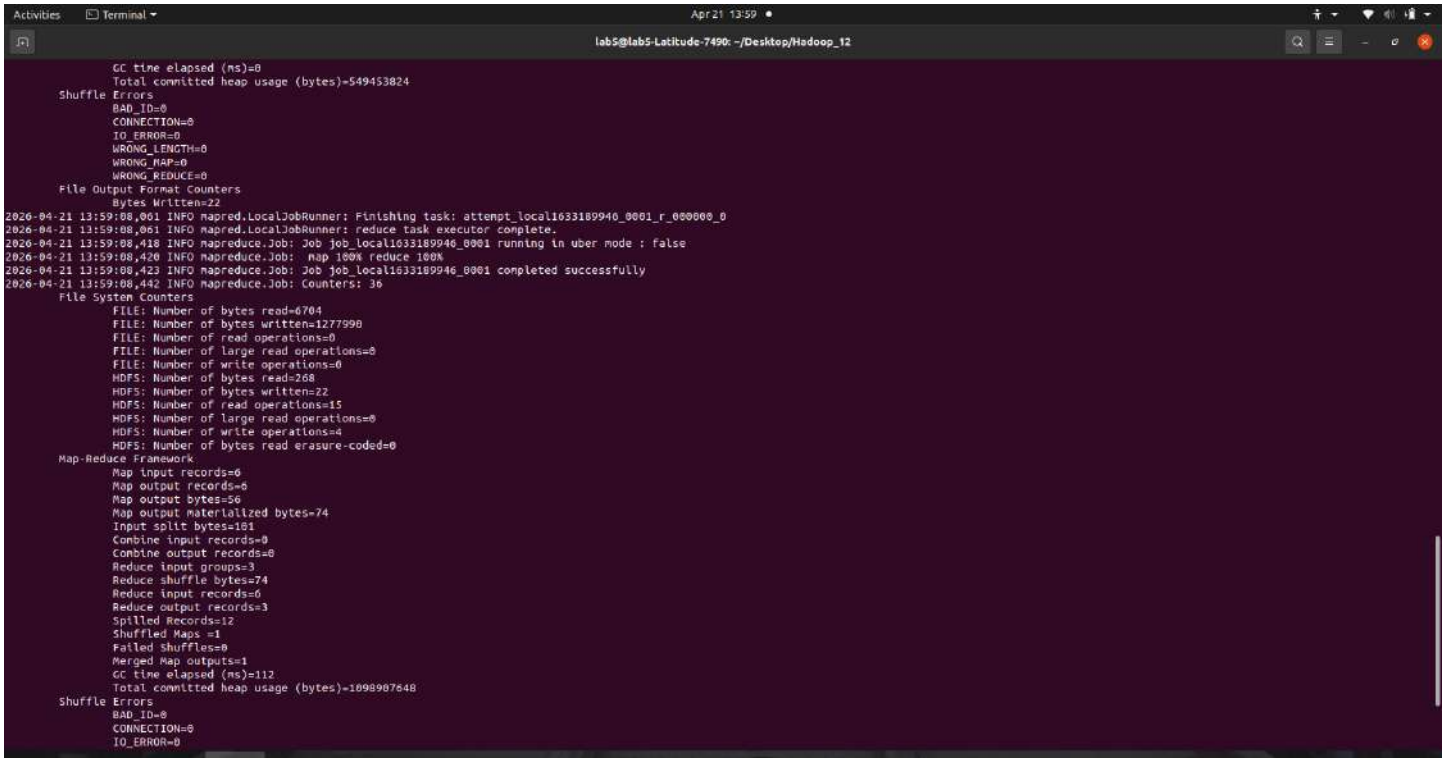

Activities
Terminal
Apr 21 13:35
lab5@lab5-Latitude-7490: ~/Desktop/Hadoop_12
lab5@lab5-Latitude-7490:~/Desktop/Hadoop_12$ mkdir classes
mkdir: cannot create directory 'classes': File exists
lab5@lab5-Latitude-7490:~/Desktop/Hadoop_12$ javac -classpath $HADOOP_HOME/share/hadoop/common/*:$HADOOP_HOME/share/hadoop/mapreduce/* \
> src/LogAnalysis.java -d classes
lab5@lab5-Latitude-7490:~/Desktop/Hadoop_12$ jar -cvf loganalysis.jar -C classes/ .
added manifest
adding: LogAnalysis$LogMapper.class(in = 1784) (out= 758)(deflated 57%)
adding: LogAnalysis.class(in = 1618) (out= 895)(deflated 44%)
adding: LogAnalysis$LogReducer.class(in = 1641) (out= 688)(deflated 58%)
lab5@lab5-Latitude-7490:~/Desktop/Hadoop_12$ start-dfs.sh
Starting namenodes on [lab5-Latitude-7490]
lab5-Latitude-7490: lab5@lab5-Latitude-7490: Permission denied (publickey,password).
Starting datanodes
localhost: lab5@localhost: Permission denied (publickey,password).
Starting secondary namenodes [lab5-Latitude-7490]
lab5-Latitude-7490: lab5@lab5-Latitude-7490: Permission denied (publickey,password).
lab5@lab5-Latitude-7490:~/Desktop/Hadoop_12$ start-yarn.sh
Starting resourcemanager
resourcemanager is running as process 3883. Stop it first and ensure /tmp/hadoop-lab5-resourcemanager.pid file is empty before retry.
Starting nodemanagers
localhost: lab5@localhost: Permission denied (publickey,password).

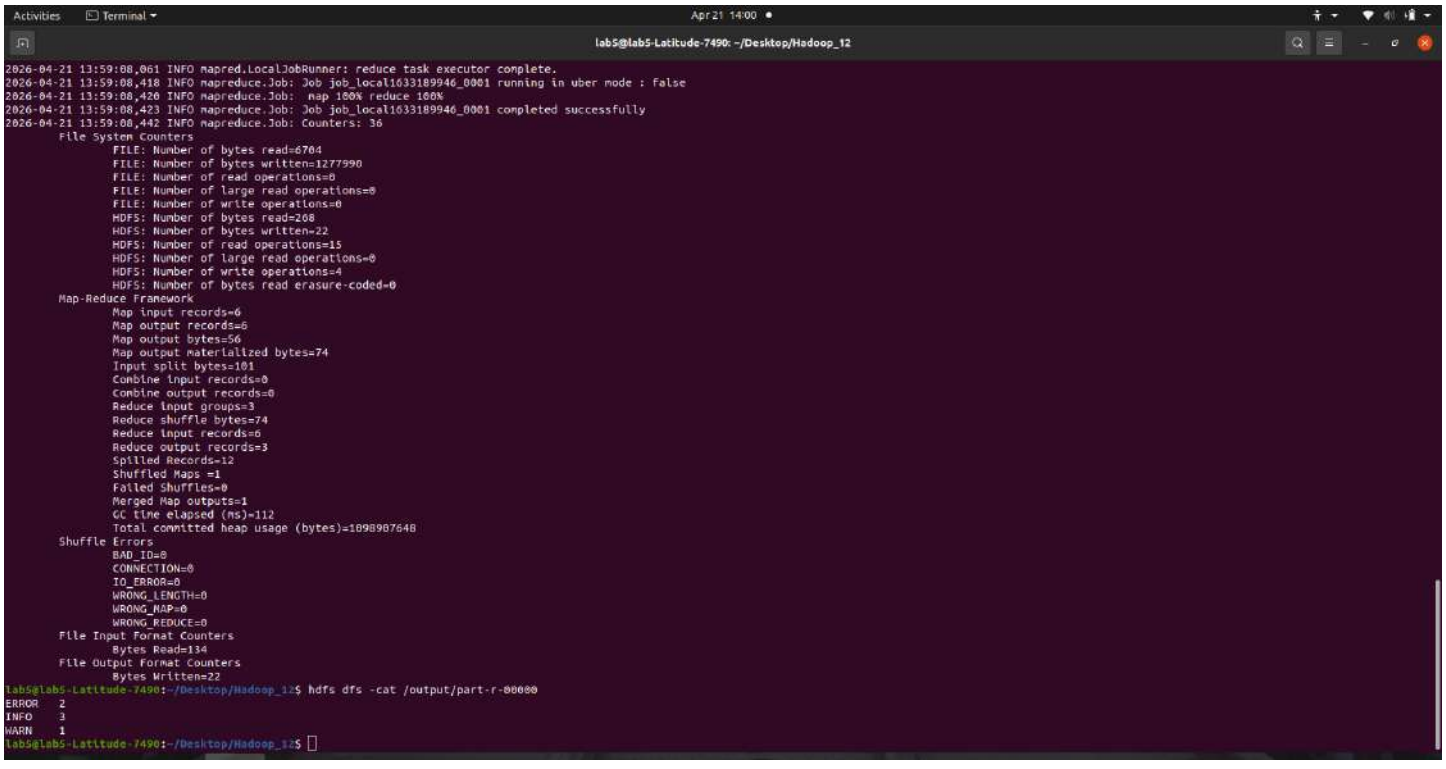

Activities
Terminal
Apr 21 13:59
lab5@lab5-Latitude-7490: ~/Desktop/Hadoop_12
lab5@lab5-Latitude-7490:~/Desktop/Hadoop_12$ ls
classes input loganalysis.jar output src
lab5@lab5-Latitude-7490:~/Desktop/Hadoop_12$ jps
11872 ResourceManager
11392 DataNode
12965 Jps
12028 NodeManager
11228 NameNode
11662 SecondaryNameNode
lab5@lab5-Latitude-7490:~/Desktop/Hadoop_12$ javac -classpath 'hadoop classpath' -d classes src/LogAnalysis.java
lab5@lab5-Latitude-7490:~/Desktop/Hadoop_12$ hdfs dfs -mkdir /input
lab5@lab5-Latitude-7490:~/Desktop/Hadoop_12$ hdfs dfs -put input/log.txt /input
put: `input/log.txt': No such file or directory
lab5@lab5-Latitude-7490:~/Desktop/Hadoop_12$ hdfs dfs -put input/logs.txt /inputlab5@lab5-Latitude-7490:~/Desktop/Hadoop_12$ hdfs dfs -ls /input
Found 1 items
-rw-r--r--   1 lab5 supergroup   134 2026-04-21 13:58 /input/logs.txt
lab5@lab5-Latitude-7490:~/Desktop/Hadoop_12$ hadoop jar loganalysis.jar LogAnalysis /input /output
2026-04-21 13:59:06,695 INFO impl.MetricsConfig: Loaded properties from hadoop-metrics2.properties
2026-04-21 13:59:06,802 INFO impl.MetricsSystemImpl: Scheduled Metric snapshot period at 10 second(s).
2026-04-21 13:59:06,802 INFO impl.MetricsSystemImpl: JobTracker metrics system started
2026-04-21 13:59:06,972 WARN mapreduce.JobResourceUploader: Hadoop command-line option parsing not performed. Implement the Tool interface and execute your application with ToolRunner to remedy this.
2026-04-21 13:59:07,126 INFO input.FileInputFormat: Total input files to process : 1
2026-04-21 13:59:07,152 INFO mapreduce.JobSubmitter: number of splits:1
2026-04-21 13:59:07,283 INFO mapreduce.JobSubmitter: Submitting tokens for job: job_local1633189946_0001
2026-04-21 13:59:07,283 INFO mapreduce.JobSubmitter: Executing with tokens: []
2026-04-21 13:59:07,412 INFO mapreduce.Job: The url to track the job: http://localhost:8080/
2026-04-21 13:59:07,413 INFO mapreduce.Job: Running job: job_local1633189946_0001
2026-04-21 13:59:07,414 INFO mapred.LocalJobRunner: OutputCommitter set in config null
2026-04-21 13:59:07,422 INFO output.PathOutputCommitterFactory: No output committer factory defined, defaulting to FileOutputCommitterFactory
2026-04-21 13:59:07,423 INFO output.FileOutputCommitter: File Output Committer Algorithm version is 2
2026-04-21 13:59:07,423 INFO output.FileOutputCommitter: FileOutputCommitter skip cleanup _temporary folders under output directory:false, ignore cleanup failures: false
2026-04-21 13:59:07,424 INFO mapred.LocalJobRunner: OutputCommitter is org.apache.hadoop.mapreduce.lib.output.FileOutputCommitter
2026-04-21 13:59:07,467 INFO mapred.LocalJobRunner: Waiting for map tasks
2026-04-21 13:59:07,468 INFO mapred.LocalJobRunner: Starting task: attempt_local1633189946_0001_m_000000_0
2026-04-21 13:59:07,493 INFO output.PathOutputCommitterFactory: No output committer factory defined, defaulting to FileOutputCommitterFactory
2026-04-21 13:59:07,493 INFO output.FileOutputCommitter: File Output Committer Algorithm version is 2
2026-04-21 13:59:07,493 INFO output.FileOutputCommitter: FileOutputCommitter skip cleanup _temporary folders under output directory:false, ignore cleanup failures: false
2026-04-21 13:59:07,507 INFO mapred.Task: Using ResourceCalculatorProcessTree : [ ]
2026-04-21 13:59:07,512 INFO mapred.MapTask: Processing split: hdfs://localhost:9000/input/logs.txt:0+134
2026-04-21 13:59:07,542 INFO mapred.MapTask: (EQUATOR) 0 kvi 26214396(104857584)
2026-04-21 13:59:07,543 INFO mapred.MapTask: mapreduce.task.io.sort.mb: 100
2026-04-21 13:59:07,543 INFO mapred.MapTask: soft limit at 83886080
2026-04-21 13:59:07,543 INFO mapred.MapTask: bufstart = 0; bufvoid = 104857600
2026-04-21 13:59:07,543 INFO mapred.MapTask: kvstart = 26214396; length = 6553600
2026-04-21 13:59:07,546 INFO mapred.MapTask: Map output collector class = org.apache.hadoop.mapred.MapTask$MapOutputBuffer
2026-04-21 13:59:07,806 INFO mapred.LocalJobRunner:
2026-04-21 13:59:07,808 INFO mapred.MapTask: Starting flush of map output
2026-04-21 13:59:07,808 INFO mapred.MapTask: Spilling map output
2026-04-21 13:59:07,808 INFO mapred.MapTask: bufstart = 0; bufend = 56; bufvoid = 104857600
2026-04-21 13:59:07,808 INFO mapred.MapTask: kvstart = 26214396(104857584); kvend = 26214376(104857504); length = 21/6553600
2026-04-21 13:59:07,815 INFO mapred.MapTask: Finished spill 0


Activities
Terminal
Apr 21 13:59
lab5@lab5-Latitude-7490: ~/Desktop/Hadoop_12
2026-04-21 13:59:07,543 INFO mapred.MapTask: bufstart = 0; bufvoid = 104857600
2026-04-21 13:59:07,543 INFO mapred.MapTask: kvstart = 26214396; length = 6553600
2026-04-21 13:59:07,546 INFO mapred.MapTask: Map output collector class = org.apache.hadoop.mapred.MapTask$MapOutputBuffer
2026-04-21 13:59:07,806 INFO mapred.LocalJobRunner:
2026-04-21 13:59:07,808 INFO mapred.MapTask: Starting flush of map output
2026-04-21 13:59:07,808 INFO mapred.MapTask: Spilling map output
2026-04-21 13:59:07,808 INFO mapred.MapTask: bufstart = 0; bufend = 56; bufvoid = 104857600
2026-04-21 13:59:07,808 INFO mapred.MapTask: kvstart = 26214396(104857584); kvend = 26214376(104857504); length = 21/6553600
2026-04-21 13:59:07,815 INFO mapred.MapTask: Finished spill 0
2026-04-21 13:59:07,828 INFO mapred.Task: Task:attempt_local1633189946_0001_m_000000_0 is done. And is in the process of committing
2026-04-21 13:59:07,833 INFO mapred.LocalJobRunner: map
2026-04-21 13:59:07,833 INFO mapred.Task: Task 'attempt_local1633189946_0001_m_000000_0' done.
2026-04-21 13:59:07,840 INFO mapred.Task: Final Counters for attempt_local1633189946_0001_m_000000_0: Counters: 23
File System Counters
FILE: Number of bytes read=3262
FILE: Number of bytes written=638958
FILE: Number of read operations=0
FILE: Number of large read operations=0
FILE: Number of write operations=0
HDFS: Number of bytes read=134
HDFS: Number of bytes written=0
HDFS: Number of read operations=5
HDFS: Number of large read operations=0
HDFS: Number of write operations=1
HDFS: Number of bytes read erasure-coded=0
Map-Reduce Framework
Map input records=6
Map output records=0
Map output bytes=56
Map output materialized bytes=74
Input split bytes=101
Combine input records=0
Spilled Records=6
Failed Shuffles=0
Merged Map outputs=0
GC time elapsed (ms)=112
Total committed heap usage (bytes)=549453824
File Input Format Counters
Bytes Read=134
2026-04-21 13:59:07,840 INFO mapred.LocalJobRunner: Finishing task: attempt_local1633189946_0001_m_000000_0
2026-04-21 13:59:07,841 INFO mapred.LocalJobRunner: map task executor complete.
2026-04-21 13:59:07,844 INFO mapred.LocalJobRunner: Waiting for reduce tasks
2026-04-21 13:59:07,844 INFO mapred.LocalJobRunner: Starting task: attempt_local1633189946_0001_r_000000_0
2026-04-21 13:59:07,853 INFO output.PathOutputCommitterFactory: No output committer factory defined, defaulting to FileOutputCommitterFactory
2026-04-21 13:59:07,853 INFO output.FileOutputCommitter: File Output Committer Algorithm version is 2
2026-04-21 13:59:07,853 INFO output.FileOutputCommitter: FileOutputCommitter skip cleanup _temporary folders under output directory:false, ignore cleanup failures: false
2026-04-21 13:59:07,854 INFO mapred.Task: Using ResourceCalculatorProcessTree : [ ]
2026-04-21 13:59:07,857 INFO mapred.ReduceTask: Using ShuffleConsumerPlugin: org.apache.hadoop.mapreduce.task.reduce.Shuffle@71479053
2026-04-21 13:59:07,859 WARN impl.MetricsSystemImpl: JobTracker metrics system already initialized!
2026-04-21 13:59:07,879 INFO reduce.MergeManagerImpl: MergerManager: memoryLimit=2589196288, maxSingleShuffleLimit=647299072, mergeThreshold=1708869632, ioSortFactor=10, memToMemMergeOutputsThreshold=10
2026-04-21 13:59:07,882 INFO reduce.EventFetcher: attempt_local1633189946_0001_r_000000_0 Thread started: EventFetcher for fetching Map Completion Events


Activities
Terminal
Apr 21 13:59
lab5@lab5-Latitude-7490: ~/Desktop/Hadoop_12
Total committed heap usage (bytes)=549453824
File Input Format Counters
Bytes Read=134
2026-04-21 13:59:07,840 INFO mapred.LocalJobRunner: Finishing task: attempt_local1633189946_0001_m_000000_0
2026-04-21 13:59:07,841 INFO mapred.LocalJobRunner: map task executor complete.
2026-04-21 13:59:07,844 INFO mapred.LocalJobRunner: Waiting for reduce tasks
2026-04-21 13:59:07,844 INFO mapred.LocalJobRunner: Starting task: attempt_local1633189946_0001_r_000000_0
2026-04-21 13:59:07,853 INFO output.PathOutputCommitterFactory: No output committer factory defined, defaulting to FileOutputCommitterFactory
2026-04-21 13:59:07,853 INFO output.FileOutputCommitter: File Output Committer Algorithm version is 2
2026-04-21 13:59:07,853 INFO output.FileOutputCommitter: FileOutputCommitter skip cleanup _temporary folders under output directory:false, ignore cleanup failures: false
2026-04-21 13:59:07,854 INFO mapred.Task: Using ResourceCalculatorProcessTree : [ ]
2026-04-21 13:59:07,857 INFO mapred.ReduceTask: Using ShuffleConsumerPlugin: org.apache.hadoop.mapreduce.task.reduce.Shuffle@71479053
2026-04-21 13:59:07,859 WARN impl.MetricsSystemImpl: JobTracker metrics system already initialized!
2026-04-21 13:59:07,879 INFO reduce.MergeManagerImpl: MergerManager: memoryLimit=2589196288, maxSingleShuffleLimit=647299072, mergeThreshold=1708869632, ioSortFactor=10, memToMemMergeOutputsThreshold=10
2026-04-21 13:59:07,882 INFO reduce.EventFetcher: attempt_local1633189946_0001_r_000000_0 Thread started: EventFetcher for fetching Map Completion Events
2026-04-21 13:59:07,917 INFO reduce.LocalFetcher: localfetcher#1 about to shuffle output of map attempt_local1633189946_0001_m_000000_0 decomp: 70 len: 74 to MEMORY
2026-04-21 13:59:07,920 INFO reduce.InMemoryMapOutput: Read 70 bytes from map-output for attempt_local1633189946_0001_m_000000_0
2026-04-21 13:59:07,922 INFO reduce.MergeManagerImpl: closeInMemoryFile -> map-output of size: 70, inMemoryMapOutputs.size() -> 1, commitMemory -> 0, usedMemory ->70
2026-04-21 13:59:07,924 INFO reduce.EventFetcher: EventFetcher is interrupted.. Returning
2026-04-21 13:59:07,924 INFO mapred.LocalJobRunner: 1 / 1 copied.
2026-04-21 13:59:07,925 INFO reduce.MergeManagerImpl: finalMerge called with 1 in-memory map-outputs and 0 on-disk map-outputs
2026-04-21 13:59:07,931 INFO mapred.Merger: Merging 1 sorted segments
2026-04-21 13:59:07,932 INFO mapred.Merger: Down to the last merge-pass, with 1 segments left of total size: 62 bytes
2026-04-21 13:59:07,933 INFO reduce.MergeManagerImpl: Merged 1 segments, 70 bytes to disk to satisfy reduce memory limit
2026-04-21 13:59:07,934 INFO reduce.MergeManagerImpl: Merging 1 files, 74 bytes from disk
2026-04-21 13:59:07,935 INFO reduce.MergeManagerImpl: Merging 0 segments, 0 bytes from memory into reduce
2026-04-21 13:59:07,935 INFO mapred.Merger: Merging 1 sorted segments
2026-04-21 13:59:07,936 INFO mapred.Merger: Down to the last merge-pass, with 1 segments left of total size: 62 bytes
2026-04-21 13:59:07,936 INFO mapred.LocalJobRunner: 1 / 1 copied.
2026-04-21 13:59:07,974 INFO Configuration.deprecation: mapred.skip.on is deprecated. Instead, use mapreduce.job.skiprecords
2026-04-21 13:59:08,040 INFO mapred.Task: Task:attempt_local1633189946_0001_r_000000_0 is done. And is in the process of committing
2026-04-21 13:59:08,043 INFO mapred.LocalJobRunner: 1 / 1 copied.
2026-04-21 13:59:08,044 INFO mapred.Task: Task attempt_local1633189946_0001_r_000000_0 is allowed to commit now
2026-04-21 13:59:08,059 INFO output.FileOutputCommitter: Saved output of task 'attempt_local1633189946_0001_r_000000_0' to hdfs://localhost:9000/output
2026-04-21 13:59:08,060 INFO mapred.LocalJobRunner: reduce > reduce
2026-04-21 13:59:08,060 INFO mapred.Task: Task 'attempt_local1633189946_0001_r_000000_0' done.
2026-04-21 13:59:08,061 INFO mapred.Task: Final Counters for attempt_local1633189946_0001_r_000000_0: Counters: 30
File System Counters
FILE: Number of bytes read=3442
FILE: Number of bytes written=639032
FILE: Number of read operations=0
FILE: Number of large read operations=0
FILE: Number of write operations=0
HDFS: Number of bytes read=134
HDFS: Number of bytes written=22
HDFS: Number of read operations=10
HDFS: Number of large read operations=0
HDFS: Number of write operations=3
HDFS: Number of bytes read erasure-coded=0
Map-Reduce Framework


Activities
Terminal
Apr 21 13:59
lab5@lab5-Latitude-7490: ~/Desktop/Hadoop_12
2026-04-21 13:59:07,934 INFO reduce.MergeManagerImpl: Merging 1 files, 74 bytes from disk
2026-04-21 13:59:07,935 INFO reduce.MergeManagerImpl: Merging 0 segments, 0 bytes from memory into reduce
2026-04-21 13:59:07,935 INFO mapred.Merger: Merging 1 sorted segments
2026-04-21 13:59:07,936 INFO mapred.Merger: Down to the last merge-pass, with 1 segments left of total size: 62 bytes
2026-04-21 13:59:07,936 INFO mapred.LocalJobRunner: 1 / 1 copied.
2026-04-21 13:59:07,974 INFO Configuration.deprecation: mapred.skip.on is deprecated. Instead, use mapreduce.job.skiprecords
2026-04-21 13:59:08,040 INFO mapred.Task: Task:attempt_local1633189946_0001_r_000000_0 is done. And is in the process of committing
2026-04-21 13:59:08,043 INFO mapred.LocalJobRunner: 1 / 1 copied.
2026-04-21 13:59:08,044 INFO mapred.Task: Task attempt_local1633189946_0001_r_000000_0 is allowed to commit now
2026-04-21 13:59:08,059 INFO output.FileOutputCommitter: Saved output of task 'attempt_local1633189946_0001_r_000000_0' to hdfs://localhost:9000/output
2026-04-21 13:59:08,060 INFO mapred.LocalJobRunner: reduce > reduce
2026-04-21 13:59:08,060 INFO mapred.Task: Task 'attempt_local1633189946_0001_r_000000_0' done.
2026-04-21 13:59:08,061 INFO mapred.Task: Final Counters for attempt_local1633189946_0001_r_000000_0: Counters: 30
File System Counters
FILE: Number of bytes read=3442
FILE: Number of bytes written=639032
FILE: Number of read operations=0
FILE: Number of large read operations=0
FILE: Number of write operations=0
HDFS: Number of bytes read=134
HDFS: Number of bytes written=22
HDFS: Number of read operations=10
HDFS: Number of large read operations=0
HDFS: Number of write operations=3
HDFS: Number of bytes read erasure-coded=0
Map-Reduce Framework
Combine input records=0
Combine output records=0
Reduce input groups=3
Reduce shuffle bytes=74
Reduce input records=6
Reduce output records=3
Spilled Records=6
Shuffled Maps =1
Failed Shuffles=0
Merged Map outputs=1
GC time elapsed (ms)=8
Total committed heap usage (bytes)=549453824
Shuffle Errors
BAD_ID=0
CONNECTION=0
IO_ERROR=0
WRONG_LENGTH=0
WRONG_MAP=0
WRONG_REDUCE=0
File Output Format Counters
Bytes Written=22
2026-04-21 13:59:08,061 INFO mapred.LocalJobRunner: Finishing task: attempt_local1633189946_0001_r_000000_0
2026-04-21 13:59:08,061 INFO mapred.LocalJobRunner: reduce task executor complete.
2026-04-21 13:59:08,418 INFO mapreduce.Job: Job job_local1633189946_0001 running in uber mode : false
2026-04-21 13:59:08,420 INFO mapreduce.Job: map 100% reduce 100%


Activities
Terminal
Apr 21 13:59
lab5@lab5-Latitude-7490: ~/Desktop/Hadoop_12
GC time elapsed (ms)=0
Total committed heap usage (bytes)=549453824
Shuffle Errors
BAD_ID=0
CONNECTION=0
IO_ERROR=0
WRONG_LENGTH=0
WRONG_MAP=0
WRONG_REDUCE=0
File Output Format Counters
Bytes Written=22
2026-04-21 13:59:08,061 INFO mapred.LocalJobRunner: Finishing task: attempt_local1633189946_0001_r_000000_0
2026-04-21 13:59:08,061 INFO mapred.LocalJobRunner: reduce task executor complete.
2026-04-21 13:59:08,418 INFO mapreduce.Job: Job job_local1633189946_0001 running in uber mode : false
2026-04-21 13:59:08,420 INFO mapreduce.Job: map 100% reduce 100%
2026-04-21 13:59:08,423 INFO mapreduce.Job: Job job_local1633189946_0001 completed successfully
2026-04-21 13:59:08,442 INFO mapreduce.Job: Counters: 36
File System Counters
FILE: Number of bytes read=6704
FILE: Number of bytes written=1277990
FILE: Number of read operations=0
FILE: Number of large read operations=0
FILE: Number of write operations=0
HDFS: Number of bytes read=268
HDFS: Number of bytes written=22
HDFS: Number of read operations=15
HDFS: Number of large read operations=0
HDFS: Number of write operations=4
HDFS: Number of bytes read erasure-coded=0
Map-Reduce Framework
Map input records=6
Map output records=6
Map output bytes=56
Map output materialized bytes=74
Input split bytes=101
Combine input records=0
Combine output records=0
Reduce input groups=3
Reduce shuffle bytes=74
Reduce input records=6
Reduce output records=3
Spilled Records=12
Shuffled Maps =1
Failed Shuffles=0
Merged Map outputs=1
GC time elapsed (ms)=112
Total committed heap usage (bytes)=1098907648
Shuffle Errors
BAD_ID=0
CONNECTION=0
IO_ERROR=0


Activities
Terminal
Apr 21 14:00
lab5@lab5-Latitude-7490: ~/Desktop/Hadoop_12
2026-04-21 13:59:08,061 INFO mapred.LocalJobRunner: reduce task executor complete.
2026-04-21 13:59:08,418 INFO mapreduce.Job: Job job_local1633189946_0001 running in uber mode : false
2026-04-21 13:59:08,420 INFO mapreduce.Job: map 100% reduce 100%
2026-04-21 13:59:08,423 INFO mapreduce.Job: Job job_local1633189946_0001 completed successfully
2026-04-21 13:59:08,442 INFO mapreduce.Job: Counters: 36
File System Counters
FILE: Number of bytes read=6704
FILE: Number of bytes written=1277990
FILE: Number of read operations=0
FILE: Number of large read operations=0
FILE: Number of write operations=0
HDFS: Number of bytes read=268
HDFS: Number of bytes written=22
HDFS: Number of read operations=15
HDFS: Number of large read operations=0
HDFS: Number of write operations=4
HDFS: Number of bytes read erasure-coded=0
Map-Reduce Framework
Map input records=6
Map output records=6
Map output bytes=56
Map output materialized bytes=74
Input split bytes=101
Combine input records=0
Combine output records=0
Reduce input groups=3
Reduce shuffle bytes=74
Reduce input records=6
Reduce output records=3
Spilled Records=12
Shuffled Maps =1
Failed Shuffles=0
Merged Map outputs=1
GC time elapsed (ms)=112
Total committed heap usage (bytes)=1098907648
Shuffle Errors
BAD_ID=0
CONNECTION=0
IO_ERROR=0
WRONG_LENGTH=0
WRONG_MAP=0
WRONG_REDUCE=0
File Input Format Counters
Bytes Read=134
File Output Format Counters
Bytes Written=22
lab5@lab5-Latitude-7490:~/Desktop/Hadoop_12$ hdfs dfs -cat /output/part-r-00000
ERROR 2
INFO 3
WARN 1
lab5@lab5-Latitude-7490:~/Desktop/Hadoop_12$

Practical No. 13

input.txt:

hello spark

hello scala

hello hadoop

apache spark hello

OUTPUT: